Parsek — Rewind to Separation

Post-implementation design specification for the mid-mission rewind system. Covers Rewind Points captured at multi-controllable vessel separations (staging, decoupling, undocking; never an EVA, see §1.5), the Unfinished Flights virtual group, the append-only supersede relation, broad reviewed-career tombstone scope, the journaled staged commit, and the load-time sweep that keeps half-finished state bounded — together with the v0.9.1 stable-leaf extension that broadens the Unfinished Flights membership predicate to include orbiting / sub-orbital non-focus siblings, adds per-slot Seal and Stash actions, and tightens Re-Fly merge classification with a focus-override auto-seal contract.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to an immutable timeline, and see previously recorded missions play back as ghost vessels alongside new ones. This document extends the flight recorder, timeline, and ledger systems with Rewind-to-Separation. It assumes familiarity with the recording DAG, BranchPoint model, controller identity, ghost chains, and the additive-only invariant (see `parsek-flight-recorder-design.md`) and with the ledger model, immutable ActionId, reservations, and career-state replay (see `parsek-game-actions-and-resources-recorder-design.md`).

Status: shipped. Core feature in v0.9.0; stable-leaf extension (broadened Unfinished Flights predicate, per-slot Seal / Stash actions, Re-Fly merge auto-seal contract, invocation linearization) in v0.9.1. Kept current through v0.10.5, including the 2026-09-27 rulings that Re-Fly is for vessel separations only (section 1.5) and that a slot follows its vessel through its own crew's EVA / Board (section 1.6), and the Rewind-to-Launch interaction rules in section 8. **Pre-implementation specs (archived):**

- `docs/dev/done/parsek-rewind-separation-design.md` — pre-v0.9.0 spec, archived as-is alongside the v0.9 rollout.
- `docs/dev/done/parsek-unfinished-flights-stable-leaves-design.md` — pre-v0.9.1 spec for the stable-leaf extension, archived alongside the v0.9.1 rollout. Promoted from research note `docs/dev/research/extending-rewind-to-stable-leaves.md` (R17, merged in PR #634).

This document supersedes both pre-impl specs as the source of truth for what shipped. **Related docs:** `parsek-flight-recorder-design.md`, `parsek-recording-finalization-design.md`, `parsek-timeline-design.md`, `parsek-game-actions-and-resources-recorder-design.md`, `parsek-logistics-supply-routes-design.md`.

---

1. Introduction

A Parsek career is a committed timeline of missions. The player flies, commits, the ghost plays back, and the next mission begins on top. Before v0.9 the timeline had a sharp edge: once a multi-controllable split happened — a stage decoupled with a probe core on each side, a lander undocked from a station — whichever half the player did not personally fly was recorded in the background and, if things went badly, committed as a crashed or destroyed sibling. There was no in-game path back to re-fly that half. The successful side was locked in with the failed side, and the only "fix" was to discard the entire tree and re-fly the whole mission.

Rewind to Separation is the feature that unsticks that edge. At every vessel separation that produces two or more controllable vessels, Parsek writes a transient KSP quicksave - a **Rewind Point** - plus a compact persistent-id-to-slot table captured at save time. If any sibling ends as an **unfinished flight** - a crashed booster, a destroyed lander, a probe abandoned in orbit, an upper stage left coasting sub-orbital - the recording appears in a read-only **Unfinished Flights** group in the Recordings tab. Clicking Fly on an Unfinished-Flight row, either in that virtual group or on the same recording's normal table row, reloads the quicksave, strips the non-selected siblings (they play back as ghosts from their committed recordings), and hands the player the other half at the exact split moment. When the re-fly ends and the player merges, the new recording supersedes the old one via an **append-only relation** - the original recording is never mutated or deleted, the ghost/claim subsystems just filter it out.

Two membership tiers, one feature. The v0.9.0 release shipped the narrow "rescue from destruction" predicate: only `TerminalKind.Crashed` siblings of multi-controllable splits qualified. The v0.9.1 stable-leaf extension broadens the predicate to also include orbiting / sub-orbital non-focus siblings (the 4-probe deploy case, the upper-stage-left-coasting case), while preserving the focus-continuation exclusion so a routine upper stage that the player flew to orbit does not pollute the list. A per-row **Seal** button lets the player close any slot permanently when they have decided the recording is canonical, and a per-row **Stash** button lets the player promote a default-excluded stable leaf into Unfinished Flights when they want it back later. Re-Fly merges of player-chosen slots that reach a stable terminal auto-seal the slot — the player explicitly engaged the slot to fly it, and the merge concludes that engagement. Terminal-failure outcomes (Destroyed) that earned nothing stay open so the player can retry the failure. Since the 2026-10-19 ledger-safety rulings (section 2.12) a flight that holds any gain, contract outcome or flight-tagged spend after its separation is never offered for a Re-Fly, so a merge can only raise the career's balances. Crashed sibling rows from v0.9.0 keep their behaviour otherwise.

1.1 Scope

The shipped feature covers:

v0.9.0 core (rescue from destruction):

- **Split detection** at joint-break and undock boundaries. `SegmentBoundaryLogic.IsMultiControllableSplit` (`Source/Parsek/SegmentBoundaryLogic.cs`) is the gate; debris-only splits and single-controllable splits do not receive a Rewind Point, and neither does an EVA split (`RewindPointAuthor.IsReFlySplitType`, §1.5).
- **Rewind Point capture.** `RewindPointAuthor.Begin` (`Source/Parsek/RewindPointAuthor.cs:52`) synchronously attaches the RP to its `BranchPoint` and to `ParsekScenario.RewindPoints`, then a one-frame-deferred coroutine drops warp to zero, populates both PID-to-slot maps from live vessels, writes a stock KSP save, and atomically moves the result into `saves/<save>/Parsek/RewindPoints/<rpId>.sfs`. The author also captures `RewindPoint.FocusSlotIndex` (the slot index of the active vessel at split time, or `-1` when no live slot matches the active focus) for the v0.9.1 predicate to consume.
- **Unfinished Flights UI group.** A virtual, read-only group in the Recordings tab (`Source/Parsek/UI/UnfinishedFlightsGroup.cs`) computed per-frame from ERS filtered by `EffectiveState.IsUnfinishedFlight`. Membership updates automatically as flights are flown, sealed, stashed, and merged.
- **Invocation.** `RewindInvoker` (`Source/Parsek/RewindInvoker.cs`) runs a five-precondition gate, captures a pre-load reconciliation bundle, copies the RP quicksave to the save-root (KSP's `LoadGame` does not accept subdirectory paths), triggers `GamePersistence.LoadGame` + `HighLogic.LoadScene(FLIGHT)`, then on the reloaded scene atomically runs Restore → Strip → Activate → provisional + `ReFlySessionMarker` write.
- **Append-only supersede.** On merge, `SupersedeCommit.AppendRelations` (`Source/Parsek/SupersedeCommit.cs:108`) appends one `RecordingSupersedeRelation` per recording in the forward-only merge-guarded subtree closure of the retired sibling. No field on a committed Recording is mutated post-commit.
- **Broad reviewed-career tombstone scope, value-gated.** Merge tombstones retire reviewed, non-seed, recording-scoped career actions from the superseded subtree, including contracts, milestones, science, funds/reputation, facilities, strategies, and kerbal consequences. Null-scoped KSC/system rows, initial seeds, already-paid rollout costs, and unknown future action types are preserved until reviewed. Tombstone refreshes immediately recalculate and patch KSP state so removed science, contract, facility, and roster consequences are actively reconciled where safe. Since the 2026-10-19 rulings (section 2.12) a slot whose retire set holds anything but losses (crew deaths, crew bookkeeping, vessel-loss reputation, building damage) cannot be re-flown, so in practice the tombstones retire only those losses, and a pre-merge dominance check refuses any merge that would still lower a balance.
- **Crashed re-fly stays rewindable.** `TerminalKindClassifier.Classify` (`Source/Parsek/TerminalKindClassifier.cs`) maps the provisional's terminal state to `Landed` (commits `Immutable`), `Crashed` (commits `CommittedProvisional` so the slot stays an Unfinished Flight), or `InFlight` (same as Landed). The v0.9.1 Re-Fly auto-seal contract (§4.9) tightens this for player-chosen slots reaching stable terminals.
- **Journaled staged commit.** `MergeJournalOrchestrator.RunMerge` (`Source/Parsek/MergeJournalOrchestrator.cs:149`) drives the merge through eleven phase checkpoints, all reflected in `MergeJournal.Phase` (`Source/Parsek/MergeJournal.cs:53`). A load-time finisher drives every phase to completion via idempotent re-run, `Begin` included: the attempt's tree is already committed when the journal is first written, so no phase on disk is a pre-merge snapshot to roll back to. The `Begin` save runs after the in-place-continuation fork migrates into the committed tree (`TreeMerge`), and `Split` (cuts the origin recording at the rewind UT) carries its own DurableSave barrier.
- **Load-time sweep.** `LoadTimeSweep.Run` (`Source/Parsek/LoadTimeSweep.cs:51`) validates the re-fly marker's six durable fields via `MarkerValidator.Validate` (`Source/Parsek/MarkerValidator.cs:86`), discards zombie NotCommitted provisionals and session-scoped RPs not referenced by a valid marker, warn-logs orphan supersede/tombstone rows, and clears stray `SupersedeTargetId` fields. Normal staging RPs with `CreatingSessionId == null` are retained across merge-dialog scene loads.
- **Revert-during-re-fly dialog.** `RevertInterceptor` (`Source/Parsek/RevertInterceptor.cs`) prefixes `FlightDriver.RevertToLaunch` via Harmony and routes the player to `ReFlyRevertDialog` (`Source/Parsek/ReFlyRevertDialog.cs`) with three options: Retry from Rewind Point, Discard Re-fly, Continue Flying.
- **Difficulty flags (owner ruling S7, 2026-09-26).** Rewind and Re-Fly ignore `Flight.CanQuickLoad` / `Flight.CanRestart`: no Parsek path reads them. Stock builds the Esc-menu Revert Flight button, its Revert to Launch entry and the flight-results Revert to Launch button only when `Flight.CanRestart` is true, so on the Hard preset the re-fly Retry choice (this dialog) is not offered (owner decision 2026-09-26; Parsek does not flip the flag). Merge and Discard stay reachable through scene exit (`SceneExitInterceptor`'s Re-Fly merge dialog). `ReFlyRevertButtonGate.Apply` logs an Info line naming the case.

v0.9.1 stable-leaf extension:

- **Broadened predicate.** `EffectiveState.IsUnfinishedFlight` is rewritten to include stable-terminal non-focus controllable leaves (`Orbiting`, `SubOrbital`) alongside the v0.9.0 `Destroyed` qualification. (v0.9.1 also admitted stranded EVA kerbals; that branch was removed by the 2026-09-27 ruling, §1.5: an EVA kerbal never qualifies.) The structural-leaf gate, controllable-subject gate,

and slot-still-open gate are all preserved or tightened. Focus-slot continuation siblings stay excluded so routine upper stages do not pollute the list.

- **One Unfinished Flights group, broader membership.** No new virtual group. The existing tooltip and group affordances stay; the predicate behind it changes.
- **Per-row Seal action.** A new in-table action that permanently closes a slot by flipping its effective chain+supersede tip recording from `CommittedProvisional` to `Immutable`, so the player can clear over-included rows and let the reaper free disk space. Open/closed is read solely from the tip `MergeState`, the single source of truth (the old separate `ChildSlot.Sealed` bit was collapsed into `MergeState`). See §6.24.
- **Per-row Stash action.** A complementary in-table action that promotes a default-excluded stable terminal leaf into Unfinished Flights while the backing Rewind Point still exists. Manual Stash is conservative: only spawnable stable terminals (`Landed`, `Splashed`, `Orbiting`, `SubOrbital`) qualify; `Recovered`, `Docked`, `Boarded`, and recordings whose retire set holds a sealing row (section 2.12; before 2026-10-19 only a credited `ScienceEarning`) are not stashable.
- **Re-Fly merge auto-seal (§4.9).** When the player Re-Flies a slot and the merge commits the chain tip at a stable terminal (`Orbiting`, `Landed`, `Splashed`, plus the existing hard-safety set `Recovered` / `Docked` / `Boarded`), or the session authors a structural mutation (decouple / stage / undock / joint break / EVA), or the re-fly filed a sealing row (any gain, contract outcome or flight-tagged spend, section 2.12), the slot seals (its tip commits `Immutable`). Terminal-failure outcomes (Destroyed) that filed no sealing row stay `CommittedProvisional` so the player can retry the failure. `SubOrbital` is NOT auto-sealed: a suborbital arc is still in flight (the vessel will crash, land, splash, or with a burn reach orbit), so the slot stays open and the recording commits as `CommittedProvisional`. The player can manually Stash + Seal a SubOrbital row if they want to close the engagement, or Re-Fly the slot again to let the arc resolve.
- **New persistent fields.** `ChildSlot.Stashed` (+ `StashedRealTime`), `RewindPoint.FocusSlotIndex`, `ReFlySessionMarker.SupersedeTargetId`, and `ReFlySessionMarker.PreSessionBranchPointIds`. (The original `ChildSlot.Sealed` / `SealedRealTime` fields were later removed when Seal collapsed into `MergeState`; see §6.24.) Legacy ConfigNodes load with safe defaults.
- **Helper extraction.** `TryResolveRewindPointForRecording` and friends move from `UI/RecordingsTableUI.cs` into a new `UnfinishedFlightClassifier` static class so non-UI consumers (`RecordingStore`, `SupersedeCommit`) can call them without a layering inversion.
- **Closure-helper split + invocation linearization.** `EffectiveState.ComputeSessionSuppressedSubtree(marker)` becomes a thin wrapper around `ComputeSubtreeClosureInternal(marker, rootOverride)`. `RewindInvoker.AtomicMarkerWrite` stamps `marker.SupersedeTargetId = priorTip` so chain extension produces a linear graph instead of a star. Required prerequisite for the v0.9.1 Site B-1 merge classifier (see §6.22 and §10.10).

1.2 Out of scope

The shipped feature deliberately does NOT attempt:

- Re-emitting retired contract completions, milestone flags, tech research, facility upgrades, strategies, or the KSP subsystems that own them (`ContractSystem`, `ProgressTracking`, `ScenarioUpgradeableFacilities`, `StrategySystem`, `ResearchAndDevelopment`) without a ledger replay. These stock systems are "sticky", so broad tombstones must be followed by a recalculation and KSP patch pass rather than relying on stock events to fire again.
- Nested re-fly sessions. The precondition gate rejects a rewind invocation while another session is active (`RewindInvoker.cs:108`).
- Cross-tree supersedes. The feature does not produce them; `EffectiveState.EffectiveRecordingId` does not yet halt at cross-tree boundaries but has a TODO marker for when cross-tree supersedes become producible (see Known Limitations).
- Auto-purge policies for long-lived reap-eligible RPs. Monitoring via the disk-usage diagnostic (with a split into live-crashed / stable-open / concluded buckets) is the answer; a TTL-based reaper or "Wipe All Sealed RPs" button is deferred.
- A recovery-snapshot mechanism for corrupt quicksaves. The pre-impl `SplitTimeSnapshot` concept was dropped before v0.9.0 shipped; the KSP quicksave is the sole source of truth.
- Voluntary-action heuristics for stable-leaf surfacing (orbit-shift, mid-chain-surface, body-change classifiers). The R1-R3 exploration in the stable-leaves research note explicitly rejected this; over-inclusion of the broadened predicate is handled by the per-row Seal button instead.
- Migration sweep for legacy star-shaped supersede graphs. The v0.9.1 closure-helper split (§6.22) tolerates legacy stars and grows linearly off whichever leaf the walker reaches; the existing star portions in pre-v0.9.1 saves are not flattened.
- An in-game un-Seal path. The Seal action is permanent and intentionally one-way: it flips the tip to `Immutable`, which is never demoted back. Tree-scoped Full-Revert is the only undo. There is no Un-Seal affordance (the earlier "deferred unless playtest demands it" hedge is retired).

1.3 Who benefits

- Players who launch rockets with recoverable boosters or stages and whose booster crashed instead of landing — they want to re-fly the booster back to the pad while the upper stage's ascent to orbit continues to play back as a ghost.
- Players who lost a lander on descent while the orbiting mothership survived — they want to re-fly the lander.
- (v0.9.1) Players running constellation-deploy missions (probes, satellites, ground stations) who want to come back and fly individual deployed objects later. The 4-probe mothership is the canonical case (§4.4): the player flies the mothership home, and the four probes appear in Unfinished Flights so any of them can be flown to a real mission later.
- (v0.9.1) Players who left an upper stage in a sub-orbital arc they intended to circularize but never got around to.
- (v0.9.1) Players who want a cleanup tool: the per-row Seal button closes any UF slot permanently when the player has decided the recording is canonical, freeing the rewind point's quicksave on the next reaper sweep.

What this feature is **not** for: focus-continuation slots (your active mission's upper stage that reached orbit safely), routine dockings, and safe returns are excluded by default — they do not auto-populate Unfinished Flights. The v0.9.1 manual Stash button is the explicit escape hatch for stable terminals the player wants re-flyable later, and only while the backing Rewind Point still exists.

1.4 Relationship to prior features

Rewind to Separation is layered on top of, not inside, the existing subsystems:

- **Flight recorder:** the recording DAG, segment boundary rule, controller identity, ghost chains, background recording, terminal kinds. Rewind to Separation does not change any of these. It adds new persistent state — Rewind Points, supersede relations, tombstones, a session marker, a journal — stored alongside the existing recording tree in `ParsekScenario`.
- **Recording finalization:** Rewind-to-Separation assumes each sibling recording has a trustworthy terminal state and endpoint. The finalization reliability contract in `parsek-recording-finalization-design.md` is the upstream dependency that prevents Unfinished Flights from depending on stale last-sample inference when KSP unloads, deletes, or destroys a vessel before scene exit.
- **Timeline / ledger:** the immutable `ActionId`, the recalculation engine, the resource modules. The feature adds `GameAction.ActionId` as a hard precondition (written with every row) and introduces `LedgerTombstone` as an append-only retirement filter, but the recalculation walk itself is unchanged — `LedgerOrchestrator.Recalculate*` now feeds from `EffectiveState.ComputeELS()` (the tombstone-filtered view) instead of raw `Ledger.Actions`.
- **Game actions & resources:** contracts, milestones, facilities, strategies, tech, science, funds, kerbals. Merge tombstones now cover reviewed recording-scoped career actions, while seeds, null-scoped KSC/system rows, rollout costs, and unknown future types remain preserved. The post-tombstone recalc/patch path owns live KSP reconciliation for removed consequences; see §2 and §10.

1.5 Separations only: an EVA is not a rewind split (owner ruling 2026-09-27)

> Re-Fly is for vessel separations only (staging, decoupling, undocking). An EVA is not a separation event and never gets a Re-Fly. We are not responsible for every player mistake: they can F9 or Discard if something happens. The point of Re-Fly was to be able to return-land boosters etc., not to prevent mistakes.

What that means in code:

- **No Rewind Point at an EVA split.** An EVA is still a split in the recording DAG (an `EVA` BranchPoint with the vessel continuation and the kerbal as children, `parsek-flight-recorder-design.md` section 4.6), but `RewindPointAuthor.Begin` refuses it before anything is written (`RewindPointAuthor.IsReFlySplitType`; every authoring path funnels through `Begin`, so this is the one gate) and logs `[Rewind] Split type=EVA: no Rewind Point (Re-Fly is for vessel separations only) bp=<id>`. Undock, JointBreak and Breakup authoring is unchanged.
- **Leaving an external seat is an EVA, not an undock (owner ruling 2026-10-24).** V: "a kerbal leaving an external seat is an EVA, as we get a vehicle and a kerbal; undock is between vehicles, as we get two vehicles at the end". A seat dismount and a claw releasing a kerbal reach Parsek through stock's `Part.Undock` with no EVA event, so `ParsekFlight.OnVesselsUndocking` checks the shape first: a split that leaves a vehicle and an EVA kerbal is recorded as an EVA branch (the kerbal's name on the child, no Rewind Point, never a Re-Fly slot), and only a split that leaves two vehicles is an Undock. Taking a seat stays a Dock merge that stamps the seat's vessel. Lane SEAT-1 (todo GC11-SEAT-DISMOUNT-RECORDED-AS-UNDOCK).
- **An EVA kerbal is never an Unfinished Flight.** `UnfinishedFlightClassifier` rejects a chain tip or subject carrying `EvaCrewName` with `evaNotSeparation`, whatever its terminal (`Destroyed` included), before the terminal is read; the candidate-shape and stash-shape helpers refuse the same recordings. A slot of a legacy RP authored at an EVA BranchPoint is refused with the same reason on the VESSEL side too: rewinding to the EVA moment to fly the vessel is still an EVA Re-Fly.

- **Legacy EVA Rewind Points get no load-time reap.** An RP an older build authored at an EVA BranchPoint loads best effort (1.0 save-format ruling, 2026-11-04): it never surfaces in UF (`evaNotSeparation`) and is subject only to the ordinary reaper.
- **A separation slot follows its own vessel through the crew's EVA** (section 1.6): an EVA from the separated vessel no longer ends that slot's flight; a crew member who steps out and climbs back in is part of the vessel's history, and re-flying the vessel from its separation rewrites that EVA.
- **Unchanged:** an EVA taken DURING a Re-Fly of a vessel is still a structural mutation that auto-seals the slot on merge (section 4.9). A kerbal who goes EVA from the re-flown vessel during the session keeps its own recording: the session's own new recordings are never supersede sources, so the merge closure does not reach them (section 6.3; enforced by `EffectiveState.CollectActiveSessionOwnedRecordingIds`, section 6.12 step 4). How such a session merges is section 6.12's "A session that went EVA".

1.6 A separation slot follows its own vessel through the crew's EVA (owner ruling 2026-09-27)

> Re-Fly is available from the point of separation, respecting the sealing rules defined in the design docs.

section 1.5 removed the EVA Rewind Point that used to cover a crewed vessel whose flight continued after an EVA. Before this section the separation slot's walk stopped at the vessel's first EVA branch point, so a stage that staged, put Jeb out for an EVA report, took him back and then crashed was refused `downstreamBp`, while the same crash with no EVA was offered. The owner decisions of 2026-09-27:

1. **The walk.** `EffectiveState.WalkSlotVessel` (behind `ResolveTerminalRecordingAcrossSwitchContinuations`, and therefore behind `EffectiveTipRecordingId`, `ChildSlot.EffectiveRecordingId`, the reaper, Seal, Stash, `IsVisibleUnfinishedFlight`, Site A / B-1 / B-2, RP slot resolution and `LoadTimeSweep`) follows, in addition to chain hops and `VesselSwitchContinuation` points: an `EVA` branch point the vessel authored, to the child with the vessel's own `VesselPersistentId` and no `EvaCrewName`; and a `Board` branch point whose merged child carries the vessel's pid and whose every other parent is one of the vessel's OWN EVA crew recordings (the kerbal's recording, its chain siblings and its switch-continuation segments, registered when the walk crossed the EVA). A walk that starts mid-stretch (the vessel continuation after an EVA is a slot member too) first registers the crew of every own EVA behind its start, so it reaches the same tip as the walk from the origin. The chain tip, terminal, stable-leaf / focus rules and the rule-4 downstream check are then read at the vessel's real tip. Every other branch point still stops the walk: `Dock`, `Undock`, `JointBreak`, `Breakup`, and a `Board` by a kerbal from ANY other vessel (`boardForeignParent`: the vessel merged with a foreign vessel's crew, the same kind of world interaction as a dock). Every hop logs `[Supersede] SwitchContinuationWalk: hop ... kind=switch|eva|board`; a stop logs its reason (`evaNoVesselContinuation`, `boardNotSameVessel`, `boardForeignParent`, `notSwitchBranchPoint`, ...). The one reader that keeps the switch-only walk is the map-presence chain-tip segment borrow in `GhostMapPresence`, a rendering read that is not a slot question.
2. **A kerbal who left for another vessel blocks the slot.** When a kerbal who went EVA from the vessel after the separation did NOT come back to it - his history ends in a `Board` or `Dock` the vessel walk did not pass through - the slot is not re-flyable: `UnfinishedFlightClassifier` rejects it with the grep-stable reason `evaCrewJoinedForeignVessel` (logged with `kerbalRec=`, `joinBp=`, `joinType=`), right after the `downstreamBp` check and before the terminal is read, and Site B-1 auto-seals on it like `downstreamBp`. A kerbal left standing on EVA at the end, or who died on the EVA, does not block. With several EVAs, one kerbal who left for a foreign vessel is enough to block the slot.
3. **The re-fly rewrites the EVA.** The kerbal was aboard at the rewind point, so the Re-Fly closure (section 6.3) admits the vessel's own EVA crew recordings hanging off its EVA branch points after the rewind UT, everything they produced (their `GroundPartPlaced` members, found by parent id; their flags live on the kerbal recording), and the vessel segment after a same-vessel Board. At merge time the closure root is the walked tip, so the closure is seeded with the own-vessel stretch walked BACKWARD from the root to the slot origin (stopping before any recording whose sampled content starts before the rewind UT, and at a re-fly fork). When that stretch starts with the tree-branching parent that spans the rewind point (section 4.8), `RecordingTreeSplitter` splits THAT recording (not only the marker's `SupersedeTargetId`) and the forward closure from its TIP reaches the whole walked history. A kerbal DEATH on such an EVA is undone by the re-fly: its `KerbalAssignment` +Dead row (and bundled death penalty) is tagged to the kerbal's EVA recording, which is in the supersede write-set, so the ordinary tombstone path (section 6.13) retires it exactly like a crash death. The mixed-parent halt stays for anything that merges with a foreign vessel (decision 2 makes that slot non-re-flyable anyway).
4. **Sealing rules across the walked history.** A sealing row (section 2.12: any gain, contract outcome or flight-tagged spend) filed on any recording of the walked stretch - the vessel's later segments, the EVA crew's recordings, the parts they placed - seals the slot (the retire set walks the full merge closure of the slot's tip, `SupersedeCommit.ComputeRetireClosure`, plus the forward closure of the spanning stretch ancestor the merge's splitter would cut, `RecordingTreeSplitter.FindSpanningOwnVesselStretchAncestor`, whose rows after the rewind point move onto its TIP and are retired), the same as one filed aboard. Before 2026-10-19 only credited science (`ScienceEarning`) sealed and automatic consequence rows stayed non-blocking; only loss rows stay non-blocking now. Every member of the stretch reaches the same verdict as the origin: the classifier walks from the stretch HEAD (`UnfinishedFlightClassifier.ResolveSlotStretchHead`) and the value gate reads the retire set of the slot's tip, which is the same for every member. An EVA taken during a Re-Fly still auto-seals on merge (section 4.9); the hard-safety terminals `Recovered` / `Docked` / `Boarded` are read on the vessel's real tip.

5. **The Kerbals window's Lost hover** ("If that mission has a rewind point, re-flying it can undo the loss.") and the stock-screen Lost explanation name that way back only when an open Unfinished Flight exists whose re-fly would actually reach the loss: the kerbal's death recording lies in the stretch the re-fly of an OPEN slot (tip `CommittedProvisional`) rewrites - he died aboard that slot's vessel, or on an EVA from it (`EffectiveState.ComputeOpenSlotReFlyReachRecordingIds`, pure predicate `KerbalsPresentation.ShouldOfferLostReFlyRemedy`). Otherwise the sentence is omitted.

Rewind and strip need nothing special: at the rewind point the kerbal is aboard the vessel in the quicksave, the strip keeps the selected vessel with its crew, and the kerbal's old EVA recording is session-suppressed, so it plays no ghost during the re-fly.

A kerbal-free probe slot (no EVA possible) walks exactly as before. A kerbal who re-boards the vessel AFTER it docked with something cannot be reached: the walk already stopped at the `Dock` (`downstreamBp`).

1.7 Debris-only staging and a switch-away do not end a slot (owner ruling 2026-09-28)

The walk still stops at every `JointBreak` / `Breakup` / `Launch` branch point, but `UnfinishedFlightClassifier.HasBlockingDownstreamBranch` no longer reads each of them as `downstreamBp`. Two branch points leave nothing re-flyable downstream and do not block:

1. A `Breakup` / `JointBreak` with at least one recorded child, ALL of them debris with another pid, and no Rewind Point of its own: a spent stage dropped by a vessel that kept flying (the crash coalescer's breakup-continuous shape, where the parent keeps recording past the split). A branch with no recorded child, or a child missing from the tree, still blocks.
2. A post-switch `Launch` branch point: the stamp the recorder leaves on a vessel the player switched away from while idle.

Every branch point the walked TIP is a parent of, at or after the Rewind Point's own branch point, is checked by parent id, not only the tip's `ChildBranchPointId`: a later staging overwrites that link, so an earlier controllable split on the tip segment (a probe core dropped before the debris) still blocks. An earlier segment of the walk is not re-scanned (CHILD-BRANCH-SINGLE-SLOT-OVERWRITE). A controllable child, a same-vessel child, any other branch type, a resolved downstream Rewind Point, or an unresolvable branch point still blocks, as before. A non-blocking downstream branch logs `[UnfinishedFlights] ... downstream bp=<id> ignored`; a blocking one names itself as `blockingBp=` on the `downstreamBp` verdict.

The ruling covers the Re-Fly session too: at merge, Site B-1 reads the same verdict (no `classifierClosed:downstreamBp`), and the structural-mutation gate (`SupersedeCommit.HasReFlySessionStructuralMutation`, section 4.9) skips a debris-only session staging through the same `UnfinishedFlightClassifier.IsNonBlockingDownstreamBranch` test (Verbose `[Supersede] StructuralMutation scan: ... ignored N debris-only staging branch point(s)`). A Re-Fly that only drops debris therefore does not seal its slot; one that drops a probe core or crew, goes EVA or undocks still does.

2. Design Philosophy

These principles governed every design and implementation decision. They are listed up front because they inform every section that follows.

2.1 Correct visually, minimal, efficient

Borrowed from the project-wide recording-design principle. A Rewind Point is 1–2 MB of quicksave. Many splits do not end in regret. Therefore: RP creation is cheap, deferred, and speculative; reap is eager (as soon as no slot can be re-flown, the file is deleted); cached ERS/ELS views rebuild only when source state changes; supersede filtering is a derived lookup, not a stored field. The v0.9.1 Unfinished Flight predicate runs per-frame on every committed recording in ERS; it is structured as cheap structural gates first, expensive terminal-state-and-focus checks last, with single-line shortcut returns. The closure helper is cached.

2.2 Append-only history; slot-level close signal

The recording tree never shrinks. Supersede is a relation stored in a separate list, not a mutation on a recording. Tombstones are append-only. Once a committed Recording's `MergeState` leaves `NotCommitted`, the only field that subsequently changes is `MergeState` itself: it transitions between `CommittedProvisional` (slot open / re-flyable) and `Immutable` (slot sealed / closed) as the slot is sealed (Seal or auto-seal) or re-opened (Stash). No other field on a committed Recording is rewritten. `EffectiveState.IsVisible` (`Source/Parsek/EffectiveState.cs:129`) is computed by walking `RecordingSupersedeRelation` entries; the Recording itself carries no `SupersededByRecordingId` field. Tree-scoped rewind-state purge (triggered by merge-dialog Discard or any other whole-tree discard path, §6.17) is the only path that ever removes supersede relations or tombstones, and it clears all such rewind artifacts for the tree atomically; the exceptions are the discard of a Re-Fly session an in-session load resumed after its merge, which removes only the rows and tombstones that name the attempt it deletes (section 6.14, owner ruling OQ-1), and an in-session load that resumes a flight from a save taken before a Re-Fly merged into it, which hands that flight's rows back to the save as part of the future it retires (section 6.16). A plain Rewind-to-Launch removes nothing, a dangling row included (section 8). A load

of a save file (F9, the Load menu) removes nothing either: it is a cold load that replaces memory with the save's own lists, so the rows of a merge made after that save are simply not loaded (section 6.16, `docs/dev/design-load-restores-the-save.md`). The Revert-during-re-fly dialog's Discard Re-fly option is NOT a tree purge — it is session-scoped (§6.14) and preserves the tree's supersede / tombstone / RP state. The tree's committed recordings themselves are never rewritten — separate pending-tree discard handles removal of pre-commit recordings, and committed recordings stay in the timeline indefinitely.

A slot's open/closed state is the slot's effective chain+supersede tip recording's `MergeState`: `CommittedProvisional` is open (re-flyable Unfinished Flight or stashed stable leaf), `Immutable` is closed (sealed / concluded / canon). This is the single source of truth. Seal (player invocation) and auto-seal (the merge-time contract, §4.9) both flip the tip `CommittedProvisional -> Immutable`; the transition is permanent and never reversed in-game. (An earlier design used a separate `ChildSlot.Sealed` bit decoupled from `MergeState`; that bit was removed and its meaning folded into `MergeState`.) `ChildSlot.Stashed` remains a separate, monotonic intent signal: it lets a default-excluded stable leaf appear in Unfinished Flights (Stash opens the slot by demoting its tip `Immutable -> CommittedProvisional`) and, once set, blocks any re-stash so a stash-then-seal cannot reopen a closed slot.

2.3 Broad tombstone semantics behind a value gate; Seal as the player override

Supersede is both a physical-visibility/claim-tracker mechanism and, for reviewed recording-scoped career actions, an Effective Ledger Set retirement mechanism. The tombstone-eligible type list includes contracts, milestones, facilities, strategies, science, funds/reputation, and kerbal consequences whose source recording lies in the superseded subtree. Initial seeds, null-scoped KSC/system rows, already-paid rollout costs, and unknown future action types stay preserved until explicitly reviewed. Because stock KSP systems do not reliably re-emit retired career events, `CommitTombstones` immediately runs a full post-tombstone recalculation and KSP patch pass to reconcile removed active contracts, tombstoned Parsek-managed terminal contracts, science subjects, facility levels, and safe roster-created kerbals while preserving unrelated finished contract history.

The eligible type list stays broad so that the two flights can never both count. What changed on 2026-10-19 (section 2.12) is which flights can be retired at all: a slot whose retire set holds a sealing row (any gain, contract outcome or flight-tagged spend) is not re-flyable, so a merge only ever retires loss rows (crew deaths and bookkeeping, vessel-loss reputation, building damage) and can only raise the career's balances. The broad list and the value gate are one design: narrowing the list would bring back double credit, and dropping the gate would bring back retired credit that later purchases relied on.

The same narrowness governs the v0.9.1 broadened predicate. The classifier auto-includes obvious-feeling cases (Crashed, Orbiting non-focus, SubOrbital non-focus). Over-inclusion is handled by the player Sealing the row. Under-inclusion (rover drove 20m and the player wants it re-flyable) is handled by Stashing the row while its backing Rewind Point still exists. **No heuristic predicates** beyond the simple terminal-state-plus-focus rule. The R1-R3 voluntary-action heuristic exploration was explicitly rejected.

2.4 Crash-recoverable merge

Every staged-commit merge step is journaled through `MergeJournal.Phase` (`Source/Parsek/MergeJournal.cs:53`). Eleven phase strings mark the merge's progress; on load, the finisher reads the phase and drives the merge to completion via idempotent re-run (in-memory mutations of `TreeMerge` / `Split` / `Supersede` / `Tombstone` / `Finalize` are recovered by re-running the same helpers, since each step is value-write idempotent or has its own dedup guard). No phase rolls back: `MergeDialog.MergeCommit` commits the attempt's tree (fork, EVA kerbals, payloads, their ledger rows) before `RunMerge` writes the journal, so the `Begin` save already holds the committed attempt (todo MERGE-BEGIN-ROLLBACK-LEAKS-ATTEMPT). The `Begin` save runs after the in-place-continuation fork migrates into the committed tree (`TreeMerge`), and `Split` (cuts the origin recording at the rewind UT) carries its own DurableSave barrier. The forward-completion path therefore starts at `Begin` (resumed as `TreeMerge`), not `Durable1Done`. The journal IS the durability barrier; the next natural `ScenarioModule.OnSave` flushes each phase's in-memory state to disk. Tests inject a synchronous save stub to exercise every window (see `MergeCrashRecoveryMatrixTests.cs`).

2.5 Atomic provisional + marker write

The provisional re-fly Recording and its `ReFlySessionMarker` must land in the scenario in the same synchronous block; no KSP `OnSave` may capture the first without the second. `RewindInvoker.AtomicMarkerWrite` (`RewindInvoker.cs:463`) enforces this: the provisional is added, the marker is constructed and assigned, and a `try/catch` rolls back both if anything throws. There is no coroutine yield, no deferred save, and no frame gap between `CheckpointA:BeforeProvisional` and `CheckpointB:AfterMarker` — the `CheckpointHookForTesting` observer (`RewindInvoker.cs:47`) is the only thing that can see in between.

2.6 Player-visible opt-in

A re-fly never happens behind the player's back. It requires an explicit click on the Rewind button for an Unfinished Flight row; both the virtual group copy and the normal recordings-table copy route through `RewindInvoker` instead of the legacy tree-root launch rewind. The confirmation dialog names the canonical semantics (`RewindInvoker.ShowDialog` body at `RewindInvoker.cs:157`).

Rewind Points are written automatically on every multi-controllable split, but they are invisible until a sibling ends up Unfinished — the player sees the UI entry only when it is actionable.

2.7 Observable from logs alone

Every decision point in section 6 and every RP / marker / journal / sweep / reap state transition emits a log line. The tag catalog in section 12 is what a KSP.log reader needs to reconstruct a session's history: the `[Rewind]`, `[RewindSave]`, `[RewindUI]`, `[ReFlySession]`, `[Supersede]`, `[LedgerSwap]`, `[MergeJournal]`, `[LoadSweep]`, `[UnfinishedFlights]`, `[CrewReservations]`, `[RevertInterceptor]`, `[Recording]`, `[ReconciliationBundle]`, `[ERS]`, `[ELS]` tags appear exactly where section 6 and section 12 say they do. The predicate gates emit `[UnfinishedFlights] Verbose` lines with structured reasons (`crashed`, `stableLeafUnconcluded`, `evaNotSeparation`, `evaCrewJoinedForeignVessel`, `sealedTipClosed`, `downstreamBp`, `noFocusSignalOrbiting`, `stableTerminalFocusSlot`, `recordingAction:*`; a verdict read through the vessel's own EVA carries `walkedEva=N walkedBoard=M` after its `side=` token); Site A and Site B-1 emit `[UnfinishedFlights] Info` on promotion with the resolved slot+RP; Seal accept/cancel logs the slot's tip `mergeState=<old>-><new>`. A KSP.log reader can reconstruct why every row appeared or didn't, and what the Seal action did to each slot.

2.8 Predicate is shared; safety gates cap retry intent

The Unfinished-Flight membership predicate is shared between Site A (original tree commit) and Site B-1 (re-fly merge classification) for diagnostics and crash detection. Manual Stash is the player's intent signal for a stable leaf, but retry intent is capped by hard safety gates: no recovered/docked/boarded terminal, no downstream structural/world interaction, and no sealing row in the slot's retire set (section 2.12; before 2026-10-19 only a credited `ScienceEarning` counted). The gates read the slot's whole walked history (section 1.6): the vessel's real tip past its own EVA and re-board, a crew member who left for a foreign vessel (`evaCrewJoinedForeignVessel`), and science earned on the crew's EVA recordings. Site A keeps `stableLeafUnconcluded` and `stashedStableLeaf` rows open so the player can come back to them — the original-tree commit does not represent player intent to conclude a slot. **Site B-1 (re-fly merge) is stricter:** the slot the player chose to Re-Fly is treated as the merge-time focus regardless of `RewindPoint.FocusSlotIndex`, and reaching a stable terminal on that slot now seals it, because the player explicitly engaged the slot to fly it themselves and the merge concludes that engagement. A Re-Fly that authors a structural branch point during the session (decouple / stage / undock / joint break / EVA) also seals on merge regardless of where the chain tip lands, since those are explicit player-driven shape mutations the player wants preserved. Terminal failure outcomes (`Destroyed`) remain retryable only while the slot's retire set holds loss rows alone (crew deaths and bookkeeping, vessel-loss reputation, building damage), because retiring a loss can only raise the timeline (section 2.12). Any other row seals: an automatic consequence (milestone, contract result, funds or reputation earning, recovery, rescue) and a flight-tagged spend are value the timeline may already have spent at the Space Center, so retiring it could leave a later purchase unpaid. KSC-scene player decisions carry no flight tag and never reach the retire set; the rollout vessel-build cost is never eligible. Background protection still applies for vessels the player merely controlled and then switched away from at scene exit (handled by Site A's natural-promotion path, which does not flow through the focus override). See section 4.9 for the full Re-Fly auto-seal contract and section 6.21 for the algorithm.

2.9 Layering: classifier owns the helpers, UI consumes

`TryResolveRewindPointForRecording`, `IsUnfinishedFlightCandidateShape`, `IsVisibleUnfinishedFlight` live in `UnfinishedFlightClassifier` (extracted in v0.9.1 from `UI/RecordingsTableUI.cs`). `RecordingStore`, `SupersedeCommit`, and the Seal handler all call them from a non-UI namespace. UI continues to consume the classifier; the classifier never reaches into UI.

2.10 Forward-only for vessels

Pre-feature saves with legacy live RPs whose Orbiting siblings are Immutable do NOT retroactively populate UF — the `FocusSlotIndex == -1` short-circuit suppresses Orbiting/SubOrbital qualification on legacy RPs (no focus signal to discriminate routine upper stages from probe deploys). v0.9.1 also surfaced stranded EVA kerbals retroactively; the 2026-09-27 ruling removed that exception (§1.5), so no legacy row surfaces retroactively.

2.11 Seal is slot-only; supersede can retire reviewed career actions

Unfinished Flights is a recording/slot affordance. Seal closes only the `ChildSlot` (see §4.7); it must not prune ledger actions, stored game-state events, or Effective Ledger Set (`EffectiveState.ComputeELS`) input. Re-fly merge is different: it appends supersede relations for ERS/ghost visibility and may append broad reviewed-career tombstones via `SupersedeCommit.CommitTombstones`. A post-tombstone recalculation and KSP patch pass then decides effective credit from the surviving ledger and reconciles stock state. A later re-fly may append new actions, but duplicate or once-ever credit is resolved by the recalculation modules over the post-tombstone ELS, not by mutating historical `Ledger.Actions`. Since the 2026-10-19 ledger-safety rulings (section 2.12) a flight a merge can retire holds no gain by construction, so the tombstones retire losses and crew bookkeeping only.

2.12 Ledger safety: a Re-Fly can only raise the timeline (owner rulings 2026-10-19)

Status: designed 2026-10-19, Option 1 confirmed by the owner the same day, built (the plan below is archived); the seal reads the replay's credit since resources design 3.15 build order step 8 (owner ruling 2026-10-26). Analysis: `docs/dev/done/research/refly-ledger-paradox-2026-10-19.md`. Implementation plan: `docs/dev/done/plans/refly-ledger-safety-implementation.md`. Todo: `REFLY-MERGE-RETIRES-VALUE-LATER-SPENDING-USED`. Where this section and an older paragraph of this document disagree, this section wins, and the older paragraph is stale until it is rewritten.

Why. Re-Fly is the only operation in Parsek that rewrites past events. Its purpose is narrow: let the player fly both halves of one separation (the upper stage to orbit, the booster back to a landing). The 2026-05-06 broadening of the tombstone scope (`f4763de1`, `d66626ba`) fixed double credit (both flights paying out) by retiring every career row of the replaced flight. That broke the game-actions Layer 1 promise ("existing spendings that were valid when committed cannot become invalid", `parsek-game-actions-and-resources-recorder-design.md` section 1.4): the replaced flight's earnings could already have paid for a Space Center purchase later in the timeline, so the merge left that purchase unpaid (funds below zero, a tech node never unlocked, a milestone reward lost). Owner rulings (V, 2026-10-19):

> We absolutely never change past behavior that could affect the ledger in a negative value way.

> We have to find a solution to NEVER introduce ANY time travel paradoxes.

> If F2 gained anything and the player merges, then it gets sealed.

> During a re-fly the player can fly only the re-flown vessel and its own crew on EVA; switching to any other vessel is blocked. File each award under the vessel that caused it.

> Discard should never be refused; it's all a temporary unmerged flight until it's merged. Quickloading takes the whole game state back to what it was during that specific moment and flight, so there should not be any related paradoxes.

> A quickload also undoes Space Center purchases made after the quicksave.

Neither scope on its own is safe: the narrow scope double-counts, the broad scope takes credit back. The rules below make the two flights never both count AND never take a credit back, by allowing a Re-Fly only of a flight that has nothing to give back.

Invariant LS (ledger safety). Let `L` be the Effective Ledger Set just before a Re-Fly merge with the attempt's rows left out, and `L'` the Effective Ledger Set just after the merge. For funds, science and reputation, at every UT, the balance computed over `L'` is at least the balance computed over `L`; every spending row that is affordable in `L` is affordable in `L'`; and no committed row or recording after the rewind UT uses a kerbal or a facility that `L'` leaves dead (past any stock respawn `KerbalsModule` models) or destroyed at that UT where `L` did not. Invocation, the live session, Retry and every Discard leave `L` exactly as it was before the invocation, and no Discard is ever refused (Rule 3). One consequence the rules rely on: a Re-Fly can move a once-ever credit (a first-time milestone, a capped science subject, a contract completion) only EARLIER, because the recalculation credits the earliest effective row and a merge only removes losses and adds the new flight's rows.

Terms.

- **Retire set** of a slot: the ledger rows a merge into that slot would tombstone if it ran now. It is the rows tagged to a recording of the merge closure rooted at the slot's current tip (section 6.3, including the walked own-vessel stretch of section 1.6 and its descendants), that pass `TombstoneEligibility.IsSupersedeTombstoneEligible`, and that `TombstoneAttributionHelper.IsPreRewindAttributedAction` does not keep as earned before the rewind point. It is exactly the set `SupersedeCommit.CommitTombstones` writes (section 6.13); one helper computes both, so the gate and the merge can never disagree.
- **Loss row**: a row in the retire set that only takes value from the player and that the replacement flight re-records by itself if it repeats the event, because the row is captured from the flight's own events rather than gated by stock state. The complete list: `KerbalAssignment` with end state `Dead`, `Aboard` or `Unknown` (crew bookkeeping and deaths), `ReputationPenalty` with source `KerbalDeath` or `Other` (crew death and vessel loss), and `FacilityDestruction`.
- **Sealing row**: every other row in the retire set that the ledger replay counts (a row the replay leaves out as a duplicate does not seal, see below). That covers every gain (`ScienceEarning`, `ScienceLabEarning`, `StrategyScienceCredit`, `FundsEarning`, `ReputationEarning`, `MilestoneAchievement`, `ContractComplete`, `KerbalRescue`, `KerbalHire`, `KerbalStandIn`, `KerbalExperience`, `KerbalRecovered`, `VesselRecovered`, `ResourceScanUnlocked`, `KerbalAssignment` with end state `Recovered`), every contract outcome (`ContractAccept`, `ContractFail`, `ContractCancel`, and `ReputationPenalty` with source `ContractFail` or `ContractDecline`), every flight-tagged spend or exchange (`ScienceSpending`, `FundsSpending` other than `VesselBuild`, `StrategyScienceDebit`, `ReputationPenalty` with source `Strategy` or `StrategyConverter`), facility and strategy rows (`FacilityUpgrade`, `FacilityRepair`, `StrategyActivate`, `StrategyDeactivate`), and any type added later until it is reviewed into the loss list. A contract failure is a sealing row although it costs the player: during a session stock already holds the contract as failed, so the replacement can never fail it again, and retiring the old failure would erase the penalty for good.

The seal covers the failing flight's own slot only: another open slot whose rewind point is earlier may complete the same contract before the failure, and then the walk credits the completion and leaves the later failure out as `ContractOutcomeAfterEnd` (owner ruling 2026-10-27: the earliest outcome wins, as for every repeated event after a rewind).

A sealing row seals only when the replay counts it (resources design section 3.15 item 10, build order step 8; owner ruling 2026-10-26). Built.

- A non-loss row in the retire set seals only when the ledger replay counts it, even at zero value, so a first-reached milestone with no award still seals.
- A duplicate the replay does not count does not seal: a world-first, a record threshold, a contract outcome or parameter the timeline already credited or ended, a purchase the walk already owns, science of a subject whose cap is already credited. The player sees it as more crashed flights offered for Re-Fly: a flight whose only gains after its separation were repeats of credit the timeline already holds.
- Only when stock's payment of the duplicate cannot be kept: stock paid it nothing, or the replay takes the payment back (resources design 3.15 step 5), or it is a purchase already owned (a kept charge is the player's loss). A payment stock kept stays as an untagged pool adjustment in no retire set, so a re-fly could be paid again. These not-counted rows therefore seal: a completion past the walk's deadline (a deadline expiry is no holder), a completion the same-tick failure resolved, and a paid repeat below the take-back era or with no timeline break between it and its holder.
- Why: reading only whether a row exists gave two slot states for the same play. When the live mirror stopped stock's payment no row existed and the slot stayed open; when the replay took the payment back afterwards the row existed and the slot sealed. Reading the replay's credit gives one state for both.
- The losses-only rule is unchanged. A not-counted row is still in the retire set and the merge still retires it, which retires nothing the replay counts: the credit stays with the row that holds it, and that row seals its own slot. Rule 5's balance check still runs on `L` and `L'`.
- The verdict comes from a fresh full walk of the ELS on clones (plus the rows a commit is about to file, when the gate reads them), never from `Effective` flags an earlier walk left on the shared rows: a stale flag must not open a slot that holds a real counted gain. Every doubt seals (a walk that throws, no walk modules, a row the walk did not see, a not-counted reason nobody has reviewed, a not-counted row with no counted holder or a kept payment). One filter, `SupersedeCommit.SelectSealingRows`, serves `RetireSetHoldsSealingRow` (every site of Rule 1 below) and the merge's own Rule 1 re-check in `ReFlyMergeDominance.Evaluate`, which reads its own walk of `L`. Details: resources design section 3.15 item 10.

Rule 1: value gate. A slot whose retire set holds a sealing row (a counted non-loss row) cannot be re-flown. One predicate, `SupersedeCommit.RetireSetHoldsSealingRow(slot, out summary)`, replaces the `ScienceEarning`-only `IsRetryBlockingRecordingAction` at every site that read it:

- **Site A** (Unfinished Flights membership, every `UnfinishedFlightClassifier` arm that runs the recording-action check: `crashed`, `stashedStableLeaf`, `stableLeafUnconcluded`, the focus override): a slot whose retire set holds a sealing row does not qualify, reason `recordingAction:<Type>:<ActionId>` (the existing prefix, so existing greps and the auto-seal reason catalog keep working). On the original tree's commit Site A runs inside `CommitTree`, before `NotifyLedgerTreeCommitted` files the tree's rows, so promotion reads the rows the commit is about to file (`LedgerOrchestrator.BuildTreeCommitRowsPreview`, scoped by `SupersedeCommit.UsePendingCommitRows`); a slot its own new rows seal is never promoted. When several sealing rows are present the reason names the first one that is not a zero-award milestone (`SupersedeCommit.PickNamedSealingRow`), so a first-reached milestone with no award does not hide the earning the player recognises; every sealing row seals the same, and a zero-award milestone alone still seals (no milestone exception, owner ruling 2026-10-20).
- **The Stash resolver** refuses such a slot with the same reason.
- **`RewindInvoker.CanInvoke`** re-checks it (precondition 7, section 6.8), so a ledger that changed after the row was drawn cannot slip through.
- **Site B-1**: after the merge the re-fly is the slot's tip, so its own retire set (the rows the attempt filed, minus anything Rule 2 tombstoned) decides whether the slot stays open. A re-fly that gained anything seals the slot, whatever its terminal; a re-fly that only crashed again, earning nothing, keeps it open.
- **`ReFlyAutoSealPreview`** and every other mirror of the old gate read the same predicate.

Rule 1 is what makes a merge raise the timeline: the replaced flight's retire set holds only loss rows, so retiring it can only return value, and nothing on the old side can double-count with the new flight. It also closes the bank-then-retire exploit (a flight earns, the player spends the money at the Space Center, then re-flies the earning away), because the earning seals the slot before the player can come back to it.

The purpose of Re-Fly survives: a booster that crashed in the background earned nothing, so it stays re-flyable, and landing it adds its recovery as a new earning. The cost is accepted: a crashed flight that earned science, funds, reputation, a milestone or a contract result after its separation is no longer offered.

Rule 2: the loss swap. At merge the replaced flight's loss rows are retired and the re-fly's own rows count. That can still lower reputation when the re-fly loses more than the replaced flight did (it kills more crew, or loses a vessel the replaced flight did not lose). The merge therefore evaluates two variants on cloned modules before the journal begins (Rule 5):

- **Variant A**: the replaced flight's loss rows are retired; the re-fly's rows count. Taken when reputation over `L'` is at least reputation over `L` at every UT.
- **Variant B**: as A, except that the replaced flight's `ReputationPenalty` loss rows are kept and the re-fly's `ReputationPenalty` loss rows are tombstoned by the same merge, with `RetiringRecordingId` set to the re-fly's own recording (so the writer-side rules of sections 6.14 and 6.16 treat them as the attempt's rows). Taken otherwise.

Deaths themselves always follow the re-fly (a kerbal who died in the replaced flight and lived in the re-fly is alive; one who died only in the re-fly is dead), because the roster must match the world the player flew. A facility the re-fly destroys that the replaced flight did not is checked by Rule 5's world-object clause. The chosen variant is logged (`[LedgerSwap] Info: Loss swap sess=<s> variant=<A|B> repLossOld=<x> repLossNew=<y>`).

Rule 3: session isolation. The session must not change `L`, and the attempt must own exactly the rows it caused:

- **Switching is blocked.** While a session is live the player can fly only the re-flown vessel (its own walked stretch, section 1.6, as it continues through the session) and its own crew on EVA. Every player-initiated switch to any other vessel is refused: `[` / `]`, map Switch-To and map focus Fly, Tracking Station Fly, the KSC marker Fly and the Jump button of a fired stock alarm. This includes a vessel the re-fly itself split off by a separation (it keeps a session rewind point and can be re-flown after the merge). Changes of the active vessel that stock makes as part of an EVA, a board, a dock or an undock are not switches and stay allowed. The pre-switch Merge / Discard dialog of `MergeDialog.ShowPreSwitchDecisionDialog` (Cases A to C) is therefore never reached during a session. Refusals log `[ReFlySession] Info: Switch refused sess=<s> path=<p> target=<pid> reason=<r>`.
- **Awards are filed by cause.** A ledger row is tagged to the recording of the vessel whose event caused it (the vessel a contract parameter, a recovery, a destruction or a science subject names). An event that names no vessel, or names a vessel with no recording in the session (an unrelated live vessel the strip left alone), goes to the live recording, so it is always the attempt's row: it can seal the slot and a Discard removes it. No session row is filed untagged, except an event the clock causes whatever the flight does (a contract deadline expiry, a contract stock ends for lost requirements, a strategy reaching its duration; owner ruling 2026-10-24): it is filed untagged and written to the ledger at once. Today every award is tagged to the live active recording (`ParsekFlight` / `GameStateRecorder`) and an empty tag falls back to the provisional (`ReFlyAwardTagging.Resolve`), which can file a background vessel's gain under the re-fly and wrongly seal it, or a re-fly gain under a background vessel and miss it.
- **The session's contract deadline is the session's clock.** The invoke-time recalculation walks the whole ledger (horizon `double.MaxValue`, so the session shows the committed career, section 6.8 step 5), but `ContractsModule.PrePass` must judge a contract's deadline against the current UT, not against the walk horizon. Today the horizon is passed as the deadline clock and every deadline contract reads as expired during the session. The clock never judges earlier than the last committed row the walk reads: a deadline a committed row walks past has expired in the committed timeline, and the walk fails it there, exactly as every clock-less walk of the same ledger does (an injected fail moves contract state only; its penalty reaches a pool when stock itself fails the contract, game-actions design 3.13) (judged at the clock alone, the contract left the walk's active set at that row with no penalty, todo `REFLY-WALK-DEADLINE-EXPIRES-CONTRACT-WITHOUT-PENALTY`). The dominance check's two walks share one such instant. A stock deadline expiry during the attempt is the clock's event, not the attempt's (owner ruling 2026-10-24): it is filed untagged and written to the ledger at once, so both walks carry it and a Discard keeps it; a fail before the deadline still counts as the attempt's loss (todo `REFLY-DEADLINE-EXPIRY-REFUSES-THE-MERGE`).
- **A contract accepted after the session's clock stays out of stock.** The whole-ledger walk ends with a contract accepted at the Space Center after the rewind point active, and the state patch would restore it Active at the session's earlier clock, so the re-fly could complete or fail it before it was accepted. `KspStatePatcher.PatchContracts` keeps a walk-active contract out of stock's active list until the clock reaches its committed accept (`SelectContractsAcceptedByNow` ; todo `REFLY-SESSION-SHOWS-CONTRACT-ACCEPTED-LATER`).
- **A vessel the committed future uses is held back at invoke.** When the rewind point brings back a vessel that a committed recording flies, changes or recovers after the rewind UT (matched by launch, `VesselLaunchIdentity`), the post-load strip hands it to the existing claimed-vessel hold (`VesselGhoster` plus `ParsekFlight.UpdateClaimedVesselHoldGhosts`, which already holds chain-claimed vessels after a rewind): it is despawned, drawn as a hold ghost in its rewind-save shape until its first committed recording starts, and from then on that recording's ghost plays it. Nothing in the session can dock with it, board it, bump it or recover it, and its committed rows (a later recovery included) stay. This replaces today's invoke-time retirement of the recovery rows of such a vessel (`RewindInvoker.RetireResurrectedVesselRecoveryRows` , the "Step 3b" of `ConsumePostLoad`), which ran outside any merge, bypassed the drawdown guard and outlived a Discard. A dock or board onto the vessel by a committed recording after the rewind UT counts as a use of it, whether or not the docking flight's ghost chain is still active (the chain feed drops a chain whose flight ended destroyed or recovered); a dock by a recording that no longer plays (superseded, or the session's own replaced flight) does not. The held vessels are saved with the session marker, so a reload during the session (F9, a scene reload) holds them again and redraws their hold ghosts from the rewind point's save; the hold despawned them, so no

save written during the session contains them. Unrelated live vessels with no committed future after the rewind UT stay live; the player cannot switch to them, and what the re-fly does to them is filed under the re-fly (awards bullet below).

- **The session shows the replaced flight's losses until the merge** (owner ruling 2026-10-21; todo REFLY-SESSION-CAREER-SHOWS-THE-REPLACED-FLIGHTS-FUTURE closed). Under Rule 1 the replaced flight holds only losses after its split, and the session's whole-ledger walk keeps showing them until the merge retires them. Hiding them for the session was rejected: a Discard would bring them back, a transient drop, and the session never shows more than its outcome would. Known exception: the Re-Fly start walks the whole ledger, so during the attempt the funds pool also includes the committed future's earnings (RW-5: 150632 → 191064) until the merge or the discard takes them back out (open todo REFLY-ATTEMPT-LIVE-POOL-SHOWS-COMMITTED-FUTURE).
- **Discard and Retry restore `L` exactly.** Every discard path (the Esc-menu Discard Re-fly, the scene-exit merge dialog Discard, the rewind end, Retry and the load-time sweep's zombie discard) removes the ledger rows tagged to the attempt's recordings instead of untagging them (`Ledger.ClearRecordingTagForRecordings` keeps the row and its career effect today), and (until Phase 3 of the load design deleted it with the in-session resume that needed it) removed the tombstones and rewind retirements a resumed merged attempt wrote. The attempt was never committed, so removing its rows is consistent with the append-only rule (section 2.2), the same way a pending-tree discard removes uncommitted recordings; the Re-Fly discard paths already revert stock state through the rewind point's quicksave or the post-discard recalculation, so no row needs re-homing (the non-rewind `PreserveIrreversibleLiveGameplayOnDiscard` does not apply). Untagged rows written during the session (section 7.42) stay. Discard is never refused, on any path: the attempt is a temporary, unmerged flight until it merges.
  An attempt a load resumed after its merge (section 6.14, owner ruling OQ-1) is unmerged again: a load of a save file is a cold load that reads the save, taken during the session before the merge, so neither the merge's rows nor anything committed after the save is in memory (owner ruling 2026-10-19). Its Discard and Retry follow the same rule as any other. Rewinds keep ruling D2: a rewind frees nothing.

Rule 4: once-ever credit only moves earlier. The replaced flight holds no once-ever credit to retire (Rule 1), and the recalculation credits the earliest effective row, so the re-fly's gains can only move a credit earlier. The re-fly can still REMOVE one with its own rows (it fails a contract a later committed flight completes, so the completion pays nothing); Rule 5 refuses that merge. Tests pin both.

Rule 5: dominance check before the merge. The attempt's own rows reach `Ledger.Actions` only at tree commit (`MergeDialog.MergeCommit` runs `CommitPendingTree`, then `LedgerOrchestrator.NotifyLedgerTreeCommitted`, whose `FileRecordingCommitRows` files them and runs a live recalc and patch, and only then `TryCommitReFlySupersede` and `RunMerge`). A check placed before the journal's `Begin` would run after the tree is already committed, so a refusal there would leave a committed Re-Fly tree with no supersede rows and the marker set. The check therefore runs in `MergeDialog` BEFORE `CommitPendingTree`: when the merge dialog opens (to decide what it offers) and again when Merge is clicked. It needs the attempt's rows without committing them, so the implementation splits the row filing into a pure build step (the pending tree's events and crew end states to a scratch list of rows, on copies; no `Ledger.Actions` write, no seed write, no patch) and the existing append-and-recalc step, and the check consumes the build step's output. The optimizer pass that runs after `CommitPendingTree` only splits recordings and moves row tags between the halves; it changes no amount and no UT, so it cannot change a balance. With those rows the check re-checks Rule 1 on the retire set, picks the Rule 2 variant, computes `L` and the chosen `L'` on cloned resource modules (no live state touched) and checks Invariant LS. Rules 1 to 3 cover the replaced flight's side; the re-fly's own rows are what the check exists for: a re-fly that fails a contract still active in the committed timeline (the penalty lands earlier than anything in `L`, and a later committed completion would pay nothing), or that kills a kerbal a later committed recording flies before stock's respawn returns him, or destroys a facility a later committed row uses. When the check fails the merge is not offered: the existing merge dialog shows why in its body text and keeps Discard and, where stock offers it, Retry. Log `[LedgerSwap] Info: Merge dominance check sess=<s> result=<pass|refuse> pool=<p> ut=<u> before=<b> after=<a>` (one line; on refuse it names the first failing pool and UT, or `world=<kerbal|facility>:<id>` for the world-object clause).

What the player sees. No new UI. Fewer crashed flights appear in Unfinished Flights (only those that earned nothing after their separation); the group's existing hover gains one sentence saying so; the existing Re-Fly confirmation dialog's career paragraph is rewritten to match; a refused switch does nothing; a refused merge is explained in the existing merge dialog.

Migration. Forward-only. On load, `LoadTimeSweep` concludes (flips `Immutable`) the open tip of every slot whose retire set holds a sealing row, stashed slots included, so its rewind point can be reaped; one summary line `[LoadSweep] Value-sealed slots (section 2.12): found=<n> sealed=<m>`. Tombstones already written and deficits already in a save are not repaired: repairing them would itself rewrite past events.

Outside Re-Fly. Quickload trims, revert-to-launch pruning, recordings that fail to load, route credits after a rewind, and post-rewind flights that insert a loss before a committed spend break the same property without Re-Fly. They are filed separately (todo `LEDGER-RETIREMENT-PATHS-OUTSIDE-REFLY`) and are not changed by this section. The owner ruled every one of them on 2026-10-21; the general rule (committed outcomes do not move earlier, known obligations are reserved, a click that digs into a reservation is greyed, any other loss stops at the reserved floor with the rest carried as a debt) is game-actions design section 3.13.

---

3. Terminology

This section fixes the vocabulary. Throughout the doc, "recording" (lowercase) is the general concept; **Recording** (capitalized, code-font) is the class (`Source/Parsek/Recording.cs`). "Session" always means a Rewind-to-Separation re-fly session unless qualified.

- **Split event** — a `BranchPoint` whose type is `JointBreak`, `Breakup` or `Undock`: a vessel separation. The common case is staging (joint break on the decoupler); a lander undock is the same shape. An `EVA` BranchPoint is still a split in the recording DAG (`parsek-flight-recorder-design.md` section 4.6) but it is NOT a rewind split: it never gets a Rewind Point (§1.5).
- **Controllable entity** — a vessel with at least one `ModuleCommand` part (the classifier also counts an EVA kerbal, but an EVA split never reaches RP creation, §1.5). The classifier `SegmentBoundaryLogic.IsMultiControllableSplit` gates RP creation on `count >= 2`. A crewed part with no command module (a decoupled Hitchhiker with a kerbal inside) is recorded as its own branch but is NOT a controllable entity: it is never a slot and never counts toward the two (2026-10-22).
- **Multi-controllable split** — a split event that produces two or more controllable entities. Only these get a Rewind Point.
- **Rewind Point (RP)** — the durable object at a multi-controllable split. Holds the quicksave filename, the save UT, the ChildSlots, the PidSlotMap, the RootPartPidMap, plus bookkeeping fields. Defined in `Source/Parsek/RewindPoint.cs`. "Session-provisional" during re-fly; "persistent" after session merge.
- **Child slot** — a stable, ordered entry on an RP identifying one controllable output of the split. Carries `OriginChildRecordingId` (immutable) and derives `EffectiveRecordingId` via a forward walk through `RecordingSupersedeRelation`. Defined in `Source/Parsek/ChildSlot.cs`.
- **PidSlotMap** — `Dictionary<uint, int>` mapping `Vessel.persistentId` to `ChildSlot.SlotIndex`. Populated at quicksave-write time by `RewindPointAuthor.ExecuteDeferredBody` (`Source/Parsek/RewindPointAuthor.cs:202`). The authoritative primary match for the post-load strip.
- **RootPartPidMap** — `Dictionary<uint, int>` mapping root-part `Part.persistentId` to `ChildSlot.SlotIndex`. Populated at the same time as PidSlotMap. Fallback match when KSP has reassigned a vessel-level persistentId between save and load. `Part.persistentId` is the stable cross-save/load identity for a physical part; `Part.flightID` is session-scoped and unstable.
- **Finished recording** — a Recording with `MergeState == Immutable`.
- **Unfinished Flight** — a Recording matching `EffectiveState.IsUnfinishedFlight` (see §5). Visible in the ERS with a live RewindPoint entry on the parent BranchPoint (non-null `RewindPointId`), and a qualifying terminal state: Crashed, or a non-focus Orbiting or SubOrbital stable leaf. An EVA kerbal never qualifies (§1.5). See §5 for the full `EffectiveState.IsUnfinishedFlight` predicate.
- **Re-fly session** — the period between a Rewind Point invocation and its merge / discard. Identified by a `SessionId` GUID generated at invocation (`RewindInvoker.cs:230`). Ends on merge, discard, retry (new SessionId), full-revert, or load-time validation failure.
- **Session-provisional RP** — an RP whose `SessionProvisional` flag is `true`. Newly created RPs are session-provisional; the flag flips to `false` during the merge's `TagRpsForReap` step (`MergeJournalOrchestrator.cs:441`) for RPs whose `CreatingSessionId` matches the merging session. Load-time sweep discards session-provisional RPs only when they are session-scoped (`CreatingSessionId` non-empty) and not referenced by a valid marker; normal staging RPs created outside a re-fly session have `CreatingSessionId == null`, survive merge-dialog scene loads, and are promoted to persistent when the owning tree commits.
- **Supersede** — an append-only relation `RecordingSupersedeRelation { RelationId, OldRecordingId, NewRecordingId, UT, CreatedRealTime }` stored in `ParsekScenario.RecordingSupersedes`. No Recording is mutated; `EffectiveState.IsVisible` filters via the list.
- **Supersede chain** — the forward walk through `RecordingSupersedes` from a `ChildSlot.OriginChildRecordingId`. Chains can extend through multiple re-fly attempts (AB → AB' → AB'' if AB' was merged as a crashed `CommittedProvisional`).
- **Session-suppressed subtree** — during an active re-fly, a second narrow carve-out applied to physical-visibility subsystems only (ghost walker, chain tip, map presence, watch mode). Forward-only closure from the marker's `OriginChildRecordingId`, halting at mixed-parent BranchPoints. Each member also expands to its chain siblings (same `ChainId` AND same `ChainBranch`) so a `RecordingOptimizer.SplitAtSection` env split between HEAD and TIP is suppressed atomically — without this, the merge-time supersede would only retire the BP-linked HEAD and leave the terminal-Destroyed TIP visible. Different `ChainBranch` values stay independent (parallel ghost-only continuations). Computed by `EffectiveState.ComputeSessionSuppressedSubtree` (called from `Source/Parsek/SessionSuppressionState.cs`). Career and ledger subsystems do NOT consume this view — they read ERS / ELS directly.
- **Tombstone** — an append-only `LedgerTombstone { TombstoneId, ActionId, RetiringRecordingId, UT, CreatedRealTime }` in `ParsekScenario.LedgerTombstones`. Keyed by the retired `GameAction.ActionId`. Multiple tombstones for the same ActionId are tolerated; the ELS filter is "at least one tombstone exists." Defined in `Source/Parsek/GameActions/LedgerTombstone.cs`.
- **ReFlySessionMarker** — the singleton marker persisted while a session is live. Seven fields total; six are validated on load (`MarkerValidator.Validate`), `InvokedRealTime` is informational and never fails a load. Defined in `Source/Parsek/ReFlySessionMarker.cs`.

- **Provisional re-fly Recording** — the live Recording created at invocation with `MergeState == NotCommitted`, `CreatingSessionId` set to the session GUID, `SupersedeTargetId` set to the retired sibling's id, `ProvisionalForRpId` set to the invoked RP's id. Becomes `Immutable` or `CommittedProvisional` at merge.
- **MergeJournal** — the singleton journal that carries the merge through eleven phase strings (`Begin`, `TreeMerge`, `Split`, `Supersede`, `Tombstone`, `Finalize`, `Durable1Done`, `RpReap`, `MarkerCleared`, `Durable2Done`, `Complete`). Its presence on load triggers the finisher. Defined in `Source/Parsek/MergeJournal.cs`.
- **Effective Recording Set (ERS)** — the derived, read-only view of committed recordings that count right now. Computed by `EffectiveState.ComputeERS` (`Source/Parsek/EffectiveState.cs:236`). Filters out `NotCommitted`, superseded, and (during an active session) session-suppressed recordings.
- **Effective Ledger Set (ELS)** — the derived, read-only view of ledger actions that count right now. Computed by `EffectiveState.ComputeELS` (`EffectiveState.cs:310`). Filters out actions whose `ActionId` appears in any `LedgerTombstone`. **The ONLY filter on ELS is the tombstone check.** Supersede affects ELS only through explicit tombstones; preserved rows are the ones the reviewed tombstone classifier leaves untouched, such as seeds, null-scoped KSC/system actions, rollout vessel-build costs, and unknown future action types.
- **UF** — Unfinished Flight. A recording that satisfies `IsUnfinishedFlight(rec)` and is therefore visible in the Unfinished Flights virtual group.
- **Stable terminal** — a terminal state in `{ Orbiting, SubOrbital, Landed, Splashed, Recovered, Docked, Boarded }`. The opposite is `Destroyed` (Crashed) and the absent terminal (no `TerminalStateValue` set). Note: `SubOrbital` is included here because manual Stash treats it as a spawnable stable terminal, but it is NOT auto-sealed by the Re-Fly merge contract (§4.9): a suborbital arc is still in flight (the vessel will crash, land, splash, or with a burn reach orbit), so the slot stays open until the arc resolves. Only manual Stash + Seal closes a SubOrbital row.
- **Stable leaf** — a recording whose chain TIP has a stable terminal AND whose `ChildBranchPointId` is null OR equals the matched RP's `BranchPointId` (the breakup-survivor case).
- **Focus slot** — the `ChildSlot` in a `RewindPoint` whose vessel was the active focus at the moment the multi-controllable split fired. Identified by `RewindPoint.FocusSlotIndex` (default `-1` = no focus signal).
- **Non-focus slot** — any slot other than the focus slot. `slot.SlotIndex != RP.FocusSlotIndex AND RP.FocusSlotIndex >= 0`.
- **No-focus-signal RP** — a RewindPoint with `FocusSlotIndex == -1`. Either a legacy RP that predates the v0.9.1 field, OR a new RP where no slot was focused at split time (rare: e.g. the player was focused on an unrelated vessel outside the split).
- **Sealed slot** — a slot whose effective chain+supersede tip recording is `MergeState.Immutable` (closed / concluded). Either the player explicitly closed this slot via the Seal button (flipping the tip `CommittedProvisional -> Immutable`), or the Re-Fly merge auto-seal gate (§4.9) committed the provisional tip `Immutable` after a hard-close signal: a stable terminal on the player-chosen slot (`Orbiting` / `Landed` / `Splashed`, NOT `SubOrbital` which stays open as still-in-flight), a sealing row filed by the re-fly (any gain, contract outcome or flight-tagged spend, section 2.12), a structural mutation (decouple / stage / undock / EVA / joint break), or a hard safety terminal (Recovered / Docked / Boarded). The row drops from UF and the reaper treats an Immutable tip as closed for reap eligibility. There is no separate persisted seal flag; open/closed is the recording's `MergeState`. The auto-seal reason is logged at merge time; persisted `SealedBy` / `SealedReason` metadata remains a deferred follow-up.
- **Stashed slot** — a `ChildSlot` with `Stashed == true`. The player explicitly promoted a default-excluded stable terminal leaf into Unfinished Flights via the Stash button. Only spawnable stable terminals (`Landed`, `Splashed`, `Orbiting`, `SubOrbital`) qualify; `Recovered`, `Docked`, `Boarded`, and rows whose retire set holds a sealing row (section 2.12) are rejected by the resolver. Stash does not resurrect already-reaped RP quicksaves.
- **Retire set** — the ledger rows a merge into a slot would tombstone if it ran now: rows tagged to the merge closure of the slot's current tip, tombstone-eligible, and not kept as earned before the rewind point. The same helper feeds the value gate and `CommitTombstones` (section 2.12).
- **Loss row / sealing row** — the two classes of the retire set (section 2.12). Loss rows (crew deaths and bookkeeping, vessel-loss and crew-death reputation, building damage) can be retired; any sealing row (every gain, contract outcome and flight-tagged spend) makes the slot not re-flyable.
- **Stranded EVA** — (retired 2026-09-27) formerly an EVA kerbal recording with a non-`Boarded` terminal, which qualified as an Unfinished Flight. An EVA kerbal now never qualifies, whatever its terminal (`evaNotSeparation`, §1.5).
- **Prior tip** — the slot's current effective recording id at re-fly invocation time, before any new supersede relation has been appended for the current re-fly. Equals `slot.OriginChildRecordingId` on the first re-fly into a slot; equals the previous re-fly's recording id on chain extension. Stamped into `ReFlySessionMarker.SupersedeTargetId` by `RewindInvoker.AtomicMarkerWrite` (v0.9.1, see §6.22).
- **Site A** — `RecordingStore.ApplyRewindProvisionalMergeStates`. The MergeState-promotion call site that runs at original tree commit. Reads the broadened predicate; promotes stable-leaf siblings to `CommittedProvisional` so they appear in Unfinished Flights.
- **Site B-1** — `SupersedeCommit.FlipMergeStateAndClearTransient`. The MergeState-promotion call site that runs at re-fly merge. Applies the v0.9.1 auto-seal contract using the merge-time slot as `focusSlotOverride`.

- **Site B-2** — `MergeDialog.TryCommitReFlySupersede`'s in-place continuation path. Consumes Site B-1's verdict; clean stable Re-Fly outcomes commit `Immutable` + auto-seal at Site B-1 and Site B-2 lets the reaper remove the RP. Safety-closed outcomes follow the same path. Terminal-failure outcomes (`Destroyed`) whose retire set holds no sealing row (section 2.12 Rule 1) stay `CommittedProvisional` so the player can retry the failure.

---

4. Mental Model

The tree diagrams below use `(V)` for "visible in ERS" and `(H)` for "hidden (superseded)". `Immutable` is abbreviated `I`; `CommittedProvisional` is `CP`; `NotCommitted` is `NC`.

4.1 A mission with one staging split, one side crashes, player re-flies and lands

Launch: one vessel, `NotCommitted`, recording live.

```
ABC (NC, live)   (V)
```

Staging: `ABC` splits into upper stage `AB` (continues to orbit) and booster `A` (falls back). Both are controllable (probe cores on both). `SegmentBoundaryLogic.IsMultiControllableSplit` returns true; `RewindPointAuthor.Begin` creates RP-1 with two ChildSlots and schedules the deferred quicksave. On the next frame the quicksave is written and atomically moved into `Parsek/RewindPoints/rp_<guid>.sfs`; `PidSlotMap` and `RootPartPidMap` are populated from live vessels.

```
ABC (NC, terminates at split-1)   (V)
 |
 +-- [slot-0: AB]  (NC, live, focus)        (V)
 +-- [slot-1: A]   (NC, BG-recording)       (V)
     RP-1 (SessionProvisional, maps captured)
```

Orbit + merge: player flies AB to orbit, then returns to Space Center. Merge dialog appears. Booster A crashed in the background. On "Merge to Timeline" the recorder commits ABC and AB as `Immutable`; A's terminal kind classifies as `Crashed`, so its `MergeState` becomes `CommittedProvisional`. RP-1's slots: slot-0's effective recording is AB (`Immutable`), slot-1's effective recording is A (`CommittedProvisional`). RP-1 is NOT reap-eligible (slot-1 is still open), so the quicksave and scenario entry persist.

```
ABC (I)                                   (V)
 |
 +-- AB  (I, Orbited)                     (V)
 +-- A   (CP, Crashed)                    (V)  <- Unfinished Flight
     RP-1 persistent
     Unfinished Flights = {A}
```

`A` now satisfies `IsUnfinishedFlight`: visible, terminal kind classified as Crashed, parent BranchPoint has a live RP. The Recordings tab shows it in the Unfinished Flights virtual group (`STASH`) with `Fly` + `Seal` buttons.

Invocation: player clicks Rewind on row A. `RewindInvoker` runs through preconditions, captures the reconciliation bundle, copies the RP quicksave to the save-root as `Parsek_Rewind_<sessionId>.sfs`, triggers `GamePersistence.LoadGame` + `HighLogic.LoadScene(FLIGHT)`. On the reloaded scene, `ParsekScenario.OnLoad` calls `RewindInvoker.ConsumePostLoad`, which atomically:

1. Restores the pre-load reconciliation bundle over the quicksave-loaded state (recordings, supersedes, tombstones, reservations, etc. preserved).
2. Runs `PostLoadStripper.Strip(rp, slotIdx=1)` — AB is matched via `PidSlotMap` and stripped (`Vessel.Die()`); A is matched and kept; unrelated vessels are left alone; ghost ProtoVessels are skipped.
3. Calls `FlightGlobals.SetActiveVessel(A)`.
4. Creates the provisional re-fly Recording `A'` with `MergeState = NotCommitted`, `CreatingSessionId = sessionId`, `SupersedeTargetId = A.Id`, `ProvisionalForRpId = rp.Id`, and writes the `ReFlySessionMarker` — same synchronous block, no yield.
5. Recalculates the ledger via `LedgerOrchestrator.RecalculateAndPatch()`.

During the re-fly: ERS = `{ABC, AB, A}`. A' is `NotCommitted` so it is not in ERS. The ghost walker further filters ERS by `SessionSuppressedSubtree(marker)`, which contains `{A}` — so A does not render as a ghost. AB and ABC play back as ghosts. A' is the live active vessel, handled by the normal active-vessel path.

```
ABC (I)                                  (V)
 |
 +-- AB  (I)                             (V)  <- ghost playback
 +-- A   (CP, Crashed)                   (V)  <- in SessionSuppressedSubtree (invisible)
 +-- A'  (NC, live, CSId=sess1,          (V)  <- active vessel
          SupersedeTargetId=A)
     ReFlySessionMarker active
```

Successful re-fly + merge: player lands A' on the launch pad. Recording terminates with `TerminalKind = Landed` (via `TerminalKindClassifier`). Merge dialog fires `MergeJournalOrchestrator.RunMerge`. The orchestrator:

1. Writes the journal at `Phase = Begin`.
2. `SupersedeCommit.AppendRelations`: subtree closure for A is `{A}`; one `RecordingSupersedeRelation { OldRecordingId = A.Id, NewRecordingId = A'.Id, UT = now }` appended.
3. `SupersedeCommit.CommitTombstones`: scans ledger actions in-scope and tombstones any reviewed recording-scoped career actions owned by A. This example has no such A-scoped career action rows, so it writes zero tombstones.
4. `SupersedeCommit.FlipMergeStateAndClearTransient`: A' becomes `Immutable` (Landed). `SupersedeTargetId` cleared. Supersede state version bumped. Marker preserved for step 7.
5. Durable Save #1 (deferred — journal phase string is the durable barrier).
6. `TagRpsForReap`: RP-1's `SessionProvisional` was already false (from merge-1); its `CreatingSessionId` does not match sess1 (it was `null` when A's tree committed), so nothing is tagged here. `RewindPointReaper.ReapOrphanedRPs`: RP-1's slot-0 effective is AB (`Immutable`), slot-1's effective is A' (`Immutable`). Both slots are `Immutable` → RP-1 is reap-eligible. Quicksave file deleted, scenario entry removed, BranchPoint.RewindPointId cleared.
7. Marker cleared. Supersede state version bumped again.
8. Durable Save #2 (deferred).
9. Journal set to `Complete`, then cleared. Durable Save #3 (deferred).

After merge: A is hidden (superseded), A' is the effective representative of slot-1. Unfinished Flights is empty.

```
ABC (I)                                  (V)
 |
 +-- AB  (I)                             (V)
 +-- A   (CP, Crashed, SUPERSEDED)       (H)
 +-- A'  (I, Landed)                     (V)  effective slot-1
     RP-1 reaped (quicksave deleted)
     RecordingSupersedes: {A -> A'}
     Unfinished Flights = {}
```

4.2 Crashed re-fly stays rewindable (chain extension)

Same setup as §4.1 except A' also crashes. The player wants to try again.

On merge: `TerminalKindClassifier.Classify(A')` returns `Crashed`. `SupersedeCommit.FlipMergeStateAndClearTransient` assigns `A'.MergeState = CommittedProvisional`. Supersede relation `{A -> A'}` still appends. A is hidden; A' is the new effective slot-1 representative AND a new Unfinished Flight (it satisfies the predicate: visible, CP, Crashed, parent BP has RP-1).

```
ABC (I)                                  (V)
 |
 +-- AB  (I)                             (V)
 +-- A   (CP, Crashed, SUPERSEDED)       (H)
 +-- A'  (CP, Crashed)                   (V)  <- NEW Unfinished Flight
     RP-1 still persistent (slot-1 still open)
     RecordingSupersedes: {A -> A'}
     Unfinished Flights = {A'}
```

The player invokes RP-1 again. `PidSlotMap` still carries the original vessels' PIDs from the quicksave — that's the same quicksave, nothing about it changed. After strip + activate, a new provisional `A''` is created with `SupersedeTargetId = A'.Id` (note: targets `A'`, not `A`; the UI's "effective recording for slot-1" is A' now).

A'' lands safely. Merge. `TerminalKind = Landed`, so `A''.MergeState = Immutable`. Supersede relation `{A' -> A''}` appends. The chain is now `A -> A' -> A''`; `ChildSlot.EffectiveRecordingId(supersedes)` walks forward: start at A, find `{A -> A'}`, step to A', find `{A' -> A''}`, step to A'', no further relation → return A''. Slot-1's effective recording is A''. Both slots immutable → RP-1 reap-eligible → quicksave deleted, scenario entry removed.

```
ABC (I)                                  (V)
 |
 +-- AB  (I)                             (V)
 +-- A   (CP, H)                         (H)
 +-- A'  (CP, H)                         (H)
 +-- A'' (I, Landed)                     (V)  effective slot-1
     RP-1 reaped
     RecordingSupersedes: {A -> A'}, {A' -> A''}
     Unfinished Flights = {}
```

Key invariants shown:

1. The tree never shrinks. Every provisional stays around; only relations accumulate.
2. Supersede chains can extend arbitrarily through crashed re-flies.
3. Only `Immutable` closes the rewind opportunity; `CommittedProvisional` keeps the slot rewindable.
4. Career state reads ELS, which is unchanged in this example only because A had no reviewed recording-scoped career actions to tombstone.

4.3 Session-suppressed subtree with mixed-parent halt

The session-suppressed subtree is the physical-visibility carve-out applied during an active re-fly. It is a **forward-only** walk from the marker's `OriginChildRecordingId`, stopping at any descendant whose parents include a recording outside the subtree.

Suppose mission `ABC` decouples into `AB` and `C` (split-1). `AB` then decouples into `A` and `B` (split-2). Later, `B` docks with a station `S` from an unrelated tree; the dock merges B with S into a new recording `BS`.

```
ABC
 |
 +-- AB
 |    |
 |    +-- A
 |    +-- B ----+
 |              |
 +-- C          v
               BS  (parents = {B, S-tree-tip})
 S-tree:
   S (own lineage)
```

Player invokes RP at split-1 targeting C (C crashed). SessionSuppressedSubtree is the forward closure of `OriginChildRecordingId = C.Id`, which is just `{C}` because C has no descendants — that's fine.

Now suppose the player invokes RP at split-1 targeting AB (AB crashed — hypothetical). The closure starts at AB, walks to its descendants {A, B}, and would then consider BS. But BS has parents `{B, S-tree-tip}`, and `S-tree-tip` is NOT in the subtree — the closure halts here. `BS` stays visible in ERS during the session; the mixed-parent constraint means a descendant that has any outside parent is not session-suppressed. Without this halt, suppressing BS would also suppress S-tree activity that the player may be relying on.

Chain-sibling expansion. Each member added to the closure also expands to its chain siblings — committed recordings sharing both `ChainId` and `ChainBranch`. Merge-time `RecordingOptimizer.SplitAtSection` splits a single live recording at env boundaries (atmo↔exo) into a `ChainId`-linked HEAD + TIP where the HEAD keeps the parent-branch-point link and the TIP carries the terminal state. Without this expansion, walking by `ChildBranchPointId` alone would dequeue the HEAD, find no BP descendant (its `ChildBranchPointId` was moved to the TIP at split time), and stop — leaving the terminal-Destroyed TIP visible alongside the new "kerbal lived" provisional after merge. Different `ChainBranch` values stay independent (parallel ghost-only continuations are not auto-suppressed together). Same dual-key contract as `EffectiveState.IsChainMemberOfUnfinishedFlight` and `ResolveChainTerminalRecording`.

4.4 The 4-probe canonical example (v0.9.1 stable-leaf extension)

The motivating gameplay for the v0.9.1 broadened predicate:

```
Mun orbit, mothership M with 4 attached probes P1-P4.
Player triggers staging; all 4 decouplers fire simultaneously.
Multi-controllable split: M + P1 + P2 + P3 + P4 = 5 controllables.
RewindPointAuthor.Begin captures RP-1 with 5 ChildSlots.
RP-1.FocusSlotIndex = (slot index for M)  // the player's active vessel.
```

```
Player flies M back to Kerbin. P1-P4 background-record their parking coast.
Tree commits.
```

Under v0.9.0 alone: all 5 recordings commit `Immutable` (none crashed). `RewindPointReaper.IsReapEligible` sees all-slots-Immutable → reap RP-1. Quicksave deleted, BranchPoint.RewindPointId cleared. P1-P4 are inert rows in the Recordings table forever.

Under the v0.9.1 extension:

```
Site A (ApplyRewindProvisionalMergeStates) walks tree.Recordings:
  M:   focus slot, terminal Orbiting (Kerbin) -> not UF -> Immutable
  P1:  non-focus, terminal Orbiting (Mun)     -> UF        -> CommittedProvisional
  P2:  non-focus, terminal Orbiting (Mun)     -> UF        -> CommittedProvisional
  P3:  non-focus, terminal Orbiting (Mun)     -> UF        -> CommittedProvisional
  P4:  non-focus, terminal Orbiting (Mun)     -> UF        -> CommittedProvisional

RewindPointReaper.IsReapEligible: 4 slots are CP (open) -> RP-1 stays alive.
Recordings tab Unfinished Flights group shows P1, P2, P3, P4 with Fly + Seal buttons.
```

Player can pick any probe individually, hit Fly, rewind to the staging UT, and fly that probe to a real mission (land it, transfer it elsewhere, etc.). The other three probes ghost-play-back from their committed coast. Merge produces a re-fly that supersedes the original probe. Per the v0.9.1 auto-seal contract (§4.9): if the re-fly reaches a stable terminal (`Orbiting` , `Landed` , `Splashed`) on the chosen slot, OR authors a structural branch point during the session (decouple / stage / undock / joint break / EVA), the merge commits `Immutable` and auto-seals the slot — the player engaged this slot to fly it themselves and the merge concludes that engagement. `SubOrbital` is excluded: a suborbital arc is still in flight, so the slot stays open (`CommittedProvisional`) until the arc resolves. Recovers, docks, boards, downstream structural/world interaction, and sealing rows (any gain, contract outcome or flight-tagged spend, section 2.12) all still trigger the existing safety-close path. Destroyed outcomes stay `CommittedProvisional` for another retry unless the re-fly filed a sealing row. A probe whose original flight already earned something after the split (it transmitted science, completed a contract) is not offered at all (section 2.12 Rule 1). If the player decides one probe is "done" before re-flying it, they hit Seal and the row drops.

4.5 Focus exclusion: routine upper stages don't pollute the list

```
Two-stage rocket: booster B (probe core, parachutes) + upper stage U (player's mission).
Player flies U to orbit. B BG-records, parachutes auto-deploy, B lands safely.
Tree commits.

RP-1.FocusSlotIndex = (slot index for U)
B:  non-focus, terminal Landed   -> not UF (Landed is "stable conclusion")
U:  focus, terminal Orbiting     -> not UF (focus exclusion)

All slots Immutable -> RP reaps. Same as v0.9.0 behavior.
```

Inversely (the "spaceplane that recovers a strap-on booster" case):

```
Player flies B (the booster) to recover it. U (upper stage) BG-records, ends Orbiting.

RP-1.FocusSlotIndex = (slot index for B)
B:  focus, terminal Landed       -> not UF (focus exclusion + Landed)
U:  non-focus, terminal Orbiting -> UF -> CommittedProvisional
```

The booster is excluded as the focus mission; the upper stage is the unfinished off-mission sibling. RP stays alive while U's slot is open.

4.6 EVA carve-out (retired 2026-09-27)

This section used to describe the stranded-EVA carve-out: a kerbal branch in `TerminalOutcomeQualifies` , ahead of the focus short-circuit, that surfaced a non-boarded EVA kerbal as an Unfinished Flight even from legacy RPs. The owner ruling of 2026-09-27 retired it (§1.5). The kerbal gate still runs first, before the terminal is read, but it now rejects: an EVA kerbal (the chain tip or the subject carries `EvaCrewName`) returns `evaNotSeparation` whatever its terminal, `Destroyed` included, and a slot of a legacy RP authored at an EVA BranchPoint returns the same reason on either side. A player who regrets an EVA uses F9 or Discard.

4.7 Seal closes a slot by sealing its tip

The player Seals a slot via the per-row Seal button. The seal flips the slot's effective chain+supersede tip recording `CommittedProvisional -> Immutable` (open/closed is read from the tip `MergeState`, the single source of truth; the old separate `ChildSlot.Sealed` bit was collapsed into `MergeState`, see §2.2). Sealing only changes the tip's `MergeState`; the recording's trajectory data is untouched and it continues to play back as a ghost on any future rewind exactly as before. Only the re-fly opportunity for that slot is closed.

The reaper (§6.23) treats a slot as closed iff its effective tip is `Immutable` (or unresolved). A tip in `CommittedProvisional` (open) or `NotCommitted` (live recorder) keeps the RP alive; keeping `NotCommitted` open is the live-recorder guard so the quicksave is never deleted while a re-fly is still recording.

4.8 Tree-branching parent that ends Destroyed

A tree-branching split can capture a RewindPoint with both the continuing parent vessel and the new side-off child as controllable slots without closing the parent's Recording row. In that model, the parent keeps flying under the same `RecordingId` after the BranchPoint, so `Recording.ParentBranchPointId` and `Recording.ChildBranchPointId` can both stay empty even though the RP slot's `OriginChildRecordingId` points at the parent. If the parent later ends `Destroyed`, it is still an unfinished flight when its retire set holds no sealing row (section 2.12 Rule 1): destruction is conclusive and focus exclusion does not apply, but a sealing row still closes the retry row. The same origin-only route can surface a non-focus tree-branching parent that later ends `Orbiting` or `SubOrbital`, matching the stable-leaf focus-exclusion rules from §4.4-§4.5.

Resolution order is part of the contract. `UnfinishedFlightClassifier.TryResolveRewindPointForRecording` first walks the legacy child/parent BranchPoint links so chain-continuation recordings keep their existing precedence. Only after those walks fail does it scan live RewindPoints and call `EffectiveState.ResolveRewindPointSlotIndexForRecording` to match by slot origin id, choosing the latest matching RP by UT when the same parent recording appears in multiple origin slots. `TryQualify` labels this fallback `side=origin-only` when the matched RP is justified by the slot origin rather than by a parent/child BranchPoint match. The log line therefore distinguishes the new tree-branching-parent path from the existing `side=active-parent-child` chain-continuation parent and regular `side=child` branches.

The downstream-BranchPoint guard is naturally a no-op for this shape because the still-open parent is its own chain tip and has no `ChildBranchPointId`. The controllable gate still applies through the RP slot and `rec.IsDebris`, so debris breakup rows do not become Re-Fly candidates merely because they share a parent id. Site B-1 merge classification uses the same origin-slot idea against the marker's supersede target for separate provisional re-flights; in-place continuations keep their existing terminal-kind fallback policy.

4.9 Re-Fly auto-seal contract (v0.9.1)

The original v0.9.0 design held that a clean stable Re-Fly outcome stayed `CommittedProvisional` so the player could keep synchronizing the slot against other recordings. v0.9.1 playtesting showed this conflicts with the player's mental model: when the player explicitly *picks a slot to Re-Fly* and *flies it themselves*, the merge concludes their engagement with that slot. Leaving the slot open meant a successful re-fly to orbit still showed up as Unfinished and offered a "Fly" button the player did not want. Worse, it caused confusion about whether the original or the re-flown timeline was authoritative.

Revised contract. Site B-1 (re-fly merge) seals the slot (commits its tip `Immutable`) when ANY of these is true:

1. **Player-chosen slot reached a stable terminal** (`Orbiting`, `Landed`, `Splashed`, plus the existing hard-safety set `Recovered` / `Docked` / `Boarded`). The player engaged the slot to fly it; the merge concludes the engagement. `SubOrbital` is NOT in this set — a suborbital arc is still in flight (the vessel will crash, land, splash, or with a burn reach orbit), so the slot stays open until the arc resolves. The player can manually Stash + Seal a SubOrbital row.
2. **Structural mutation during the session** — the Re-Fly authored a new `BranchPoint` of type `Breakup`, `Undock`, `EVA`, or `JointBreak` in the provisional's tree. Decouple, stage, undock, joint break, kerbal EVA — these are explicit player-driven shape mutations the player wants preserved, regardless of where the chain tip ends.
3. **Sealing row in the re-fly's retire set** (section 2.12 Rule 1, replacing the earlier `ScienceEarning`-only gate). After the merge the re-fly is the slot's tip, so the rows it filed are what a later re-fly of the slot would retire. Any sealing row among them (every gain, including credited science, lab science, funds and reputation earnings, milestones, recoveries and rescues; every contract outcome; every flight-tagged spend) seals the slot, on any recording of the walked history (the vessel's segments, its own EVA crew's recordings and the parts they placed, section 1.6, and the rest of the merge closure). Only loss rows (crew deaths and bookkeeping, crew-death and vessel-loss reputation, building damage) leave the slot open. KSC-scene player decisions (`ContractAccept` / `Cancel`, `KerbalHire`, `FacilityUpgrade` / `Repair`, `StrategyActivate` / `Deactivate`, `ScienceSpending`) emit with no flight-recording tag and never reach the retire set; the rollout `FundsSpending(VesselBuild)` adopted by `TryAdoptRolloutAction` is never tombstone-eligible, so it is not in the retire set either. The reason keeps the existing prefix: `recordingAction:<Type>:<ActionId>`.

4. **Downstream structural/world interaction** — the chain tip's `ChildBranchPointId` resolves to a deeper RP, or (section 1.6) a crew member who went EVA from the vessel joined a foreign vessel (`evaCrewJoinedForeignVessel`). The chain tip is the vessel's real tip, read through its own EVA and re-board.

The slot **stays open** (tip `CommittedProvisional`) only on the terminal-failure outcome `Destroyed` with nothing but loss rows filed, so the player can retry the failure. Before 2026-10-19 automatic consequence rows also kept the slot open, on the reasoning that a later retry would retire them through the broad tombstone path; section 2.12 removed that, because the retired value could already have paid for a later Space Center purchase.

Site A is unaffected by this contract (it reads the section 2.12 value gate like every other arm, but not the focus override). The original-tree-commit predicate that surfaces stable-leaf siblings into Unfinished Flights does not know about Re-Fly intent; it continues to promote `stableLeafUnconcluded` and `stashedStableLeaf` rows so the player can come back to them. The auto-seal revision lives entirely in the Re-Fly merge call site (Site B-1 / Site B-2). Background vessels the player merely controlled and then switched away from at scene exit still take the Site A path and stay re-flyable.

Mechanics.

- *Focus override.* `UnfinishedFlightClassifier.TryQualify` accepts an optional `int? focusSlotOverride`. `SupersedeCommit.ClassifyMergeStateOrThrow` populates it with the merge-time `slotListIndex`. The override path runs **early** in `TerminalOutcomeQualifiesInternal` — after the Destroyed and EVA branches (so terminal-failure outcomes still take their existing retry-keep-open path) but **before** the stashed-keep-open branch and **before** the static-focus / `noFocusSignalOrbiting` checks. When the override matches the resolved slot AND the chain tip is in `{Orbiting, Landed, Splashed}`, the classifier runs the section 2.12 value gate first (so a sealing row's `recordingAction:*` reason still wins; loss rows are ignored) and otherwise returns `reason=stableTerminalFocusSlot`. `SubOrbital` is excluded from the override set: a suborbital arc falls through to `stableLeafUnconcluded` (qualifies as Unfinished Flight, slot kept open). Recovered/Docked/Boarded fall through to the existing `stableTerminal` close + `IsHardSafetyTerminal` auto-seal path because they are already correctly handled there. The verdict log emits `focusSlot=N focusSlotOverride=M` (separate fields) so the source of the focus decision is greppable in both `[Supersede]` and `[UnfinishedFlights]` lines.
- *Override-before-stashed.* The override precedes the stashed-keep-open branch by design: a player who Re-Flies a stashed slot to a stable conclusion is concluding the stash, not extending it. Otherwise stashed slots would always escape the override and remain re-flyable indefinitely.
- *Override-before-noFocusSignal.* The override precedes the `rp.FocusSlotIndex < 0` early return because the override IS the focus signal for this merge. Without this ordering, a Re-Fly merge from an RP captured with no static focus (player was controlling an unrelated vessel at split time) would log `noFocusSignalOrbiting` and never reach the override seal.
- *`stableTerminalFocusSlot` auto-seals slot.* `SupersedeCommit.ShouldAutoSealReFlySlotAfterMerge` commits the provisional tip `Immutable` for `closeReason.Detail == "stableTerminalFocusSlot"` on both the static-focus and override paths.
- *Structural-mutation gate.* `SupersedeCommit.HasReFlySessionStructuralMutation(provisional, marker)` walks the provisional's tree (resolved via `marker.TreeId`) for any `BranchPoint` with `Type ∈ {Breakup, Undock, EVA, JointBreak}` (a debris-only `Breakup` / `JointBreak` excepted, section 1.7) whose `UT > rp.UT` (the resolved rewind-point UT, NOT `marker.InvokedUT` — the latter is the live UT at click time, typically much greater than rp.UT and would reject normal post-rewind decouples as if pre-rewind). The gate excludes any BP whose id is in the marker's new `PreSessionBranchPointIds` baseline (snapshot at marker creation, prevents `SpliceMissingCommittedRecordingsIntoLoadedTree` false positives from re-grafted pre-RP BPs). The gate also scopes detection to BPs whose `ParentRecordingIds` intersects the **Re-Fly target's lineage set** — the provisional's recording id plus every committed recording sharing `(TreeId, ChainId, ChainBranch)` with the provisional. Without this lineage scope, a background sibling vessel in the same tree that staged / undocked during the session would author a structural BP whose parent is the sibling, not the player-chosen slot, and the gate would auto-seal a slot the player never mutated. Legacy markers with `PreSessionBranchPointIds=null` skip the gate conservatively. Detection fires inside `ShouldKeepReFlySlotOpenAfterMerge` after the existing `IsTerminalFailureReFlyOutcome` keep-open branch, so crashed retries are unaffected.
- *Reachability under the focus override.* The override path closes every stable-terminal Re-Fly outcome on the player-chosen slot (Orbiting / Landed / Splashed) before the structural gate runs, so the structural-mutation gate is a defensive backstop for those terminals. For `SubOrbital` the override does not fire (suborbital arc is still in flight), so the structural gate is the PRIMARY (and only) seal trigger: a SubOrbital re-fly that decoupled / staged / undocked / EVA'd seals via `structuralMutation:*` rather than `stableTerminalFocusSlot`. The structural gate remains testable at the helper level and would still fire if a future call site ever invoked `ShouldKeepReFlySlotOpenAfterMerge` from a path that does not pass the override.

What does NOT count as structural mutation. Detection is on `BranchPointType`, not on `Recording.PartEvents`. Solar panel deploy, ladder extend, landing leg toggle, cargo bay open/close, control surface deflect, animation toggle, gear lower/raise — none of those author a `BranchPoint` and none of them auto-seal. Branch points are only authored when KSP changes vessel topology (decouple / stage / undock / joint break / EVA / dock / board / launch / terminal); only the structural-split subset closes the slot.

Reap consequence. An auto-sealed slot may make its rewind point reap-eligible. `MergeDialog.TryCommitReFlySupersede`'s post-merge reap call deletes the RP's quicksave file when all slots in that RP are closed. A successful Re-Fly to stable orbit, or a Re-Fly that staged debris, cannot be re-flown again afterward — the on-disk file is gone. This is the intended outcome ("player is done with that line of flight") but is irreversible for that session.

---

5. Data Model

Every class in this section lives in `Source/Parsek/`. ConfigNode VALUE keys are listed alongside the node name so the save-file layout is self-documenting.

5.1 MergeState

File: `Source/Parsek/MergeState.cs`.

```
public enum MergeState
{
    NotCommitted = 0,
    CommittedProvisional = 1,
    Immutable = 2
}
```

Tri-state replacement of the legacy binary `committed` flag, which is no longer read. An absent `mergeState` reads as `Immutable` (the pre-feature invariant — every recording reachable from a committed tree was already sealed). Serialized as `mergeState = <value>` on the RECORDING node.

- `NotCommitted`: recording is still being produced (recorder live, or re-fly provisional with `SupersedeTargetId` set).
- `CommittedProvisional`: session merge has stamped the recording but the rewind slot remains available (BG-recorded children under an RP; re-fly that ended in a crash and is still re-rewindable).
- `Immutable`: recording is sealed. Never mutated after this point; the rewind slot is closed.

5.2 Recording additions

File: `Source/Parsek/Recording.cs:193–208`.

Field	Type	Purpose
`MergeState`	`MergeState`	Tri-state commit state (§5.1).
`CreatingSessionId`	`string`	Session GUID for recordings produced during an active re-fly. `null` outside sessions. Used by the load-time spare-set logic (§6.16) when a session crashed.
`SupersedeTargetId`	`string`	Transient — set only on `NotCommitted` provisional re-fly recordings to signal the intended supersede target. Cleared at merge when a concrete `RecordingSupersedeRelation` is appended. A non-empty value on `Immutable` / `CommittedProvisional` recordings triggers a Warn log and is treated as cleared (load-path legacy safety; see §6.16 step 5 and `LoadTimeSweep.ClearStraySupersedeTargets`).
`ProvisionalForRpId`	`string`	Back-pointer to the invoked RP's id. Set at invocation by `RewindInvoker.BuildProvisionalRecording` (`RewindInvoker.cs:527`). Null outside sessions.

`SupersededByRecordingId` is **NOT** a field on Recording. Visibility is a derived lookup through `RecordingSupersedeRelation`.

5.3 BranchPoint additions

File: `Source/Parsek/BranchPoint.cs:47`.

Field	Type	Purpose
`RewindPointId`	`string`	The id of the RP attached to this BranchPoint, or `null` if no RP was written (fewer than two controllable children, scene-guard abort, save failure). Stamped synchronously by `RewindPointAuthor.Begin` (`RewindPointAuthor.cs:138`).

Serialized as `rewindPointId = <value>` on the BRANCH_POINT node when non-null.

5.4 GameAction additions

File: `Source/Parsek/GameActions/GameAction.cs`.

Field	Type	Purpose
`ActionId`	`string`	Stable, immutable `act_<Guid-N>` id, written with every row. Precondition for tombstoning.

5.5 RewindPoint

File: `Source/Parsek/RewindPoint.cs`.

Field	Type	Purpose
`RewindPointId`	`string`	Stable id in the format `rp_<Guid-N>`. Must satisfy `RecordingPaths.ValidateRecordingId`.
`BranchPointId`	`string`	Weak link back to the `BranchPoint`.
`UT`	`double`	`Planetarium.UT` at quicksave write.
`QuicksaveFilename`	`string`	Relative path under the save dir; format `Parsek/RewindPoints/<rpId>.sfs`.
`ChildSlots`	`List<ChildSlot>`	One entry per controllable sibling at split time.
`PidSlotMap`	`Dictionary<uint, int>`	`Vessel.persistentId` → `ChildSlot.SlotIndex`. Primary match for post-load strip.
`RootPartPidMap`	`Dictionary<uint, int>`	Root `Part.persistentId` → `ChildSlot.SlotIndex`. Fallback when KSP reassigned a vessel persistentId between save and load.

Field	Type	Purpose
`SessionProvisional`	`bool`	`true` while the creating session still owns the RP. Normal split RPs created outside a re-fly session remain `true` with `CreatingSessionId == null` only until their owning tree is accepted; load-time sweep retains them during merge-dialog scene loads, and the tree-commit path promotes them to persistent. Session-scoped RPs flip `false` at `TagRpsForReap` during merge.
`Corrupted`	`bool`	`true` when quicksave validation failed (file missing at deferred save, all slots undetectable, PartLoader deep-parse failed). RP kept for diagnostic visibility; Rewind button disabled.
`CreatingSessionId`	`string`	Session GUID when the RP was created inside an active session; `null` otherwise.
`CreatedRealTime`	`string`	Wall-clock ISO-8601 UTC timestamp.
`FocusSlotIndex`	`int`	(v0.9.1) Slot index (0-based, into `ChildSlots`) of the vessel that was the active focus at split time. `-1` means "no focus signal available" — either a legacy RP that predates this field, OR a new RP where no slot was focused at split time (rare: e.g. background joint break, or a split where the player was focused on an unrelated vessel outside the split). Used by `IsUnfinishedFlight`'s `TerminalOutcomeQualifies` to gate stable-terminal qualification: focus-continuation slots are excluded from auto-UF for `Orbiting` / `SubOrbital`. `Crashed` terminals qualify regardless of focus (an EVA kerbal never qualifies, §1.5). `FocusSlotIndex == -1` short-circuits Orbiting/SubOrbital to false (the conservative choice when no focus signal is available), preserving v0.9.0 Crashed-only behavior for legacy RPs and avoiding retroactive flooding from background-only splits. The Re-Fly merge auto-seal contract (§4.9) treats the merge-time slot as `focusSlotOverride`, which routes around this static field.

Node layout (emitted by `RewindPoint.SaveInto`, `RewindPoint.cs:75`):

```
POINT
{
    rewindPointId = rp_...
    branchPointId = bp_...
    ut = 1742538.43
    quicksaveFilename = Parsek/RewindPoints/rp_...sfs
    sessionProvisional = True|False
    corrupted = True             # omitted when False
    creatingSessionId = sess_... # omitted when empty
    createdRealTime = 2026-04-17T21:35:12Z
    focusSlotIndex = 2           # (v0.9.1) omitted when -1

    CHILD_SLOT { slotIndex = 0  originChildRecordingId = rec_... controllable = True }
    CHILD_SLOT { slotIndex = 1  originChildRecordingId = rec_... controllable = True }

    PID_SLOT_MAP      { pid = 12345678  slot = 0 }
    PID_SLOT_MAP      { pid = 87654321  slot = 1 }

    ROOT_PART_PID_MAP { pid = 11223344  slot = 0 }
    ROOT_PART_PID_MAP { pid = 99887766  slot = 1 }
}
```

5.6 ChildSlot

File: `Source/Parsek/ChildSlot.cs`.

Field	Type	Purpose
`SlotIndex`	`int`	Zero-based position within `RewindPoint.ChildSlots`.
`OriginChildRecordingId`	`string`	Immutable id of the recording originally created for this slot at split time.
`Controllable`	`bool`	Reserved for future classifier churn; currently always `true` (only controllable entities produce slots).
`Disabled`	`bool`	`true` if the Rewind button for this slot must be grayed out (PidSlotMap lookup failed for this slot at split time, or a loader sanity check failed).
`DisabledReason`	`string`	Human-readable reason shown in the UI tooltip. Current reasons: `no-live-vessel`.
~~`Sealed`~~ / ~~`SealedRealTime`~~	(removed)	The original v0.9.1 per-slot seal bit + timestamp were removed by collapse-seal-into-mergestate. Open/closed is now read from the slot's effective tip `MergeState` (open iff `CommittedProvisional`, sealed iff `Immutable`). Seal flips the tip to `Immutable`; there is no in-game un-seal path (tree-scoped Full-Revert is the only undo). See §2.2 / §6.24.
`Stashed`	`bool`	(v0.9.1) `true` when the player invoked the per-row Stash action on this slot, promoting a default-excluded stable terminal leaf into Unfinished Flights. Only spawnable stable terminals (`Landed`, `Splashed`, `Orbiting`, `SubOrbital`) qualify; `Recovered`, `Docked`, `Boarded`, and rows whose retire set holds a sealing row (section 2.12) are rejected by the resolver. Default `false`; legacy saves load with `false`. Stash is only possible while the backing RewindPoint still exists; it does not resurrect already-reaped RP quicksaves.
`StashedRealTime`	`string`	(v0.9.1) Wall-clock ISO-8601 UTC timestamp the Stash was applied. Diagnostic only; null when `Stashed` is false.

Derived:

- `EffectiveRecordingId(IReadOnlyList<RecordingSupersedeRelation>)` delegates to `EffectiveState.EffectiveRecordingId` (`EffectiveState.cs:78`). Forward walk from `OriginChildRecordingId`, cycle-guarded via visited set, returns the last-reached id when no further relation matches.

Node layout:

```
CHILD_SLOT
{
    slotIndex = 0
    originChildRecordingId = rec_...
    controllable = True
    disabled = True              # omitted when False
    disabledReason = no-live-vessel  # omitted when null
    sealed = True                # (v0.9.1) omitted when False
    sealedRealTime = 2026-...    # (v0.9.1) omitted when null
    stashed = True               # (v0.9.1) omitted when False
    stashedRealTime = 2026-...   # (v0.9.1) omitted when null
}
```

5.7 RecordingSupersedeRelation

File: `Source/Parsek/RecordingSupersedeRelation.cs`.

Field	Type	Purpose
`RelationId`	`string`	Stable id in the format `rsr_<Guid-N>`.
`OldRecordingId`	`string`	The superseded recording (stays in `RecordingStore`, hidden from ERS).
`NewRecordingId`	`string`	The superseding recording (the re-fly provisional promoted on merge).

Field	Type	Purpose
`UT`	`double`	`Planetarium.UT` at which the merge occurred.
`CreatedRealTime`	`string`	Wall-clock ISO-8601 UTC.

Stored in `ParsekScenario.RecordingSupersedes`. Append-only during normal play. Removed by whole-tree discard (`TreeDiscardPurge.PurgeTree`, §6.17), and by load-time cleanup when a row is fully orphaned (both endpoint ids are missing from `RecordingStore`). One-sided orphan relations stay in place with a Warn log on every load (`LoadTimeSweep.cs`) because an old-present/new-missing row still suppresses the retired old recording; the forward walk in `EffectiveState.EffectiveRecordingId` handles an unresolved `NewRecordingId` as a chain terminator.

Node layout (emitted as `ENTRY` children of the `RECORDING_SUPERSEDES` parent node):

```
ENTRY
{
    relationId = rsr_...
    oldRecordingId = rec_...
    newRecordingId = rec_...
    ut = 1742800.00
    createdRealTime = 2026-04-17T22:10:00Z
}
```

5.8 LedgerTombstone

File: `Source/Parsek/GameActions/LedgerTombstone.cs`.

Field	Type	Purpose
`TombstoneId`	`string`	Stable id in the format `tomb_<Guid-N>`.
`ActionId`	`string`	The superseded action's immutable `GameAction.ActionId`. Aliases the design-spec's `SupersededActionId`; the code uses `ActionId` to match the field it refers to.
`RetiringRecordingId`	`string`	The new recording whose merge caused this tombstone to be written. Under section 2.12 Rule 2 (variant B) a merge also tombstones some of the re-fly's OWN reputation-penalty rows; those carry the re-fly's recording id here too, so every writer-side rule (sections 6.14, 6.16) treats them as the attempt's rows.
`UT`	`double`	`Planetarium.UT` at which the merge occurred.
`CreatedRealTime`	`string`	Wall-clock ISO-8601 UTC.

Stored in `ParsekScenario.LedgerTombstones`. Append-only. Multiple tombstones for the same `ActionId` are tolerated — the ELS filter is "at least one tombstone exists." Removed only by whole-tree discard when the underlying action's recording is in the discarded tree.

Node layout (emitted as `ENTRY` children of `LEDGER_TOMBSTONES`):

```
ENTRY
{
    tombstoneId = tomb_...
    actionId = act_...
    retiringRecordingId = rec_...
    ut = 1742800.00
    createdRealTime = 2026-04-17T22:10:00Z
}
```

5.9 ReFlySessionMarker

File: `Source/Parsek/ReFlySessionMarker.cs`.

Singleton marker present in `ParsekScenario` iff a re-fly session is live. The v0.9.0 release defined seven fields (six validated, one informational); v0.9.1 adds `SupersedeTargetId` (weakly validated) and `PreSessionBranchPointIds` (informational baseline for the structural-mutation gate).

Field	Validated?	Purpose
`SessionId`	yes	Unique GUID per invocation / retry.
`TreeId`	yes	RecordingTree this re-fly belongs to. Must exist in `RecordingStore.CommittedTrees`.
`ActiveReFlyRecordingId`	yes	The `NotCommitted` provisional re-fly recording. Must resolve in `CommittedRecordings` with `MergeState == NotCommitted`. It keeps naming the fork the recorder started on at the rewind point for the whole session, even after an EVA or a Switch-To ends that fork at a branch point and the vessel flies on as a continuation: the continuation is ordinary `Immutable` data, so a re-pointed marker would fail this check on the next load. The merge resolves the vessel's real tip itself (section 6.12).
`OriginChildRecordingId`	yes	The slot's IMMUTABLE origin id. Must resolve in `CommittedRecordings`. Existing consumers (`RevertInterceptor.FindSlotForMarker`, in-place continuation, ghost suppression) key off this immutable origin.
`SupersedeTargetId`	weak (v0.9.1)	The slot's CURRENT EFFECTIVE recording id at invocation time — equals `slot.EffectiveRecordingId(supersedes)` (= prior tip on chain extension; = slot origin on first re-fly). Used by `SupersedeCommit.AppendRelations` as the root of the subtree closure walk so chain extension produces a linear graph instead of a star. Always written by post-feature invocations; legacy markers load with null and `AppendRelations` falls back to `OriginChildRecordingId`. `MarkerValidator.Validate`: when non-null, must resolve in `CommittedRecordings` or the matching pending tree; failure logs `[ReFlySession] Warn: Marker invalid field=SupersedeTargetId; clearing` and clears the field. Null is always valid. See §6.22.
`RewindPointId`	yes	The invoked RP's id. Must resolve in `ParsekScenario.RewindPoints`.
`InvokedUT`	yes	`Planetarium.UT` at invocation. Must not be strictly greater than the current UT.
`InvokedRealTime`	**no**	Wall-clock ISO-8601 UTC. Informational; purely for human-readable logs and diagnostics.
`PreSessionBranchPointIds`	**no** (v0.9.1)	Snapshot of every `BranchPoint.Id` present in the marker's tree at the moment the marker was created. Consumed by `SupersedeCommit.HasReFlySessionStructuralMutation` as a session-local baseline so post-RP structural BPs that the load-time `SpliceMissingCommittedRecordingsIntoLoadedTree` path re-grafts back into the loaded tree are excluded from the gate. Without it, any old structural BP from the original future would auto-seal a stashed slot the player never mutated. Populated by `RewindInvoker.SnapshotTreeBranchPointIds(treeId)` called inline in `AtomicMarkerWrite`. Legacy markers persisted before this field shipped load with `null`; the structural-mutation gate skips on those markers so in-flight sessions at upgrade time preserve the pre-fix keep-open behavior on merge.

Persisted as a single `REFLY_SESSION_MARKER` ConfigNode on `ParsekScenario`:

```
REFLY_SESSION_MARKER
{
    sessionId = sess_...
    treeId = tree_...
    activeReFlyRecordingId = rec_...
    originChildRecordingId = rec_...
    supersedeTargetId = rec_...                # (v0.9.1) always written post-feature
    rewindPointId = rp_...
    invokedUT = 1742800.00
    invokedRealTime = 2026-04-17T22:10:00Z
```

```
    preSessionBranchPointIdsPresent = true      # (v0.9.1) presence sentinel
    preSessionBranchPointId = bp_...            # (v0.9.1) repeated value entries
    preSessionBranchPointId = bp_...
}
```

The `preSessionBranchPointIdsPresent` sentinel distinguishes "post-fix marker on a tree that had no BPs at invocation" (sentinel set, list empty — gate runs and treats every current BP as session-authored) from "legacy marker from before this field shipped" (sentinel absent — gate conservatively skips). The reader populates the list only when the sentinel is set.

Cleared on: re-fly merge (after Durable Save #1), re-fly discard (return-to-Space-Center), retry (a fresh marker with a new `SessionId` replaces the old one), full-revert (tree discard clears it if scoped to the discarded tree), load-time validation failure (cleared + Warn-logged).

5.10 MergeJournal

File: `Source/Parsek/MergeJournal.cs`.

Singleton journal present on `ParsekScenario` only during a staged-commit merge.

Field	Type	Purpose
`JournalId`	`string`	Per-run id `mj_<Guid-N>`. Addition to the pre-impl design spec for log readability.
`SessionId`	`string`	Merge session GUID (matches the retiring marker's `SessionId`).
`Phase`	`string`	One of the `Phases` constants (below).
`StartedUT`	`double`	`Planetarium.UT` at merge start.
`StartedRealTime`	`string`	Wall-clock ISO-8601 UTC.
`LossSwapVariant`	`string`	(Section 2.12 Rule 2, designed 2026-10-19.) `A` or `B`, chosen by the pre-Begin dominance check (section 6.12 step 0) and read by the Tombstone step and its crash-recovery re-run so both write the same set. Optional; absent reads as `A`, the only behaviour before the rule. Persisted as `lossSwapVariant` (omitted when `A`). Not a recording-schema change.

The `Phases` vocabulary (`MergeJournal.cs:53`):

Phase string	What has happened on disk	Recovery on load
`Begin`	Journal written, together with everything `MergeCommit` did before `RunMerge`: the attempt's tree committed (fork, EVA kerbals, payloads, debris), its ledger rows filed, and the in-place fork migrated into the committed tree.	Forward-completion, resumed as `TreeMerge`.
`TreeMerge`	`MigrateActiveReFlyForkIntoCommittedTree` ran + durably persisted: the in-place-continuation fork's RECORDING node migrated from the active tree into the committed tree; `tree.ActiveRecordingId` promoted to the fork id. Gated on `marker.InPlaceContinuation`.	Forward-completion (idempotent re-run via `MigrateActiveReFlyForkIntoCommittedTree`; `AddOrReplaceRecording` overwrites by id, `ActiveRecordingId` swap is value-write idempotent).
`Split`	`SplitOriginAtRewindUT` ran + durably persisted: origin split into HEAD + TIP if it spanned the rewind UT, `marker.SupersedeTargetId` rewritten to TIP, `tree.ActiveRecordingId` promoted from HEAD to TIP.	Forward-completion (idempotent re-run via `SplitOriginAtRewindUT`; existing TIP is detected and skipped).

Phase string	What has happened on disk	Recovery on load
`Superseded`	Supersede relations half-written or fully written in memory.	Forward-completion (`AppendRelations` skips existing rows).
`Tombstone`	Tombstones half-written or fully written in memory.	Forward-completion (`CommitTombstones` dedups via `alreadyTombstoned`).
`Finalize`	MergeState flipped + transient cleared in memory.	Forward-completion (`FlipMergeStateAndClearTransient` is value-write idempotent).
`Durable1Done`	First durable save flushed: supersedes + tombstones + MergeState on disk. Marker + RPs still live.	Complete from here (tag RPs → reap → clear marker → ...).
`RpReap`	Session-prov RPs tagged + reaped. Marker still live.	Complete (clear marker → ...).
`MarkerCleared`	Marker cleared in memory.	Complete (Durable Save #2).
`Durable2Done`	Second durable save flushed.	Complete (clear journal → Durable Save #3).
`Complete`	Journal set to `Complete` ; Durable Save #3 interrupted before clear.	Idempotent clear.

Every phase drives forward. `MergeJournal.IsKnownPostBeginPhase` (and its twin `IsPostDurablePhase`) returns true for every phase but `Begin` (`TreeMerge` / `Split` / `Supersede` / `Tombstone` / `Finalize` / `Durable1Done` / `RpReap` / `MarkerCleared` / `Durable2Done` / `Complete`). Each post-Begin step is idempotent (`MigrateActiveReFlyForkIntoCommittedTree` 's `AddOrReplaceRecording` + `ActiveRecordingId` writes are value-write idempotent, `SplitOriginAtRewindUT` 's idempotency check detects an existing TIP and skips re-creation, `AppendRelations` skips existing rows, `CommitTombstones` dedups, `FlipMergeStateAndClearTransient` is value-write idempotent), so forward-completion via re-run is safer than rollback (which would otherwise have to undo serialized state mutations after the durable barrier). The `TreeMerge` migration (shipped in `fix-refly-abandon-and-fork-persist`) runs before the `Begin` durable save, so the fork's RECORDING node persists into the committed tree with the journal and before the splitter runs; the `TreeMerge` phase string stays for the finisher's dispatch. The `Split` phase (shipped in Task A4 + A5) sits between `TreeMerge` and `Supersede` with its own `DurableSave("split", persistSynchronously: true)` barrier, isolating the in-memory split's transactional snapshot rollback (handled internally by `RecordingTreeSplitter`) from the post-durable forward-completion path. Phase count 11; durable-save barrier count 5 (`begin` , `split` , `durable1` , `durable2` , `durable3`). An unknown phase string (a newer build's) re-runs the whole tail from `TreeMerge` , every step being idempotent and safe on a cleared marker.

Persisted as a single `MERGE_JOURNAL` ConfigNode on `ParsekScenario` :

```
MERGE_JOURNAL
{
    journalId = mj_...
    sessionId = sess_...
    phase = Begin
    startedUT = 1742800.00
    startedRealTime = 2026-04-17T22:10:00Z
}
```

5.11 ParsekScenario additions

The scenario module owns five new persistent collections, each wrapped in its own parent node on save (`ParsekScenario.OnSave` ; verified at `Source/Parsek/ParsekScenario.cs:716–825`):

```
PARSEK
{
    ...
    REWIND_POINTS        { POINT { ... } POINT { ... } ... }
    RECORDING_SUPERSEDES { ENTRY { ... } ENTRY { ... } ... }
    LEDGER_TOMBSTONES    { ENTRY { ... } ENTRY { ... } ... }
    REFLY_SESSION_MARKER { ... }  # present only during active session
    MERGE_JOURNAL        { ... }  # present only during staged-commit merge
    ...
}
```

All five sections are additive — a save written before v0.9 simply has none of them, and `OnLoad` treats the absence as empty lists / null singletons. Saves written by v0.9 and loaded on v0.8.x would fail cleanly because the pre-v0.9 scenario does not know these keys; no downgrade path is provided.

Runtime-only state counters on the scenario drive the ERS / ELS cache invalidation:

Counter	Bumped by
`SupersedeStateVersion`	Every mutation to `RecordingSupersedes`, marker identity change, MergeState flip.
`TombstoneStateVersion`	Every mutation to `LedgerTombstones`.
`RecordingStore.StateVersion`	Every mutation to `CommittedRecordings`.
`Ledger.StateVersion`	Every mutation to `Actions`.

5.12 Directory layout

```
saves/<save>/
    persistent.sfs                          (KSP standard)
    Parsek/
        Recordings/
            <recordingId>.prec              (existing, unchanged)
            <recordingId>_vessel.craft      (existing, unchanged)
            <recordingId>_ghost.craft       (existing, unchanged)
            <recordingId>.pcrf              (existing, unchanged)
        RewindPoints/
            <rewindPointId>.sfs             (NEW; KSP-format quicksave)
    Parsek_Rewind_<sessionId>.sfs           (NEW; transient, root-level)
```

The transient `Parsek_Rewind_<sessionId>.sfs` at save-root exists only between `RewindInvoker.StartInvoke` (which copies `Parsek/RewindPoints/<rpId>.sfs` to this location) and the post-load cleanup in `RewindInvoker.ConsumePostLoad` (`RewindInvoker.cs:453`). The copy is required because `GamePersistence.LoadGame(fileName, folder, ...)` does not accept subdirectory paths. The cleanup runs in a `finally` block regardless of success, so a crash mid-invocation leaves at most one stale copy that the next invocation overwrites.

5.13 Unfinished Flights virtual UI group

File: `Source/Parsek/UI/UnfinishedFlightsGroup.cs`.

Not stored in `GroupHierarchyStore`. Membership derived per frame from ERS filtered by `EffectiveState.IsUnfinishedFlight`. As a system group it **cannot be hidden** and **cannot be a drop target** for manual group assignment. Rename on an individual member row still persists through the standard `Recording.VesselName` path; hide on a member row warns and refuses via `ParsekLog` → `ScreenMessages` advisory.

The virtual group is not the only safe affordance. The same recording can also appear in the normal recordings table, where its tree root still carries a legacy launch rewind save. RP-backed unfinished-flight rows therefore preempt the normal `RecordingStore.CanRewind` / `InitiateRewind` path and resolve the row's child slot before any legacy fallback is considered.

---

6. Behavior

6.1 Effective Recording Set (ERS)

File: `Source/Parsek/EffectiveState.cs:236`.

```
ERS = { r in RecordingStore.CommittedRecordings
      : r.MergeState != NotCommitted
        AND IsVisible(r, ParsekScenario.RecordingSupersedes)
        AND (activeMarker == null OR r.RecordingId not in SessionSuppressedSubtree(activeMarker)) }

IsVisible(r, supersedes) :=
    walk forward from r.RecordingId via supersedes,
    return true iff the walk terminates at r.RecordingId itself.
```

Cached. Rebuild triggered by any change to `RecordingStore.StateVersion`, `ParsekScenario.SupersedeStateVersion`, or the active `ReFlySessionMarker` identity. Every rebuild emits a single `[ERS]` Verbose line:

```
[Parsek][Verbose][ERS] Rebuilt: <N> entries from <raw> committed
(skippedNotCommitted=<k> skippedSuperseded=<s> skippedSuppressed=<u>
marker=<sessionId|none>)
```

6.2 Effective Ledger Set (ELS)

File: `Source/Parsek/EffectiveState.cs:310`.

```
ELS = { a in Ledger.Actions : a.ActionId not in { t.ActionId for t in ParsekScenario.LedgerTombstones } }
```

One filter only: tombstones. There is no recording-level filter. A superseded recording's action remains in ELS unless its `ActionId` has an explicit `LedgerTombstone`. The broad merge predicate decides which reviewed recording-scoped career actions receive tombstones; seeds, null-scoped rows, rollout costs, and unknown future action types remain visible until reviewed.

Cached. Rebuild triggered by any change to `Ledger.StateVersion` or `ParsekScenario.TombstoneStateVersion`. Every rebuild emits a single `[ELS]` Verbose line with the count and skipped-tombstoned counter.

6.3 Session-suppressed subtree (physical-visibility only)

Computed by `EffectiveState.ComputeSessionSuppressedSubtree(marker)`. Forward-only closure from `marker.OriginChildRecordingId`:

```
closure(r) = { r }
  for each child c whose ParentRecordingIds contain r:
    if c.ParentRecordingIds is a subset of the current closure:
      include c (and recurse)
    else:
      stop — c has a parent outside the subtree (mixed-parent halt)
```

The same-PID gate, chain-sibling, PID-peer and debris expansions are unchanged. Owner ruling 2026-09-27 (section 1.6) adds four rules, all logged in the closure summary line (`stretchSeeds=`, `evaKerbalsAdded=`, `sameVesselBoards=`, `placedPartsAdded=`) and per admission (`admitted own-EVA kerbal child=`, `admitted same-vessel board child=`, `admitted kerbal side-branch child=`):

- **Own EVA crew.** At an `EVA` branch point whose parent is the dequeued VESSEL recording, the kerbal child (an `EvaCrewName` recording, committed, not the live session's own, split at or after the rewind UT) is admitted although its pid differs (`EffectiveState.IsOwnEvaKerbalClosureChild`, a pure gate that names the failed condition).
- **Same-vessel Board.** A `Board` child that carries the pid of a VESSEL parent already in the closure is admitted even when the kerbal parent is the one dequeued, so the order in which the vessel and the kerbal leave the queue does not matter. A `Board` with a parent outside the closure still halts (mixed-parent).
- **Placed parts and cargo partners.** A kerbal in the closure pulls in the children of his kerbal side branches, found by parent id: `GroundPartPlaced` (a part he placed) and `InventoryTransfer` (a foreign vessel whose inventory he changed, flight recorder design section 4.12). Both count in `placedPartsAdded=`.

- **Stretch seeding.** A closure rooted anywhere but the slot origin (the merge-time root is the marker's `SupersedeTargetId`, the walked tip) is seeded with the own-vessel stretch walked backward from the root (`EffectiveState.CollectOwnVesselStretchBackward`): back across switch-continuation, own-EVA and same-vessel-Board points to the slot origin, stopping at a re-fly fork and before any recording whose sampled content starts before `marker.RewindPointUT` (a spanning recording belongs to `RecordingTreeSplitter`, which now also looks for it along the stretch). Skipped for a legacy marker with no `RewindPointUT`.

A kerbal who goes EVA DURING the session is not a supersede source: his recording is the session's own and is not a committed closure member when the merge runs.

Consumed by physical-visibility subsystems:

- **Ghost playback engine** (`GhostPlaybackEngine.UpdatePlayback`) — per-frame filter with a `sessionSuppressed=<n>` counter in the frame summary log; destroys leftover ghost visuals on session entry.
- **Ghost chain walker** (`GhostChainWalker`) — suppressed recordings do not claim chain tips. Aggregated skip log.
- **Ghost map presence** (`GhostMapPresence`) — create-entry-points gate + per-tick prune of suppressed presence.
- **Watch mode** (`WatchModeController`) — refuses entry on suppressed anchor; exits on session-start if the current anchor is suppressed.

Subsystems that read ERS / ELS directly and do NOT apply the session-suppressed filter:

- Ledger recalculation (`LedgerOrchestrator`) reads ELS directly via `EffectiveState.ComputeELS()`.
- Career-state UI (Contracts / Strategies / Facilities / Milestones tabs).
- Crew reservation manager (`CrewReservationManager`), with a narrow live-re-fly-crew carve-out (see §6.4).
- Contract / milestone / facility state derivation (KSP-owned, reconciled only through the post-tombstone recalc/patch path when reviewed recording-scoped ledger rows are retired).
- Timeline view.
- Unfinished Flights membership.
- RewindPoint reap logic.

Exactly one `[ReFlySession] Start` / `End` log line is emitted per transition via `SessionSuppressionState` (`Source/Parsek/SessionSuppressionState.cs`), which observes marker-identity changes and logs once per transition.

6.4 Kerbal dual-residence carve-out (during active session)

`CrewReservationManager.IsLiveReFlyCrew(pcm, marker)` exempts kerbals physically embodied in the live re-fly vessel from reservation lock. Without this carve-out, a kerbal whose ledger `KerbalAssignment`-to-Dead event is still in ELS (tombstone not yet written — that happens at merge) would be reservation-locked as dead, silently blocking EVA or crew transfer during the re-fly.

Other reservation effects (kerbals reserved to unrelated still-ghost recordings, stand-in generation for other unreserved slots) remain active.

6.5 Multi-controllable split detection

Every joint-break, undock, and EVA event funnels through `SegmentBoundaryLogic.ClassifyJointBreakResult` and its siblings (an EVA is classified like any split for the recording DAG, then refused at RP authoring, below). `SegmentBoundaryLogic.IdentifyControllableChildren` (`SegmentBoundaryLogic.cs:235`) enumerates the post-split vessels and counts the ones that have a `ModuleCommand` part or are EVA kerbals. `SegmentBoundaryLogic.IsMultiControllableSplit(count)` is a one-line gate: `count >= 2`.

When the gate passes, `ParsekFlight.CreateSplitBranch` / `ProcessBreakupEvent` / the EVA handler invokes `RewindPointAuthor.Begin` with the new `BranchPoint`. `Begin` refuses an EVA BranchPoint first (`RewindPointAuthor.IsReFlySplitType`, one gate for every authoring path) and logs `Split type=EVA: no Rewind Point (Re-Fly is for vessel separations only) bp=<id>`; otherwise it takes the list of `ChildSlot`s (one per controllable child), and the list of controllable child PIDs. For BG-recorded splits that originate outside the focus vessel, `BackgroundRecorder` takes the same path with a cached PID payload.

6.6 Rewind Point capture

File: `Source/Parsek/RewindPointAuthor.cs`.

`Begin` runs synchronously (same frame as the split):

1. **Scene guard**: if `HighLogic.LoadedScene != GameScenes.FLIGHT`, abort with `[RewindSave] Warn: Aborted: scene=<loaded>`. Return null.
2. **Scenario guard**: verify `ParsekScenario.Instance` is live (using `ReferenceEquals` to bypass Unity's overloaded `== null` for test fixtures).
3. Generate `rpId = "rp_" + Guid.NewGuid().ToString("N")` and validate via `RecordingPaths.ValidateRecordingId`.
4. Build the RP stub with empty `PidSlotMap` / `RootPartPidMap`, `SessionProvisional = true`, `CreatingSessionId` set from the active marker if one is live.
5. Append the RP to `ParsekScenario.RewindPoints` and stamp `BranchPoint.RewindPointId = rpId`. This is the synchronous wiring — an `OnSave` triggered between `Begin` and the deferred coroutine will already see the in-progress RP.
6. Emit `[Rewind] Info: RewindPoint begin: rp=<id> bp=<bpId> slots=<N> controllablePids=<M> ut=<x>`.
7. Start the deferred coroutine (or, in tests, the `SyncRunForTesting` hook).

`RunDeferred` / `ExecuteDeferredBody` (one-frame-deferred):

1. Re-check scene guard; if we left FLIGHT between `Begin` and the deferred frame, `RollbackBegin` removes the RP from the scenario and clears the BranchPoint back-reference.
2. Drop high warp if `CurrentRateIndex > 1`. Wrapped in a `finally` so a mid-save throw still restores the player's rate.
3. Populate `PidSlotMap` and `RootPartPidMap` by iterating ChildSlots. For each slot, resolve the expected `Vessel.persistentId` via the injected `RecordingResolver` (or `CommittedRecordings` fallback), look up the live vessel via `FlightGlobalsProvider.TryGetVesselSnapshot`, and record `{pid → slotIdx}` and `{rootPid → slotIdx}`. A slot with no live vessel is marked `Disabled = true` with `DisabledReason = "no-live-vessel"`.
4. Log `[Rewind] Info: RewindPoint slot capture: rp=<id> populated=<p>/<total> disabled=<d>`.
5. If every slot failed, mark the RP `Corrupted = true` but continue with the save so the on-disk quicksave is still available for diagnostics.
6. Call `GamePersistence.SaveGame(tempName, saveFolder, SaveMode.OVERWRITE)` to `Parsek_TempRP_<rpId>.sfs` in the save root. KSP always writes to the save root per its own convention.
7. `FileIOUtils.SafeMove` the temp file to `<saveDir>/Parsek/RewindPoints/<rpId>.sfs` (atomic tmp + rename). `RecordingPaths.EnsureRewindPointsDirectory` creates the directory on demand.
8. Log `[RewindSave] Info: Wrote rp=<id> path=<relPath> bytes=<N> ms=<t>`. On any failure in steps 6-7, call `RollbackBegin` so a partial-write never leaves a half-registered RP.
9. Restore warp rate in the `finally`; log `[RewindSave] Info: Warp restored to <rate> for rp=<id>` or `[RewindSave] Warn: Warp restore to <rate> failed for rp=<id>: <reason>`.

Failure modes: disk full, permissions, save exception, scene transition between `Begin` and deferred frame. All paths either roll back cleanly or keep a Corrupted RP for diagnostics; the split itself proceeds normally either way.

6.7 Between split and merge

The recorder keeps recording normally: the focused vessel is actively sampled, unfocused controllable siblings are BG-recorded. All have `MergeState = NotCommitted`.

On session merge (the NON-re-fly merge that commits the tree that just ran):

- Focused vessel's recording → `Immutable`.
- BG-recorded controllable children whose parent BranchPoint has `RewindPointId != null` → `CommittedProvisional`. (They might become Unfinished Flights.)
- Other BG children under a BranchPoint without an RP → `Immutable`.
- Session-provisional RPs promote to persistent during `TagRpsForReap` when their `CreatingSessionId` matches the merging session — but for tree merges outside an active re-fly session, `CreatingSessionId` is null on the RPs written during flight, so the session promote path doesn't apply. `LoadTimeSweep` retains those normal staging RPs during the scene load that presents the merge dialog; `RecordingStore.CommitTree` promotes the accepted tree's normal staging RPs to persistent. Nested/session-created RPs with a dead `CreatingSessionId` are the ones purged. Unfinished Flights membership recomputes.

Note: the distinction between "session merge of a normal flight" and "session merge of a re-fly session" is driven by whether `ActiveReFlySessionMarker` is non-null. Only re-fly merges go through `MergeJournalOrchestrator.RunMerge`.

6.8 Invoking a Rewind Point

File: `Source/Parsek/RewindInvoker.cs`.

Precondition gate (`CanInvoke` , `RewindInvoker.cs:63`). Each check emits a reason string on failure (the list grew past the original five):

1. Scene is `FLIGHT` / `SPACECENTER` / `TRACKSTATION` (not a transition).
2. `RewindInvokeContext.Pending` is false (no in-flight invocation).
3. RP is not `Corrupted` .
4. Quicksave file exists on disk at the expected path (`ResolveAbsoluteQuicksavePath`).
5. No active re-fly session (`ParsekScenario.Instance.ActiveReFlySessionMarker == null`).
6. Deep-parse precondition passes: the quicksave is loaded as a ConfigNode and every `PART` node's `name` must resolve via `PartLoader.getPartInfoByName` . Cached per RP for 60 seconds via `PreconditionCache` . Missing parts mark the RP `Corrupted` .
7. (Section 2.12 Rule 1, designed 2026-10-19.) The selected slot's retire set holds no sealing row (`SupersedeCommit.RetireSetHoldsSealingRow`). Unfinished Flights already hides such a slot; the re-check catches a ledger that changed after the row was drawn. Reason `recordingAction:<Type>:<ActionId>` .

Confirmation dialog (`ShowDialog` , `RewindInvoker.cs:138`). A `PopupDialog.SpawnPopupDialog` with a MultiOptionDialog. The message names the UT, the selected slot / origin recording, and the supersede semantics:

```
Rewind to rewind point <rpId> at UT <utText>?
Spawning the selected child (slot <N>, origin=<rec>) live; merged siblings will play as ghosts.

Career state during this attempt stays as it is now. Supersede on merge
retires reviewed recording-scoped career actions from the superseded subtree
and recalculates KSP state from the surviving ledger.
```

Section 2.12 rewrites the career paragraph (wording settled with the implementation; it must say that the replaced flight earned nothing, that its crew losses are undone if the re-fly avoids them, and that everything the re-fly earns is added).

Buttons: `Rewind` (fires `StartInvoke`), `Cancel` (logs and dismisses).

Pre-load phase (`StartInvoke` , `RewindInvoker.cs:206`). All synchronous, no yield:

1. Generate `sessionId = "sess_" + Guid.NewGuid().ToString("N")` .
2. `ReconciliationBundle.Capture()` snapshots every in-memory state that must survive the KSP scene reload: committed recordings, committed trees, ledger actions, ledger tombstones, scenario lists (RPs, supersedes, marker, journal), crew reservations, group hierarchy, milestone store, ResourcesApplied flags. Any exception aborts with `[Rewind] Error` and a user toast.
3. `CopyQuicksaveToSaveRoot` copies `saves/<save>/Parsek/RewindPoints/<rpId>.sfs` to `saves/<save>/Parsek_Rewind_<sessionId>.sfs` (root). Required because `GamePersistence.LoadGame` does not support subdirectory paths.
4. Store the session id, bundle, RP, selected slot, temp path, and slot metadata in the static `RewindInvokeContext` . This is the only state that survives the scene reload.
5. `HighLogic.CurrentGame = GamePersistence.LoadGame(tempLoadName, HighLogic.SaveFolder, true, false)` + `HighLogic.LoadScene(GameScenes.FLIGHT)` . From here, Unity tears down the current scenario; only the static context survives.

Any failure between steps 2 and 5 triggers `TryRestoreBundle` + `HandleQuicksaveMissing` (clears filename + marks Corrupted if the file really is missing) + toast + context clear.

Post-load phase (`ConsumePostLoad` , `RewindInvoker.cs:335`). Called by `ParsekScenario.OnLoad` exactly once per invocation. All synchronous:

1. `ReconciliationBundle.Restore(bundle)` re-applies the pre-load in-memory state over the quicksave-loaded state. The quicksave's `RecordingStore` , `Ledger.Actions` , scenario lists, crew reservations, etc. are discarded; the bundle's are restored. `Planetarium.UT` remains at the quicksave's UT (that's the whole point).
2. `PostLoadStripper.Strip(rp, selectedSlotIndex)` strips the non-selected siblings (§6.9).
3. `FlightGlobals.SetActiveVessel(selectedVessel)` .
4. `AtomicMarkerWrite(rp, selected, stripResult, sessionId)` — the critical section (§6.10).
5. `LedgerOrchestrator.RecalculateAndPatch()` - non-fatal; a throw logs Warn but does not fail the invocation. The walk horizon is `double.MaxValue` , so the session shows the committed career; under section 2.12 Rule 3 the contract deadline clock passed to `ContractsModule.PrePass` is the current UT, not the horizon (`RecalculateAndPatch(double.MaxValue, deadlineClockUT: <live UT>)`), so a deadline still ahead of the session does not read as expired. A contract the walk ends with active but accepts only after the session's clock is kept out of stock's active list until the clock reaches its accept (Rule 3).

6. `finally`: delete the root-level temp quicksave and clear `RewindInvokeContext`.

The code also runs a step between the strip and the marker write that this list predates ("Step 3b"). It used to be `RetireResurrectedVesselRecoveryRows`, which tombstoned the recovery rows of any vessel the rewind point brings back that the committed timeline recovered later, outside any merge, with the drawdown guard bypassed, and the tombstones outlived a Discard. Section 2.12 Rule 3 replaced it with `HoldResurrectedVesselsWithCommittedFuture`: such a vessel goes to the claimed-vessel hold (section 6.9 step 5), its recovery rows stay, and the invoke writes no tombstone.

6.9 Post-load strip (authoritative matching)

File: `Source/Parsek/PostLoadStripper.cs`.

For each live `Vessel` in the enumeration:

1. **Ghost ProtoVessel guard**: if `GhostMapPresence.IsGhostMapVessel(pid)`, skip. Do not strip — it is a Parsek-spawned map presence. Increment `GhostsGuarded`.
2. **Primary match** via `RewindPoint.PidSlotMap`: if `pid` is in the map, record the match with the slot index.
3. **Fallback match** via `RewindPoint.RootPartPidMap` using the vessel's root-part `Part.persistentId`: if primary miss but rootPart is present and the map has a matching entry, record the match and increment `FallbackMatches`; log `[Rewind] Warn: Fallback match via root-part v=<pid> rootPart=<rootPid> slotIdx=<idx>`.
4. **No match**: `LeftAlone` ++. Log `[Rewind] Verbose: Strip leaveAlone: unrelated v=<pid> name=<name>`.
5. **Used by the committed future** (section 2.12 Rule 3, designed 2026-10-19): an unmatched vessel that a committed recording of the same launch (`VesselLaunchIdentity`) flies, changes or recovers after the rewind UT is not left alone: it goes to the claimed-vessel hold (`VesselGhoster`, drawn by `ParsekFlight.UpdateClaimedVesselHoldGhosts` in its rewind-save shape until its first committed recording starts, which then plays it). Its committed rows stay, a later recovery's included. Log `[Rewind] Info: Strip heldForCommittedFuture v=<pid> rec=<id> firstUT=<ut>`. This replaces Step 3b (section 6.8). A vessel with no committed recording after the rewind UT stays live. Implemented in `RewindInvoker.HoldResurrectedVesselsWithCommittedFuture`, which runs after the post-strip spawn reconcile (pure decision `ReFlyInvokeHold.Classify`, POSITIVE launch identity: pid and both launch guids known and equal, the re-flown and stripped vessels excluded) and feeds `ParsekFlight.HoldReFlyInvokeVessels`. A recording that flies the vessel after the rewind UT claims it at its start (or at the rewind UT when it is already playing), and its end-spawn stamp on the despawned pid is released so it brings the vessel back at its end; a recovery claims it at the earliest post-rewind recovery row. A vessel a committed recording changes (a dock, a board, a part transfer) is a ghost-chain claim and stays with the chain feed. The per-vessel line also carries `evidence=<recovery|flies> sess=<s>`, followed by one summary `Resurrected vessel hold: survivors=<n> held=<m> notLive=<k> alreadyHeld=<a> matchedWithoutFuture=<j> cutoffUT=<ut>`.

After enumeration: walk the matches. The vessel matching the selected slot is kept as `SelectedVessel`; all other matches are stripped via `Vessel.Die()`. If multiple vessels match the selected slot (shouldn't happen in practice), the first is kept and the rest are stripped with a `[Rewind] Warn: Multiple vessels match selectedSlot=<n>`.

Final log: `[Rewind] Info: Strip stripped=[<indices>] selected=<idx> ghostsGuarded=<N> leftAlone=<M> fallbackMatches=<f>`.

If the stripper returns `SelectedPid == 0u` (selected vessel not present on reload), `ConsumePostLoad` toasts an error and aborts — the quicksave is stale relative to the RP's expectations.

6.10 Atomic provisional + marker write

File: `RewindInvoker.cs:463` (`AtomicMarkerWrite`).

The critical section. Runs synchronously; throws MUST leave global state untouched. NO yield, NO await, NO deferred save.

1. `CheckpointHookForTesting("CheckpointA:BeforeProvisional")`.
2. Build the provisional Recording (`BuildProvisionalRecording`, `RewindInvoker.cs:527`): fresh `RecordingId = "rec_" + Guid.NewGuid().ToString("N")`, `MergeState = NotCommitted`, `CreatingSessionId = sessionId`, `SupersedeTargetId = selected.OriginChildRecordingId`, `ProvisionalForRpId = rp.RewindPointId`, `ParentBranchPointId` cloned from the origin child (or the RP's BranchPoint as fallback), `TreeId` cloned from the origin child, `VesselPersistentId = stripResult.SelectedPid`, `VesselName = stripResult.SelectedVessel.vesselName`.
3. `RecordingStore.AddProvisional(provisional)`.
4. `CheckpointHookForTesting("CheckpointA:AfterProvisional")`.
5. Build the `ReFlySessionMarker` from the provisional, selected slot, and RP.
6. `CheckpointHookForTesting("CheckpointB:BeforeMarker")`.
7. `scenario.ActiveReFlySessionMarker = marker; scenario.BumpSupersedeStateVersion();`

8. `CheckpointHookForTesting("CheckpointB:AfterMarker")`.
9. Log `[ReFlySession] Info: Started sess=<sid> rp=<id> slot=<idx> provisional=<rid> origin=<oid> tree=<tid>`.

A `catch` around steps 5–8 rolls back both: removes the provisional via `RecordingStore.RemoveCommittedInternal(provisional)` and sets `ActiveReFlySessionMarker = null`. Both are idempotent. Rollback is the reason `try/catch` wraps those steps rather than wrapping the whole method — a mid-provisional throw doesn't leave a half-marker behind either.

KSP's `ScenarioModule.OnSave` cannot preempt this synchronous block. The `CheckpointHookForTesting` is the only observer that can see between checkpoints — tests use it to verify no save boundary is crossed.

6.11 Within a re-fly session

Ghost playback engine, chain walker, map presence, watch mode enumerate `ERS \ SessionSuppressedSubtree(marker)`. Career subsystems read ERS / ELS directly.

Nested multi-controllable splits during the session create additional session-provisional RPs with `CreatingSessionId = activeSession.SessionId`. On session merge, all promote; on discard, all purge (load-time sweep spare-set logic, §6.16). Only the provisional itself carries the session's `CreatingSessionId` (`RewindInvoker.BuildProvisionalRecording`); child recordings born in the session's splits do NOT (an earlier version of this paragraph said they did). They are the attempt's through topology instead: `MergeDialog.CollectReFlyAttemptOwnedRecordingIds` and `EffectiveState.CollectActiveSessionOwnedRecordingIds` walk forward from the provisional.

Switching during a session (section 2.12 Rule 3, owner ruling 2026-10-19). While a marker is live the player can fly only the re-flown vessel and its own crew on EVA. Before the block, the two stock paths gave different results: `[` / `]` attached the vessel switched to under a `Launch` branch point of the session's tree (attempt-owned, dropped on Discard), while a map Switch-To created a parentless segment (not attempt-owned, so its rows survived a Discard). A map Switch-To to an unloaded vessel opened the pre-switch Merge / Discard dialog, which takes no Re-Fly input (`DecidePreSwitchDialogAction`) and whose Merge runs an ordinary `CommitTreeFlight` on the live Re-Fly tree: no supersede rows, no tombstones, the marker left set. Gains earned on a vessel switched to also survived the merge without sealing the slot, so a second Re-Fly could switch over and earn them again. The rule, as built in `ReFlySwitchGate` (one pure `Decide(markerLive, targetPid, allowedPids, path)`):

- **Allowed targets:** the vessel whose live recording continues the re-flown vessel's own walked stretch (section 1.6, `EffectiveState.WalkSlotVessel` started at the provisional), and kerbals on EVA from it. A vessel the session split off by a separation is NOT an allowed target; it is background-recorded, its split keeps a session rewind point, and it can be re-flown after the merge. As built, `ReFlySwitchGate.ComputeAllowedPids` walks the session tree forward from the provisional on every switch attempt, so the set grows as the session goes EVA: chain segments, switch-continuation and dock children, the same-pid continuation across an EVA / undock / joint break / breakup, the EVA kerbal stepping out of the vessel, and a board back into it. An unresolvable session tree refuses fail-closed.
- **Refused paths:** a switch to a target outside the allowed set from any player-input entry point: the `[` / `]` switch keys, the map context menu Switch-To and focus Fly (`MapContextMenuOptions.FocusObject.OnSelect`, where the Re-Fly refusal comes before `DecidePreSwitchDialogAction`), Tracking Station Fly, the KSC marker Fly, and the Jump button of the stock alarm dialog (`AlarmClockMessageDialog.OnJump`, which calls `SetActiveVessel` in FLIGHT and saves + `StartAndFocusVessel` elsewhere; GC-13). The block sits at those entry points, not inside `FlightGlobals.SetActiveVessel`, so the active-vessel changes stock makes inside an EVA, a board, a dock or an undock are not switches and stay allowed; what such an event does to the slot is the existing contract (section 4.9: an undock or a foreign board seals). As built: `MapFocusObjectOnSelectPatch.Prefix`, `GhostTrackingFlyPatch.Prefix` (mirrored silently by `SwitchIntentTrackingStationFlyPatch` so no intent is armed), `KscVesselMarkerFlyPatch.Prefix`, `AlarmJumpSwitchPatch.Prefix` (path `alarmJump`; in FLIGHT it runs the same gates as the map Switch-To through `MapFocusObjectOnSelectPatch.PrepareOwnedVesselSwitch`, and `AlarmJumpButtonMarkPatch` hides the Jump button for a refused alarm), and `VesselCycleReFlySwitchPatch` on stock `VesselSwitching.Update` (decompiled: the only reader of `FOCUS_PREV_VESSEL` / `FOCUS_NEXT_VESSEL`), which drops refused candidates from stock's own candidate list and still switches to an allowed one (it also drops ghost map markers on every press, in or out of a Re-Fly, GC-4). The automation verb `SimulateStockSwitchClick` answers `REJECTED refly-switch-refused:<reason>`.
- **Feedback:** none beyond the refusal and its log line (`[ReFlySession] Info: Switch refused sess=<s> path=<p> target=<pid> reason=<r>`). Where the refused control is a stock control with its own disabled state (the map context menu entry, the Tracking Station Fly button), the stock-control annotation rule applies: the control is greyed with the reason appended to its stock tooltip, from the same predicate the refusal reads. Not built yet: the greying and tooltip annotation of the map context menu entry and the Tracking Station Fly button; today a refused control stays enabled and does nothing.

Award tagging during a session (section 2.12 Rule 3). A row is tagged to the recording of the vessel whose event caused it: the vessel a contract parameter, a science subject, a recovery or a destruction names. Only an event that names no vessel goes to the live recording. As built (`ReFlyAwardTagging.ResolveByCause` / `TryResolveCauseTag`, reason `cause-vessel`, an `InfoRateLimited` `outcome=tagged-by-cause` line when the tag differs from the live one): a science subject is filed under the vessel that transmitted or

recovered it (`GameStateRecorder.CaptureScienceSubject`), a lab payout under the lab's vessel (`OnScienceLabCredited`), and any event raised inside a vessel recovery, plus the strategy conversion that follows it, under the recovered vessel (`RecoveryDispatchScope.CurrentPersistentId` in `Emit`). The vessel resolves to its session recording through the active recording, then the tree's background map, then an open line end of that pid; the origin still substitutes to the provisional. Stock contracts and contract parameters carry no vessel (parameters read the active vessel), and stock destruction and crew-death events carry a crew member, not a vessel, so those rows keep the live tag (the re-flown vessel, since switching is blocked); kerbal death rows are built per recording from crew end states at commit and were already filed by cause. An empty live tag still falls back to the provisional (`unbound-empty-tag`). With switching blocked, the remaining background vessels in a session are the ones the session split off and the unrelated live vessels; filing by cause keeps each one's rows on its own recording.

6.12 Merging a re-fly session (staged commit, journaled)

File: `Source/Parsek/MergeJournalOrchestrator.cs` .

Triggered by `MergeDialog.MergeCommit` when `ActiveReFlySessionMarker` is non-null. The 14-step design-spec merge is consolidated into eleven phase transitions:

0. **Dominance check, before the tree commits** (section 2.12 Rules 2 and 5, designed 2026-10-19; `ReFlyMergeDominance.EvaluateForMerge` , the row build `LedgerOrchestrator.BuildTreeCommitRowsPreview` , the cloned walk `RecalculationEngine.WalkBalancesOnClones`). It runs in `MergeDialog` when the merge dialog opens and again on the Merge click, BEFORE `MergeDialog.MergeCommit` calls `CommitPendingTree` , because the attempt's rows are filed at tree commit (`NotifyLedgerTreeCommitted`), before any journal step; it reads them from the pure row-build step section 2.12 Rule 5 splits out of `FileRecordingCommitRows` . It computes the merge closure and the retire set the Supersede and Tombstone steps will write, picks the loss-swap variant (A, or B when A would lower reputation at some UT), and evaluates Invariant LS on cloned resource modules over `L` (ELS without the attempt's rows) and the chosen `L'` . It also refuses when the retire set holds a sealing row (Rule 1 re-checked at the last moment). Nothing live is touched and nothing is written, so a refusal needs no rollback. On refuse the merge is not offered: the existing merge dialog says why in its body and keeps Discard (and Retry where stock offers it); a refusal at the click (the state changed after the dialog opened) commits nothing and shows the dialog again with Discard only. The check is a no-op outside a session and while a merge journal is already running (the journal carries its variant). `MergeCommit` hands the chosen variant to `RunMerge` , which stores it on the journal at `Begin` so a crash-recovered Tombstone step writes the same set (`MergeJournal.LossSwapVariant` , a new optional value, absent = A). Log `[LedgerSwap] Info: Merge dominance check ...` and `[LedgerSwap] Info: Loss swap ...` (section 12.6).
1. **Journal write, phase = `Begin`** . `scenario.ActiveMergeJournal` assigned. Log `[MergeJournal] Info: sess=<sid> phase=Begin lossSwapVariant=<A|B>` .
2. **Tree merge, then the Begin save**. `MigrateActiveReFlyForkIntoCommittedTree` folds an in-place-continuation re-fly fork into the committed tree (a no-op when there is no such fork), then `DurableSave("begin", persistSynchronously: true)` writes the journal with the committed attempt and the migrated fork, and the phase advances to `TreeMerge` . The forward-completion path (`CompleteFromPostDurable`) begins at `Begin` , resumed as `TreeMerge` . Idempotent on re-run.
3. **Origin split at rewind UT** (`AdvancePhase → Split`). When the closure-root recording's lifetime strictly spans `marker.RewindPointUT` , `RecordingTreeSplitter.SplitOriginAtRewindUT(marker, scenario)` cuts it into HEAD (kept visible, pre-rewind half) + TIP (the tail from the cut onward that the fork will supersede; its `StartUT` can sit a fraction of a second BEFORE the rewind UT - RW-7 measured a TIP from 185786.41 with the rewind at 185786.49 - so nothing may assume a TIP row or sample is dated at or after the rewind point), and rewrites `marker.SupersedeTargetId = TIP.RecordingId` so the supersede write-set in step 4 retires TIP instead of the spanning origin. The splitter owns its own deep-clone snapshot + ChainIndex map + incremental retag ledger and rolls back transactionally on exception. A dedicated `DurableSave("split", persistSynchronously: true)` barrier sits between this step and step 4, isolating the in-memory split from the post-Begin forward-completion path. Idempotent on re-run: if TIP already exists (or the origin's lifetime does not span the rewind UT), the splitter exits via the existing-tip / not-yet-mutated arms without re-cutting. Log `[Supersede] Info: SplitOriginAtRewindUT: origin=<originId> rewindUT=<ut> head=<headId> tip=<tipId>` (or the existing-tip / not-spanning short-circuit log).
4. **Supersede relations** (`AdvancePhase → Supersede`). `SupersedeCommit.AppendRelations` computes the forward-only merge-guarded subtree closure via `EffectiveState.ComputeSessionSuppressedSubtree(marker)` , appends one `RecordingSupersedeRelation { old=subtreeId, new=provisional.Id, UT, CreatedRealTime }` per subtree id. Idempotent — a pre-existing relation with the same old/new pair is skipped with a Verbose log. Log `[Supersede] Info: Added <N> supersede relations for subtree rooted at <originId>` . Before any row, the provisional is validated by `SupersedeCommit.ValidateReFlySessionSupersedeSource` (see "A session that went EVA" below); a refusal writes 0 rows and logs `outcome=refused-unflown-provisional` with the payload and walk evidence. The closure never admits the live session's own recordings: the fork is fenced by the NotCommitted guards, and everything the recorder created from it (`EffectiveState.CollectActiveSessionOwnedRecordingIds` : forward along every branch point naming a member as a parent, chain segments of a descendant, anchored debris) is removed from the walk's candidate index, because a continuation carries the re-flown vessel's pid and starts after the rewind point, which is exactly the pid-peer gate. Verbose `fenced live session's own recordings ... count=N` .

5. **Tombstones** (`AdvancePhase → Tombstone`). `SupersedeCommit.CommitTombstones` walks `Ledger.Actions`, groups in-scope actions by RecordingId for bounded rep-pairing, emits `LedgerTombstone`s for every action passing `TombstoneEligibility.IsSupersedeTombstoneEligible`. See §6.13. Under section 2.12 the retire set it walks holds only loss rows (the value gate kept every other slot from being re-flown); with loss-swap variant B it skips the replaced flight's `ReputationPenalty` loss rows and tombstones the re-fly's own instead (`RetiringRecordingId` = the re-fly's recording). The re-run on crash recovery reads the variant from the journal. `TombstoneStateVersion` bumped. `CrewReservationManager.RecomputeAfterTombstones` re-derives reservations; tombstoned crew reservations disappear and roster-created kerbals queue guarded cleanup. Log `[LedgerSwap] Info: Tombstoned <N> career actions (Contract=<c>, Milestone=<m>, Facility=<f>, Strategy=<s>, ScienceSpending=<t>, Science=<sc>, Funds=<fn>, Reputation=<r>, Kerbal=<k>, Other=<o>); <M> excluded (Seed=<s>, Rollout=<r>, Other=<o>)` plus `[Supersede] Info: Supersede tombstone effects: tombstoned <N> recording-scoped career actions; ... from the old subtree are removed from ELS.`
6. **Finalize** (`AdvancePhase → Finalize`). `SupersedeCommit.FlipMergeStateAndClearTransient` flips `provisional.MergeState` to `Immutable` (Landed / InFlight) or `CommittedProvisional` (Crashed), clears `SupersedeTargetId`, bumps `SupersedeStateVersion`. When the slot walk from the provisional hops (own EVA, same-vessel Board, switch continuation), the same verdict is written on the walked tip (`SupersedeCommit.ApplyMergeStateToSessionVesselTip`, log `session vessel tip rec=... mergeState a->b`): open/closed is read there (section 4.7), and a continuation is born `Immutable`. Marker preserved (`preserveMarker: true`) so it stays live across Durable Save #1.
7. **Durable Save #1** (`AdvancePhase → Durable1Done`). Deferred to the next natural `ScenarioModule.OnSave` in production; tests inject a synchronous hook. This is the durability barrier — on-disk state is now supersedes + tombstones + MergeState all committed; marker + RPs still live.
8. **RP reap** (`AdvancePhase → RpReap`). `TagRpsForReap(marker, scenario)` flips `SessionProvisional = false` on every RP with matching `CreatingSessionId`, and defensively promotes the marker's origin RP when it is a normal staging RP with `CreatingSessionId == null`. Then `RewindPointReaper.ReapOrphanedRPs` walks all RPs; for each non-session-provisional RP, it walks every ChildSlot's `EffectiveRecordingId` and marks the RP reap-eligible if every effective recording has `MergeState == Immutable`. Eligible RPs get their quicksave file deleted (`File.Delete` is best-effort), scenario entry removed, BranchPoint back-reference cleared. Log `[Rewind] Info: ReapOrphanedRPs: reaped=<R> remaining=<rem>`.
9. **Marker clear** (`AdvancePhase → MarkerCleared`). `scenario.ActiveReFlySessionMarker = null` ; `BumpSupersedeStateVersion`. Log `[ReFlySession] Info: End reason=merged sess=<sid> provisional=<rid>`.
10. **Durable Save #2** (`AdvancePhase → Durable2Done`).
11. **Clear journal** (`ClearJournalAndFinalSave`). `journal.Phase = Complete` ; Durable Save #3; `ActiveMergeJournal = null`. Log `[MergeJournal] Info: Completed sess=<sid>`.

A session that went EVA (REFLY-SESSION-EVA-CANNOT-SUPERSEDE, fixed 2026-09-27). When the player steps a kerbal out of the re-flown vessel, the recorder ends the provisional fork at the `EVA` branch point with no terminal; the vessel flies on as a same-pid continuation, and a re-board merges it again at a `Board` point. A stock Switch-To away and back splits the fork the same way at a `VesselSwitchContinuation` point. The merge contract:

- **Validation reads the vessel, not the fork.** `ValidateReFlySessionSupersedeSource` walks the fork with `EffectiveState.WalkSlotVessel` (the one slot walk, section 1.6): the terminal is read on the walked tip, the playable payload on the fork or any vessel segment the walk stood on. With no hop it is exactly `ValidateSupersedeTarget` on the fork, so a placeholder or an unflown provisional is still refused. Info `AppendRelations outcome=validated-through-vessel-walk provisional=... terminalRec=... terminal=... evaHops= boardHops= switchHops=`.
- **Rows name the fork.** `NewRecordingId` stays the fork, the head of the new flight; the slot tip is reached from it through the same walk (`EffectiveTipRecordingId`). The marker is not re-pointed (section 5.9).
- **The session's EVA kerbal, the continuation and the re-boarded vessel are never superseded** (the closure fence in step 4).
- **Sealing** is Site B-1's usual verdict read on the walked tip (section 4.9): a stable ending seals (`stableTerminalFocusSlot`), the session's own `EVA` branch point seals a non-stable ending (`structuralMutation`), a `Destroyed` ending keeps the slot open for another try, a sealing row filed on the EVA (science, a milestone, section 2.12) seals (`recordingAction`), and step 6 writes the verdict on the walked tip.
- **A kerbal who boards a FOREIGN vessel during the session** (section 1.6 rule 2): the merge still supersedes the old stretch and the slot seals on `evaCrewJoinedForeignVessel`, even when the vessel crashed.
- **Crash recovery** needs nothing new: every journal step re-derives the same walk from the same fork (`ResolveProvisional` reads the unchanged marker).
- **Mirror shapes.** A Dock closes the fork `Docked` and staging keeps the fork's id (tree-branching split), so neither changes.
- **A session that undocks** (REFLY-SESSION-UNDOCK-CANNOT-SUPERSEDE, owner-approved and fixed 2026-09-27). An undock ends the fork at the `Undock` split with its terminal untouched (`docs/dev/dock-undock-recording-structure.md` section 4), and the slot walk stops at a separation by design (section 1.6), so the walked tip is the fork with no terminal. The marker overload of `ValidateReFlySessionSupersedeSource` accepts that tip as concluded when the walk stopped at a separation THIS session authored (`SupersedeCommit.IsSessionSeparationStop` : an `Undock` / `JointBreak` / `Breakup` branch point naming the tip as a parent and absent from `marker.PreSessionBranchPointIds`). Info `AppendRelations outcome=validated-at-session-`

`separation provisional=... bp=... type=Undock bpUT=...`. Only the missing-terminal refusal can be answered this way; a placeholder or unflown fork still refuses on payload, a separation that predates the session does not count, and a legacy marker with no pre-session baseline keeps the refusal. Rows name the fork; both halves are the fork's descendants and so are fenced out of the closure (step 4). Site B-1 then reads the fork ending at a downstream branch point and seals the slot (`classifierClosed:downstreamBp`, section 4.9 rule 4; it closes before the rule 2 structural gate is reached), whatever either half did afterwards. Crash recovery re-derives the same decision from the unchanged marker.

Failure recovery matrix (`RunFinisher`, `MergeJournalOrchestrator.cs:263`). On load, if `ActiveMergeJournal != null`:

Phase on disk	Recovery
`Begin`, `TreeMerge`	**Forward-completion.** `RunFinisher` dispatches to `CompleteFromPostDurable`, which resumes `Begin` as `TreeMerge` and re-runs `MigrateActiveReFlyForkIntoCommittedTree` (idempotent: an already-tree-resident fork is a no-op) before Split. `Begin` used to roll back (remove the session-stamped recordings, clear the marker) on the belief that the Begin save held only the journal; it holds the committed attempt, whose EVA kerbals, payloads and ledger rows carry no session stamp, so the rollback left them committed beside the restored origin (todo MERGE-BEGIN-ROLLBACK-LEAKS-ATTEMPT). The player clicked Merge and the click-time dominance check passed, so completing the merge is the outcome the player chose. `[MergeJournal] Info: Completed from phase=Begin sess=<sid> stepsDriven=<n>`.
`Split`	**Forward-completion.** `RunFinisher` dispatches to `CompleteFromPostDurable`; `SplitOriginAtRewindUT` re-runs idempotently via the existing-tip / not-yet-mutated arms (an already-cut TIP is detected and skipped, or the spanning predicate fails because `marker.SupersedeTargetId` already points at TIP). Tail then drives through Supersede → Tombstone → Finalize → Durable1Done → RpReap → MarkerCleared → Durable2Done → Complete.
`Supersede`	**Forward-completion.** `CompleteFromPostDurable.Supersede` block re-runs `AppendRelations`; existing rows are skipped via `RelationExists`, so a partial in-memory write becomes a no-op on the second pass. The subtree id list is stashed on `journal.RecoveredSubtreeIds` so the Tombstone block reuses it instead of recomputing the closure.
`Tombstone`	**Forward-completion.** `CompleteFromPostDurable.Tombstone` block re-runs `CommitTombstones`; dedup via the `alreadyTombstoned` HashSet means a pre-existing `LedgerTombstone` with the same `ActionId` is skipped. Subtree is reused from `journal.RecoveredSubtreeIds` when present, or recomputed via `RebuildSubtree` fallback.
`Finalize`	**Forward-completion.** `CompleteFromPostDurable.Finalize` block re-runs `FlipMergeStateAndClearTransient(preserveMarker: true)`; value-write idempotent because the inputs are the same `TerminalKindClassifier.Classify` verdict and the same provisional, so the same `MergeState` and the same cleared `SupersedeTargetId` get rewritten over identical values.
`Durable1Done`, `RpReap`, `MarkerCleared`, `Durable2Done`, `Complete`	**Forward-completion.** `CompleteFromPostDurable` drives the remaining steps. Idempotent: missing file deletions are no-ops, an already-cleared marker is a no-op, `journal.Phase = Complete` overwrites itself harmlessly. `[MergeJournal] Info: Completed from phase=<p> sess=<sid> stepsDriven=<n>`.

The finisher is triggered by journal presence, **not** by marker state. This is the v0.5 pre-impl correction that survived into v1. Recovery is driven by **idempotency, not by the durable barrier**: every phase drives forward because every step has its own idempotency guard (`SplitOriginAtRewindUT`'s existing-tip check, `AppendRelations`'s `RelationExists` skip, `CommitTombstones`'s `alreadyTombstoned` HashSet, `FlipMergeStateAndClearTransient`'s value-write semantics). Forward-completion via re-run is safer than rollback (which would otherwise have to undo serialized state mutations after the Split durable barrier); the Split phase has its own DurableSave between Begin and Durable1Done so a crash from Split onward is recoverable via re-run rather than via reversing the split's BranchPoint / debris / ledger retags.

6.13 Supersede tombstone-eligible scope

File: `Source/Parsek/GameActions/TombstoneEligibility.cs` + `TombstoneAttributionHelper.cs`.

An action `a` is in supersede scope when:

- `a.RecordingId` is non-null AND `a.RecordingId` is in the forward-only merge-guarded subtree closure.

For actions in scope, broad reviewed-career eligibility retires recording-owned consequences from the superseded subtree:

`GameAction.Type`	Eligible?	Condition
Initial seed rows (`FundsInitial` , `ScienceInitial` , `ReputationInitial`)	**no**	Baseline save state, not a mission consequence.
`FundsSpending` with `SpendingKind == VesselBuild`	**no**	Rollout/build cost stays outside the re-fly consequence set.
Other recording-owned actions in the supersede subtree	yes	Retired so replay uses the new branch's career/science/facility/contract consequences.

Null-scoped actions (action's `RecordingId == null`) are not tombstoned because they are not owned by the supersede subtree (§7.41).

Loss and sealing classes (section 2.12, designed 2026-10-19). The eligible rows of a slot's closure after the rewind point are its retire set. Before a slot is offered, and again before a merge, the set is split in two:

Class	Rows	What happens
Loss	`KerbalAssignment` with end state `Dead` , `Aboard` or `Unknown` ; `ReputationPenalty` with source `KerbalDeath` or `Other` ; `FacilityDestruction`	Retired by the merge. The re-fly re-records each of them by itself if it repeats the event. Reputation goes through the loss swap (section 2.12 Rule 2).
Sealing	every other eligible row: all gains, all contract outcomes (accept, complete, fail, cancel, and the `ContractFail` / `ContractDecline` reputation penalties), flight-tagged spends and exchanges, facility and strategy rows, `KerbalAssignment` with end state `Recovered` , and any type added later	The slot is not re-flyable (Rule 1), so the merge never meets one. A merge whose retire set holds one anyway is refused at step 0 of the merge (section 6.12), whatever its effect on the balances.

The class list is a closed allowlist on the loss side: a new `GameActionType` (or a new `ReputationPenaltySource`) is a sealing row until it is reviewed into the loss list, the same default-safe shape as the eligibility list above.

A superseded TIP's loss rows, `KerbalAssignment` `Aboard` / `Unknown` included, are therefore retired whatever their own UT: a TIP ends after the rewind point, and its crew rows are screened by their end. A loss crew row is placed against the rewind point by its END, not its boarding (`TombstoneAttributionHelper.ComputeAttributionUT` , the one key the pre-rewind guard and both split retags share): a `KerbalAssignment` ending `Dead` , `Aboard` or `Unknown` records the crew's fate at the end of the flight, so a crew that boarded at launch on a recording spanning the rewind point has its row moved to the TIP with the terminal and retired by the merge. Keyed by the boarding UT, an `Aboard` row stayed on HEAD outside the closure, kept the kerbal held open-ended by a flight the merge replaced (a respawn-on death in the re-fly read as a permanent loss through that co-row), and the next load re-derived a fresh, untombstoned row on the TIP (REFLY-MERGED-DEATH-PERMANENT-VIA-SUPERSEDED-CO-ROW, 2026-10-28). A `Recovered` row keeps its boarding UT.

Because the closure admits the vessel's own EVA crew recordings (section 1.6, section 6.3), the scope reaches a kerbal who died on an EVA from the re-flown vessel: his `KerbalAssignment` +Dead row and the bundled death penalty are tagged to his EVA recording and are retired exactly like a crash death aboard.

Eligibility is checked with an idempotence guard: an action that already carries a tombstone (any existing `LedgerTombstone.ActionId == a.ActionId`) is skipped. This makes the whole merge re-runnable if the finisher drives through the Tombstone phase on load.

6.14 Revert-during-re-fly dialog

Files: `Source/Parsek/RevertInterceptor.cs` + `Source/Parsek/ReFlyRevertDialog.cs` .

`RevertInterceptor` is a pair of Harmony prefixes — one on `FlightDriver.RevertToLaunch` (Esc > Revert to Launch and the flight-results dialog's Launch revert button) and one on `FlightDriver.RevertToPrelaunch` (Esc > Revert to VAB / SPH and the flight-results dialog's VAB/SPH revert button). Both prefixes delegate to the shared dispatcher `RevertInterceptor.Prefix(RevertTarget target, EditorFacility facility)` where `RevertTarget ∈ {Launch, Prelaunch}` and `facility` is captured from the stock `RevertToPrelaunch(EditorFacility)` argument (ignored for the Launch path). When `ParsekScenario.Instance?.ActiveReFlySessionMarker == null` , the prefix returns `true` and the stock revert runs. When non-null, it returns `false` (blocking stock revert) and `ReFlyRevertDialog.Show(marker, target, onRetry, onFullRevert, onCancel)` spawns a `PopupDialog` with three buttons:

- **Retry from Rewind Point** — `RetryHandler`: clear marker, generate fresh `Guid`, re-invoke `RewindInvoker.StartInvoke(rp, slot)` with the same RP + slot captured from the marker (after `MergeDialog.RepairSessionBeforeRetry`, see the OQ-1 paragraph below). The old provisional becomes a zombie for the next load-time sweep to clean up; under section 2.12 Rule 3 Retry removes the ledger rows tagged to the old attempt's recordings before the new invocation, exactly as Discard does, so a second attempt never inherits the first one's rows. RP-anchored and scene-agnostic: Retry lands the player back in FLIGHT at the split moment regardless of whether they clicked Revert-to-Launch or Revert-to-VAB/SPH. Log `[ReFlySession] Info: End reason=retry sess=<old> rp=<rp> slot=<i> target=<Launch|Prelaunch>`.
- **Discard Re-fly** — `DiscardReFlyHandler`: session-scoped cleanup. Removes the provisional re-fly recording from `CommittedRecordings` (by-id lookup in the raw committed list, then `RecordingStore.RemoveCommittedInternal`; ERS-exempt because NotCommitted is invisible to ERS) together with every other attempt-owned recording (`MergeDialog.PruneActiveReFlyAttemptOwnedTopology`, which removes the ledger rows tagged to the attempt's recordings through `Ledger.RemoveActionsForRecordings`, section 2.12 Rule 3, so the discard leaves the ledger exactly as it was before the invocation, an attempt a quickload resumed after its merge included; log field `ledgerRowsRemoved=<n>`), promotes the origin RP (the RP named by `marker.RewindPointId`) by flipping `SessionProvisional=false` and clearing `CreatingSessionId` so the post-load `LoadTimeSweep` treats it as persistent (the sweep's RP-discard pass skips every `SessionProvisional=false` RP and every normal staging RP with `CreatingSessionId == null`), clears the pending science subjects, `ActiveReFlySessionMarker` and `ActiveMergeJournal` and bumps `SupersedeStateVersion` (not `TombstoneStateVersion` - no tombstone rows are touched) through `MergeDialog.EndDiscardedReFlySession`, the session-state half the merge-dialog Discard runs too, then defensively deletes the prior session temp quicksave `saves/<save>/Parsek_Rewind_<sessionId>.sfs` if still present, then stages the RP quicksave through the same save-root copy path `RewindInvoker` uses and calls `GamePersistence.LoadGame(tempLoadName, SaveFolder, true, false)` to restore the RP's UT / career / vessels. On success, for the Prelaunch context pre-sets `EditorDriver.StartupBehaviour = EditorDriver.StartupBehaviours.START_CLEAN` (NOT `LOAD_FROM_CACHE` — the cached `ShipConstruct` is stale after `LoadGame` swaps `HighLogic.CurrentGame`) and `EditorDriver.editorFacility = facility`; then `HighLogic.LoadScene` dispatches to `GameScenes.SPACECENTER` for Launch or `GameScenes.EDITOR` for Prelaunch. Stock `FlightDriver.RevertToLaunch` / `RevertToPrelaunch` are NOT used — decompilation (via `ilspycmd` on `Assembly-CSharp.dll`) shows they restore the current flight's cached `PostInitState` / `PreLaunchState`, which are tied to the current flight's launch moment, not the RP's UT. `TreeDiscardPurge.PurgeTree` is NOT called: the tree's other Rewind Points, supersede relations, and tombstones are preserved. The origin split's crash recording plus its promoted RP still resolve, so `EffectiveState.IsUnfinishedFlight` stays true for `Immutable` legacy crashes and `CommittedProvisional` current crashes, together with the RP and a crashed terminal, and the Unfinished Flights row stays visible. Log `[ReFlySession] Info: End reason=discardReFly sess=<sid> target=<Launch|Prelaunch> facility=<VAB|SPH|-->` (the `facility=--` sentinel is logged in the Launch variant for grep-friendliness; the handler prepends `dispatched=false` and suppresses the scene call when the RP quicksave cannot be staged so the player is left in flight with a clear toast).
- **Continue Flying** — `CancelHandler`: pure logging; no state changes. Log `[ReFlySession] Info: Revert dialog cancelled sess=<sid> target=<Launch|Prelaunch>`.

Dialog body copy branches on `target`: the Launch variant says "returns you to the Space Center" as the Discard Re-fly destination; the Prelaunch variant says "returns you to the VAB or SPH" and explicitly clarifies that Retry still returns the player to FLIGHT at the split moment regardless of which Revert button they clicked. Title ("Revert during re-fly") and button labels are shared. When `ActiveMergeJournal` is non-null the Discard Re-fly button is omitted entirely (primary UX signal to the player that discard mid-merge is not allowed); the handler also refuses defensively with a `"merge in progress — retry in a moment"` toast + Warn log if called anyway. A third body hook (`ReFlyRevertDialog.BodyHookForTesting`) plus a buttons hook (`ReFlyRevertDialog.ButtonsHookForTesting`) let unit tests assert the variant + the journal gate without spawning a live popup.

The dialog takes an input lock (`DialogLockId = "ParsekReFlyRevertDialog"`) while visible so the player cannot interact with the flight scene during the decision.

The Discard Re-fly load. The scene the discard opens (`SpaceCenterMain.Start` for Launch, `EditorDriver.Start` for Prelaunch; KSP 1.12.5, decompiled) reloads persistent.sfs and runs `Game.Load` on THAT game, so the one OnLoad that follows reads persistent.sfs as last written, not the origin RP's quicksave (todo DISCARD-REFLY-LANDS-ON-PERSISTENT-NOT-THE-REWIND-POINT). The handler arms `DiscardReFlyLoadIntent` after its load succeeded; that OnLoad classifies as `DiscardReFly` (scene-checked), and the in-session handoff (section 6.16) gives it the handler's own result: the cleared marker, the null journal, memory's supersede rows / retirements / tombstones, and memory's (promoted) copy of every rewind point of a tree memory holds committed. Whatever stale marker or session-stamped point persistent.sfs carries never reaches the sweep. The same OnLoad holds back every vessel of the rewind point's world the committed future uses after the split, as a plain rewind does (section 10.13, `ApplyDiscardReFlyVesselHolds`). The file the handler writes holds the rewind point's world around the session's PARSEK node (`RevertInterceptor.SpliceLiveParsekScenarioNode`), with one exception: `STOCK_POOL_REFERENCE` stays the rewind point's own (none when it had none), because a pool reference describes the stock pools of the file that holds it, and the session's reference describes the pools its Re-Fly start wrote after walking the whole ledger. The load's first ledger walk then runs as a Re-Fly discard (an authoritative reduction to the ledger at the loaded clock, `reFlyDiscard`). Game rule: Discard Re-fly returns the career to just before the Re-Fly, and what the committed future earns after the split arrives when the clock reaches it; measured against the

session's reference, the rewind point's pools read that future as a loss and kept it out of the career for good (todo REFLY-DISCARD-AFTER-REWIND-LOSES-COMMITTED-FUTURE, live lane RW-5).

Discarding a resumed merged session (owner ruling OQ-1). A load of a save file (F9, the Load menu) taken during a Re-Fly session that has since merged is a cold load (section 6.16): the save holds the session unmerged, without the merge's rows, so the session resumes unmerged and its Discard or Retry is an ordinary one. Phase 3 of the load design (`docs/dev/design-load-restores-the-save.md` section 8) deleted the handling an in-session resume of such a session needed: the prune of the rows the first merge's attempt wrote (`MergeDialog.PruneStagedRowsNamingAttempt` and its writer-side selectors) and Retry's restore of a detached tree. Both discards still identify the attempt by its session stamp (`MergeDialog.IsCreatedByReFlySession`), so another session's fork on the same rewind point, such as an earlier merged Re-Fly of the rewind point's other slot, is committed history the discard leaves alone with its rows, tombstones and stamps (owner ruling D2). Section 2.12 note (owner ruling 2026-10-19): every Re-Fly discard removes the ledger rows tagged to the attempt's recordings and is never refused. The Esc-menu Discard still puts a tree no committed copy backs back committed through `MergeDialog.RestoreDetachedTreeAfterEscDiscard` (todo ESC-DISCARD-REFLY-AFTER-A-QUICKLOAD-MAY-LOSE-THE-TREE). Retry cancels the attempt too: `RevertInterceptor.RetryHandler` calls `MergeDialog.RepairSessionBeforeRetry` after it clears the marker and before `RewindInvoker.StartInvoke` captures the reconciliation bundle, and it removes the attempt's ledger rows. Log `[MergeDialog] RepairSessionBeforeRetry callSite=RetryHandler ... treeState=<no-tree|still-committed|not-committed> ... stagedRowsPruned=0 ledgerRowsRemoved=N restoredCommittedTree=false` (the two fixed tokens stay for the lanes that read the line).

`GameEvents.OnRevertToLaunchFlightState` / `OnRevertToPrelaunchFlightState` both fire from inside the respective stock body (after the state roll-back, before the scene unload). Because the interceptor's prefix returns `false` to block the body, neither event fires during an active session — `RevertDetector.PendingKind` stays `None` and `ParsekScenario.OnLoad`'s revert-aware branch is correctly skipped. `DiscardReFlyHandler` never re-calls the stock revert methods (it drives `LoadGame` + `LoadScene` directly), so `OnRevertToLaunchFlightState` stays un-fired across the whole revert flow; any post-load `RevertDetector` state has already been consumed (or is irrelevant) by the time the new scene's scenario loads.

6.15 Session end without merge

- **Return to Space Center** (without clicking Merge): `ParsekScenario.OnDestroy` / the merge-dialog-discard path clears the marker + discards the provisional + purges session-provisional RPs. It also removes the attempt's ledger rows (`DiscardReFlyAttemptRecordingsAndRewindPoints` calls `Ledger.RemoveActionsForRecordings`, section 2.12 Rule 3).
- **Quit without returning to Space Center**: marker stays on disk. Load-time validation (§6.16) decides what to do with it.

6.16 Load-time sweep

File: `Source/Parsek/LoadTimeSweep.cs`.

Save-file loads are cold (owner ruling 2026-10-19, `docs/dev/design-load-restores-the-save.md` 4.1): a load of a save file stock announces (F9, the pause-menu Load, the main-menu Load; `SaveFileLoadIntent`, armed by `onGameStatePostLoad`) that is not a revert classifies `Cold` with `coldReason=save-file`. It reads every list below, the recordings and the ledger from that save; the in-session handoff is dropped (`save-file-load`). The in-session handoff and the OQ-1 resumed-merged-attempt handling described in 6.14 and here are therefore reached only by scene changes, stock reverts, the Discard Re-fly load and a load of a save no stock path announced (the `saveStamp` canary logs a Warn on one). Rewinds and Re-Fly loads keep D2. The machinery stays in place until Phase 3 of the load plan deletes what no load kind reaches.

Where the sweep's inputs come from. Committed recordings and the ledger survive every in-session load in memory, so the bookkeeping that indexes them does too. `ParsekScenario.OnDestroy` (stock `ScenarioRunner.OnGameSceneLoadRequested` destroys every scenario module when a scene load is requested, before the next scene's `Game.Load` creates the new instance; an `OnAwake` fallback captures from a predecessor still registered) hands the instance's rewind points, supersede rows, retirements, tombstones, merge journal and marker to the next OnLoad (`InSessionStagedStateHandoff`, `ParsekScenario.InSessionHandoff.cs`). Every OnLoad consumes it once: a cold load drops it (`cold-load`), a plain rewind keeps its own carry (`rewind-carry-owns`), a Re-Fly start its reconciliation bundle (`refly-bundle-owns`); the main-menu transition, a save-folder change, a save-file load (`save-file-load`) and the inert game-mode return drop it too. On every other load (revert, scene change, the Discard Re-fly load, a load no stock path announced) step A, right after the plain-rewind carries, installs memory's supersede rows, retirements and tombstones, the journal when memory's is null or `Complete` (memory drove or rolled back every journal it holds; an in-flight one keeps the loaded journal with a Warn), and, on the Discard Re-fly load only, memory's cleared marker; on the other kinds the marker comes from the save and step 1 below validates it. Step B, after the active-tree restore has detached a resumed tree and the pending-tree restore ran, partitions the rewind points by owner: a session-scoped point (`SessionProvisional` with a `CreatingSessionId`) is kept from either side, memory's instance first, and the spare / discard steps below decide it against the LOADED marker (a session the save never had is purged with its files); a point of a tree memory holds committed comes from memory (a point created after the save stays listed and gated by the future-RP rule; one reaped since is not resurrected); every other point (the resumed or reverted flight, an unknown owner) follows the save, its file untouched (owner ruling OQ-3, todo REWIND-POINT-FILES-OF-AN-ABANDONED-

FUTURE-STAY-ON-DISK). Supersede rows, retirements and tombstones all come from memory: Phase 3 of the load design deleted the resumed-tree row rule and the abandoned Re-Fly fork prune, which only an in-session resume of a tree committed after the save reached, and save-file loads are cold. An in-session load of a quicksave taken during a session that has since merged (a load no stock path announces; an F9 is a cold load and resumes it from the save, unmerged, with none of the merge's rows) keeps memory's rows, and the session's points come back session-scoped from the save. A scenario instance hands off only once its own OnLoad got through step A (an OnLoad that threw earlier leaves lists no later load should install): its teardown skips the capture with a Warn (`staged-state-not-loaded`) and drops the older handoff that OnLoad never took, so the next load keeps its save's lists. Logs: `[Rewind] Staged-list handoff captured reason=<OnDestroy|OnAwake> ...` , `... dropped reason=<r>` , `In-session staged lists from memory: kind=<k> ...` , `In-session RP owner partition: ...` , and a Warn when an in-session load finds no handoff (the save's lists then stand).

Runs from `ParsekScenario.OnLoad` AFTER `MergeJournalOrchestrator.RunFinisher` and `RewindInvoker.ConsumePostLoad` , and BEFORE `RewindPointReaper.ReapOrphanedRPs` . Single-pass gather-then-delete (eligibility decisions are made against a stable snapshot before any mutation):

1. **Marker validation** (`MarkerValidator.Validate` , `MarkerValidator.cs:86`). Six durable fields checked in order: `SessionId` non-empty; `TreeId` present in `RecordingStore.CommittedTrees` ; `ActiveReFlyRecordingId` resolves in `CommittedRecordings` with `MergeState == NotCommitted` ; `OriginChildRecordingId` resolves in `CommittedRecordings` ; `RewindPointId` resolves in `ParsekScenario.RewindPoints` ; `InvokedUT` is not strictly greater than current UT. The seventh field (`InvokedRealTime`) is informational only and does not fail a load. On any failure, `[ReFlySession] Warn: Marker invalid field=<f>; clearing` + marker cleared before the sweep continues (nested cleanup below depends on the cleared marker's session id).
2. **Spare set.** If marker valid: `ActiveReFlyRecordingId` and `RewindPointId` are in the spare set; additionally, any Recording with `CreatingSessionId == marker.SessionId` and any RP with `CreatingSessionId == marker.SessionId` join the spare set.
3. **Discard set.** Any Recording with `MergeState == NotCommitted` and not in the spare set is a zombie. Any RP with `SessionProvisional == true` , non-empty `CreatingSessionId` , and not in the spare set is a dead session-provisional RP. RPs with `CreatingSessionId == null` are normal staging RPs and are retained.
4. **Orphan supersede relations.** Walk `RecordingSupersedes` ; log Warn for each relation whose `OldRecordingId` or `NewRecordingId` doesn't resolve. NOT removed — invariant 7.
5. **Orphan tombstones.** Walk `LedgerTombstones` ; log Warn for each `ActionId` that doesn't resolve in `Ledger.Actions` . NOT removed.
6. **Stray `SupersedeTargetId` .** Walk committed recordings; for each `Immutable` or `CommittedProvisional` recording with non-empty `SupersedeTargetId` , log `[Recording] Warn: Stray SupersedeTargetId on committed rec=<id>; treating as cleared` and clear the field.
7. **Nested session-prov cleanup** (§7.11). When step 1 cleared the marker, count the discard-set entries that carried the cleared marker's session id and emit `[ReFlySession] Info: Nested session-prov cleanup sess=<sid> recordings=<n> rps=<m>` . (The discard set already caught them — this is an observational log for the §7.11 contract.)
8. **Delete.** Remove discard-set recordings from `RecordingStore` , and (section 2.12 Rule 3, designed 2026-10-19) the ledger rows tagged to them, so a crash-quit session leaves the ledger as it was before the invocation. Remove discard-set RPs from `ParsekScenario.RewindPoints` (and delete their quicksave files best-effort; a lock / IO failure Warns but still clears the scenario entry).
9. **Summary log.** Single `[LoadSweep] Info: Marker valid=<b>; spare=<n> discarded=<r> orphanSupersedes=<os> orphanTombstones=<ot> strayFields=<s> discardedRps=<rps>` .
10. **Cache invalidation.** `BumpSupersedeStateVersion` + `BumpTombstoneStateVersion` + `EffectiveState.ResetCachesForTesting` so ERS / ELS rebuild on next access.

6.17 Tree-scoped rewind-state purge

File: `Source/Parsek/TreeDiscardPurge.cs` .

Triggered by merge-discard (`RecordingStore.DiscardPendingTree`) or by any other path that discards a whole tree. Note: the Revert-during-re-fly dialog's Discard Re-fly button does NOT reach this path — design §6.14's `DiscardReFlyHandler` is session-scoped and leaves the tree's supersede / tombstone / RP state untouched. `TreeDiscardPurge.PurgeTree(treeId)` removes every Rewind-to-Separation artifact tied to the tree:

- Remove every RP whose `BranchPointId` is in the tree; delete each RP's quicksave file on disk.
- Remove every `RecordingSupersedeRelation` with either endpoint in the tree.
- Remove every `LedgerTombstone` whose target `GameAction.RecordingId` is in the tree.
- Clear `ActiveReFlySessionMarker` if it references the tree.
- Clear `ActiveMergeJournal` if it references the tree.
- Recompute crew reservations via `CrewReservationManager.RecomputeAfterTombstones` when tombstones shrank.

What this does NOT do: it does not remove the tree's committed recordings themselves and does not delete the tree from `RecordingStore.CommittedTrees`. Removal of the recordings (for a pending tree being discarded before commit) is the caller's responsibility — `TreeDiscardPurge.PurgeTree` is invoked by `RecordingStore.DiscardPendingTree` BEFORE the recordings are removed from `CommittedRecordings` so the purge can still resolve recording ids to tree membership. The Revert-during-re-fly dialog's Discard Re-fly button is explicitly NOT a caller of this purge — it only clears the session artifacts and reloads the origin RP's quicksave; the tree's other RPs / supersede relations / tombstones are preserved (design §6.14).

Purging a tree's rewind-state is the **only** path that deletes supersede relations or ledger tombstones (invariant 7). Orphans left by partial crash states stay in place and are logged on every load.

6.18 Disk-usage diagnostic

File: `Source/Parsek/RewindPointDiskUsage.cs`.

Surfaced in Settings → Diagnostics as `Rewind points on disk: <size> (<N> files)`, with the live bucket counts in its hover. Backed by a 10-second snapshot cache — the directory is walked once per cache miss, so the Settings window doesn't thrash the filesystem on every repaint. The follow-up adds split buckets next to the byte/file total, derived from each slot's effective tip MergeState: live crashed-open RPs (a NotCommitted tip), live stable-open RPs (a CommittedProvisional tip), and concluded RPs (all tips Immutable). Buckets are explanatory and may overlap when a single RP contains both concluded and still-open slots; this is monitoring only, not an auto-purge trigger.

6.19 The Unfinished Flights predicate (v0.9.1)

`EffectiveState.IsUnfinishedFlight(Recording rec)` is rewritten in v0.9.1 to use the shared classifier. The full evaluation:

```
IsUnfinishedFlight(rec) :=
    // Filter 1: visible, committed recording
    rec is in ERS                                              // §1
    AND rec.MergeState in { Immutable, CommittedProvisional }       // §2

    // Filter 2: controllable subject at chain HEAD
    AND chainHead.IsDebris == false                                 // §3

    // Filter 3: matching RP with an open slot, with per-RP-context leaf check
    AND exists RP, exists slot in RP.ChildSlots such that:
        // Slot resolution -- v0.9.0 logic, unchanged shape
        slot.EffectiveRecordingId(supersedes) == rec.RecordingId
        AND (rec.ParentBranchPointId == RP.BranchPointId
             OR rec.ChildBranchPointId == RP.BranchPointId)
                                                                    // §4
        // Slot-open gate: open iff the slot's effective tip is CommittedProvisional
        AND slot.EffectiveTip.MergeState == CommittedProvisional    // §5

        // Per-RP leaf gate
        // The vessel's real tip: chain hops, switch continuations, and
        // (section 1.6) the vessel's own EVA + same-vessel Board, walked from
        // the slot stretch's head so every stretch member agrees
        AND let walk     = WalkSlotVessel(StretchHead(rec, slot))
        AND let chainTip = walk.Tip
            (chainTip.ChildBranchPointId == null
             OR chainTip.ChildBranchPointId == RP.BranchPointId)    // §6
        // section 1.6 rule 2: an own-EVA kerbal who joined a foreign vessel
        AND NOT walk.HasForeignJoin                 // evaCrewJoinedForeignVessel

        // Outcome gate
        AND TerminalOutcomeQualifies(chainTip, slot, RP)            // §7

        // Value gate (section 2.12 Rule 1, designed 2026-10-19). Run inside
        // every qualifying arm of TerminalOutcomeQualifies (crashed,
        // stashedStableLeaf, stableLeafUnconcluded, the focus override), as
        // the ScienceEarning-only recording-action check it replaced was,
        // so the verdict reason stays recordingAction:<Type>:<ActionId>.
        AND NOT RetireSetHoldsSealingRow(slot)                      // section 2.12

TerminalOutcomeQualifies(chainTip, slot, RP) :=
    let kerbal   = !string.IsNullOrEmpty(chainTip.EvaCrewName)
    let terminal = chainTip.TerminalStateValue
    let isFocus  = (slot.SlotIndex == RP.FocusSlotIndex)
    let noFocusSignal = (RP.FocusSlotIndex == -1)
```

```
    // 2026-09-27 ruling (§1.5): Re-Fly is for vessel separations only.
    // An EVA kerbal never qualifies, whatever its terminal (Destroyed
    // included). Runs before the terminal is read.
    if kerbal:
        return false                        // evaNotSeparation

    if !terminal.HasValue:
        return false                        // no terminal recorded

    // Crashed always qualifies, regardless of focus/everything.
    if terminal.Value == Destroyed:
        return true                         // Crashed

    // Stable in-flight terminals: only non-focus slots on RPs with a
    // defined focus signal qualify. noFocusSignal short-circuits to false.
    if noFocusSignal:
        return false

    if terminal.Value == Orbiting   && !isFocus: return true
    if terminal.Value == SubOrbital: return true
        // SubOrbital qualifies regardless of focus: a suborbital arc is
        // still in flight (the vessel will crash, land, splash, or with
        // a burn reach orbit), so the slot stays open for both the focus
        // and non-focus paths. Atmospheric SubOrbital that decays into
        // the planet is reclassified to Destroyed by
        // BallisticExtrapolator / IncompleteBallisticSceneExitFinalizer
        // before commit; what reaches here is a vacuum-arc SubOrbital
        // that may circularize, decay later, or land/splash on a body
        // with terrain.

    // Landed / Splashed / Recovered / Docked: stable surface or recovered
    // terminal. The universe gave the vessel a stable conclusion. Default
    // does not include the row regardless of focus.
    return false
```

The implementation lives in a new `UnfinishedFlightClassifier` static class (§6.20). The signature matches the v0.9.0 `IsUnfinishedFlight(Recording rec)` for backwards-compatible callers, but the implementation routes through `UnfinishedFlightClassifier.Qualifies(rec, slot, rp, considerSealed: true)` after the slot+RP resolution.

6.20 Classifier extraction (v0.9.1)

> SUPERSEDED (collapse-seal-into-mergestate): the `considerSealed` parameter on `Qualifies` / `TryQualify` and the `slot.Sealed` gate were removed. `TryQualify` is now a shape-only predicate; the open/closed read moved to `EffectiveState.TryResolveUnfinishedFlightRaw`, which gates on the slot's effective tip `MergeState` (open iff `CommittedProvisional`). Read `considerSealed: ...` arguments and `slot.Sealed = ...` steps in this section as "the tip-`MergeState` open-check" / "set the tip recording's `MergeState`". See §2.2 and the plan `docs/dev/done/plans/collapse-seal-into-mergestate.md`.

A new file `Source/Parsek/UnfinishedFlightClassifier.cs` (or alternatively all-in-one in `EffectiveState.cs`) owns:

- `Qualifies(Recording rec, ChildSlot slot, RewindPoint rp, bool considerSealed)` — the predicate body.
- `TerminalOutcomeQualifies(Recording chainTip, ChildSlot slot, RewindPoint rp)` — the outcome gate.

The following helpers move from `UI/RecordingsTableUI.cs` into this file (or `EffectiveState.cs`):

- `TryResolveRewindPointForRecording(Recording rec, out RewindPoint rp, out int slotListIndex)`
- `IsUnfinishedFlightCandidateShape(Recording rec)`
- `IsVisibleUnfinishedFlight(Recording rec, out string reason)`

`RecordingsTableUI` becomes a consumer of the moved helpers. Other consumers (Site A and Site B-1 predicate-call paths in §6.21, plus the Seal handler in §6.24) call them from non-UI namespaces. Site B-2 reads Site B-1's verdict downstream and does not call the predicate directly; it lives in `MergeDialog` and consumes the marker + the just-classified provisional state.

6.21 MergeState promotion: Site A display predicate, Site B merge-close policy (v0.9.1)

Site A decides whether a newly committed original-tree recording appears in Unfinished Flights. Site B decides what happens after the player re-flies one of those slots and commits the attempt back to the timeline. The policies share the same predicate but Site B also applies merge-time conclusion and safety gates: chosen stable leaves auto-seal, terminal failures can remain retryable, while

recovered/docked/boarded terminals, downstream structural/world interactions, and a sealing row in the slot's retire set (section 2.12 Rule 1) closes the slot.

Site A: original tree commit. `RecordingStore.ApplyRewindProvisionalMergeStates` (`RecordingStore.cs:715-770`) extends from the v0.9.0 Crashed-only check to the broader predicate:

```
for each rec in tree.Recordings:
    if rec.MergeState != Immutable: continue
    if rec.chainHead.IsDebris: continue

    if NOT TryResolveRewindPointForRecording(rec, out rp, out slotIdx):
        continue
    var slot = rp.ChildSlots[slotIdx]

    // considerSealed=false: a freshly committed slot is never Sealed yet.
    if NOT UnfinishedFlightClassifier.Qualifies(
            rec, slot, rp, considerSealed: false):
        continue

    rec.MergeState = CommittedProvisional
    log [UnfinishedFlights] Info: CommitTree promoted rec=<rid>
        slot=<slotIdx> rp=<rpId> reason=<crashed|stableLeafUnconcluded>
```

The existing v0.9.0 crash-only path is subsumed: `Qualifies` returns true for `terminal == Destroyed` regardless of focus.

Site B-1: re-fly merge classifier flip. `SupersedeCommit.FlipMergeStateAndClearTransient` resolves the RP slot, evaluates the UF predicate for diagnostics with the merge-time slot supplied as `focusSlotOverride` (§4.9), then applies the merge-close gate:

```
// Slot resolution: walk supersedes from each slot's OriginChildRecordingId
// forward; the helper returns the slot whose forward trail contains the
// queried provisional. After Phase 2 (AppendRelations) of the merge journal,
// the new {priorTip -> provisional} relation is in supersedes (per §6.22),
// so the walk reaches the provisional cleanly.
if NOT TryResolveRewindPointForRecording(provisional, out rp, out slotIdx):
    // Hard failure in DEBUG (Debug.Assert + log Error); fall back to the
    // v0.9.0 default in RELEASE for crash safety. A release-build occurrence
    // indicates a §6.22 prerequisite regression or AppendRelations bug --
    // not a recoverable runtime state.
    log [Supersede] Error: Site B-1 slot lookup failed for provisional=<rid>
        rpId=<marker.RewindPointId> originChildRec=<marker.OriginChildRecordingId>
        supersedeTargetId=<marker.SupersedeTargetId>
    Debug.Assert(false, "Site B-1 slot lookup failed")
    provisional.MergeState = (Classify(provisional) == Crashed)
                             ? CommittedProvisional : Immutable
    return

var slot = rp.ChildSlots[slotIdx]
// focusSlotOverride: the slot the player chose to Re-Fly is the
// merge-time focus regardless of rp.FocusSlotIndex. Inside the
// classifier, slotListIndex == focusSlotOverride takes the
// stableTerminalFocusSlot branch on Orbiting tips, after
// the existing static-focus and section 2.12 value-gate checks
// have run.
bool qualifies = UnfinishedFlightClassifier.Qualifies(
    provisional, slot, rp, considerSealed: false,
    focusSlotOverride: slotIdx)

// Terminal failure: Destroyed. Crashed retries keep their existing
// keep-open path so the player can retry. (An EVA kerbal never reaches
// here: the classifier rejects it as evaNotSeparation.)
bool terminalFailure =
    terminal == Destroyed

bool hasRetryBlockingPlayerAction =
    RetireSetHoldsSealingRow(slot)   // the slot's tip is the provisional now
    // section 2.12 Rule 1 (designed 2026-10-19): any sealing row the re-fly
    // filed (every gain, contract outcome and flight-tagged spend) seals;
    // only loss rows (crew deaths and bookkeeping, crew-death and
    // vessel-loss reputation, building damage) leave the retry path open.
    // Shipped today: HasRetryBlockingRecordingAction, credited
    // ScienceEarning only.
```

```
// Structural mutation during this Re-Fly session: any BranchPoint with
// Type in {Breakup, Undock, EVA, JointBreak} authored at UT > rp.UT
// AND not already in marker.PreSessionBranchPointIds (the snapshot
// captured at marker creation, persisted on the marker, used to filter
// out spliced-back pre-session BPs). See §4.9.
bool hasStructuralMutation =
    HasReFlySessionStructuralMutation(provisional, marker)

bool keepOpen =
    qualifies
    AND NOT hasRetryBlockingPlayerAction
    AND terminalFailure                 // ONLY terminal failure keeps open
    AND NOT hasStructuralMutation       // ... and no structural mutation

provisional.MergeState = keepOpen
    ? CommittedProvisional
    : Immutable

// Auto-seal triggers: stableTerminalFocusSlot (player-chosen slot
// reached stable terminal), structuralMutation (session authored a
// new structural BP), recordingAction (a sealing row, section 2.12),
// downstreamBp (chain tip's child BP resolves to a deeper RP), and the
// existing hard-safety terminals (Recovered/Docked/Boarded).
if (!keepOpen AND closeReason is an auto-seal reason)
    slot.Sealed = true

log [Supersede] Info: provisional=<rid> mergeState=<state> qualifies=<b>
    slot=<slotIdx> rp=<rpId> focusSlot=<rp.FocusSlotIndex> focusSlotOverride=<slotIdx>
    classifierReason=<reason> autoSeal=<b> autoSealReason=<reason>
```

Under the v0.9.1 auto-seal revision (section 4.9), a `stableLeafUnconcluded` reason can no longer come out of Site B-1 because `focusSlotOverride: slotIdx` reroutes the player-chosen-slot Orbiting/SubOrbital path to `stableTerminalFocusSlot` and the `!keepOpen` branch. `stableLeafUnconcluded` still appears at Site A and from non-Re-Fly callers (UI display, reap eligibility), where the override is `null`. Stable retry-keep-open at merge time is therefore deliberately narrowed to terminal-failure outcomes (Destroyed) without a sealing row (section 2.12). If the player wants the stable Re-Fly to remain re-flyable, they unstash via the manual STASH affordance from a separate slot - or they don't merge yet (Discard/Retry the Re-Fly).

Site B-2: in-place continuation merge handling. `MergeDialog.TryCommitReFlySupersede` consumes Site B-1's result. Under the v0.9.1 revision, clean stable in-place outcomes commit `Immutable` + auto-seal at Site B-1, so Site B-2 lets the reaper remove the RP. Safety-closed outcomes follow the same path. Terminal-failure outcomes (`Destroyed`) without a sealing row (section 2.12) still stay `CommittedProvisional` and remain retryable from the same RP.

6.22 Closure-helper split + invocation linearization (v0.9.1 prerequisite)

The v0.9.0 invocation stamped `marker.OriginChildRecordingId` and `provisional.SupersedeTargetId` from `selected.OriginChildRecordingId` (the slot's immutable origin). `SupersedeCommit.AppendRelations` then wrote `{slot.OriginChildRecordingId -> provisional}` for every re-fly. Multiple re-flies into the same slot produced a star-shaped graph rooted at the slot's origin. The walker `EffectiveState.EffectiveRecordingId` scans supersedes from the beginning and stops at the first match — on a star, it resolves to the oldest re-fly, missing later ones. Site B-1's slot lookup against the provisional then failed.

Linear semantics + closure-helper split. Marker-write change in `RewindInvoker.AtomicMarkerWrite`:

```
// Compute prior tip ONCE before the in-place vs fresh-provisional branch.
string priorTip = selected.EffectiveRecordingId(scenario.RecordingSupersedes)

// Stamp BOTH marker fields unconditionally in the shared marker-creation
// block (runs on both branches).
marker.OriginChildRecordingId = selected.OriginChildRecordingId   // unchanged
marker.SupersedeTargetId      = priorTip                          // NEW (v0.9.1)

// Guard the provisional overwrite to the fresh-provisional branch only.
// In-place branch sets provisional == null.
if (provisional != null)
    provisional.SupersedeTargetId = priorTip                      // overwrite the
                                                                  // BuildProvisionalRecording
                                                                  // value
```

Closure-helper split in `EffectiveState`:

```
// New: takes an explicit root, parameterized over the existing closure
// algorithm (PID-peer expansion via marker.InvokedUT, mixed-parent halt,
// chain-sibling expansion, all preserved). Cache key includes rootOverride.
internal static IReadOnlyCollection<string> ComputeSubtreeClosureInternal(
    ReFlySessionMarker marker, string rootOverride)
{
    // ... existing closure body, with rootOverride substituted for
    // marker.OriginChildRecordingId at the seed point.
}

// Existing public helper: thin wrapper preserving null-guard +
// cached-null-guard + defensive-copy contracts.
public static IReadOnlyCollection<string> ComputeSessionSuppressedSubtree(
    ReFlySessionMarker marker)
{
    if (marker == null)
        return Array.Empty<string>();
    var cached = ComputeSubtreeClosureInternal(
                    marker, marker.OriginChildRecordingId);
    if (cached == null)
        return Array.Empty<string>();
    return new HashSet<string>(cached, StringComparer.Ordinal);
}
```

`SupersedeCommit.AppendRelations` switches to call:

```
ComputeSubtreeClosureInternal(marker,
    marker.SupersedeTargetId ?? marker.OriginChildRecordingId)
```

`CommitTombstones` continues to receive the closure unchanged; tombstone scope correctly tracks the chain-extension's actual descendants.

Runtime ghost suppression (ghost playback engine, chain walker, ghost map presence, watch mode) continues calling `ComputeSessionSuppressedSubtree(marker)` and gets the same return value as today.

Migration concern. Existing saves with star-shaped supersede graphs from prior v0.9.0 Crashed chain extensions are tolerated as-is. The walker continues to pick the oldest re-fly in the star portion, then walks linearly from there. New chains extend linearly from whichever leaf the walker reached. The hybrid case (legacy star + new linear extension) is covered by §7.72.

6.23 Reaper rule

> Updated by collapse-seal-into-mergestate: the rule reads open/closed solely from the slot's effective chain+supersede tip `MergeState` (no `slot.Sealed` term, no `sealedSlotsContributing` counter).

`RewindPointReaper.IsReapEligible`:

```
SlotIsClosed(slot, effectiveTipRecording) :=
    effectiveTipRecording == null                       // null / orphan tip -> closed
    OR effectiveTipRecording.MergeState == Immutable    // sealed / concluded / canon

RP is reap-eligible iff (not SessionProvisional) AND every ChildSlot satisfies SlotIsClosed.
```

A tip in `CommittedProvisional` (re-flyable Unfinished Flight or stashed stable leaf) or `NotCommitted` (recorder still running) is OPEN and blocks reap. Keeping `NotCommitted` open is the live-recorder guard: never delete an RP quicksave while an active re-fly is still recording.

Logging on reap:

```
[Rewind] Info: ReapOrphanedRPs: reaped=<R> remaining=<rem>
    fileDeleteOk=<n> fileDeleteFail=<n> bpBackrefCleared=<n>
```

6.24 The Seal handler (v0.9.1)

> SUPERSEDED (collapse-seal-into-mergestate): the original v0.9.1 design below used a separate `ChildSlot.Sealed` bit decoupled from `MergeState`. That bit (and `SealedRealTime`, the `considerSealed` classifier parameter, and the

> `sealedSlotsContributing` reaper counter) was removed. Open/closed is now read solely from the slot's effective chain+supersede tip `MergeState`: `CommittedProvisional` = open, `Immutable` = sealed/closed. Seal flips the tip `CommittedProvisional -> Immutable` (permanent); the step "slot.Sealed = true" below is now "set the tip recording's MergeState = Immutable". The rest of the handler (dialog, lock, SupersedeStateVersion bump, reaperImpact, logging, persist-before-reap) is unchanged. See §2.2 and the plan `docs/dev/done/plans/collapse-seal-into-mergestate.md`.

The Seal action lives on each Unfinished Flight row in the Recordings tab. Visual layout per section 6.25.

Handler (in a new `UnfinishedFlightSealHandler` static class or as a method on the classifier):

```
1. Spawn Seal confirmation dialog (PopupDialog.SpawnPopupDialog with
   MultiOptionDialog body, see §6.25.2). Take input lock
   ParsekUFSealDialog.

2. On Cancel: log [UnfinishedFlights] Info: Seal cancelled rec=<rid>;
   release input lock; dismiss.

3. On Accept:
   - Locate slot via TryResolveRewindPointForRecording(rec, out rp, out slotIdx).
     If lookup fails: log [UnfinishedFlights] Error: Seal could not resolve
     slot for rec=<rid>; release input lock; dismiss; show toast
     "Seal failed -- slot not found." This should not happen for a row
     that was rendered as UF.
   - var slot = rp.ChildSlots[slotIdx]
   - resolve the slot's effective tip recording (slot.EffectiveRecordingId)
   - if tip.MergeState != Immutable: tip.MergeState = Immutable; tip.FilesDirty = true
   - Bump SupersedeStateVersion (so ERS / UF group cache invalidates).
   - Determine reaperImpact: walk all slots of rp, check SlotIsClosed for
     each; "willReap" if all closed, "stillBlocked" otherwise.
   - log [UnfinishedFlights] Info: Sealed slot=<slotIdx> rec=<rid>
       bp=<bpId> rp=<rpId> terminal=<state> reaperImpact=<willReap|stillBlocked>
   - Release input lock; dismiss; row drops from group on next frame
     because the predicate now sees the slot's tip MergeState == Immutable.
```

Seal flips the slot's effective tip `Recording.MergeState` from `CommittedProvisional` to `Immutable` (the single open/closed source of truth). This is permanent and never reversed in-game. See §2.2.

6.25 UI changes for Seal / Stash (v0.9.1)

6.25.1 Layout

The Recordings table has separate **Rewind** and **Re-Fly** columns. Rewind/Forward remain in the Rewind column so launch and timeline navigation stay visible. Unfinished-Flight actions live in the Re-Fly column: existing UF rows show `Fly` + `Seal`, and stable rows that are eligible for the manual escape hatch show `Stash` + `Seal`.

The separate Re-Fly column is chosen over widening/overloading the Rewind cell, a kebab menu, or right-click because:

- Rewind and Re-Fly are different workflows; keeping them in separate columns prevents stable-row Stash from hiding launch Rewind.
- UF rows need visible `Fly`, `Stash`, and `Seal` affordances. A menu would make the safety/cleanup path too easy to miss.
- KSP's stock UI has no strong right-click table-row precedent, and the mod already uses explicit row buttons for primary actions.

Cascade: every row in the Recordings tab gets the Rewind and Re-Fly columns. Non-UF rows leave the Re-Fly cell empty unless the row can be manually Stashed. Tooltip refresh on the UF group header.

The same cell appears in two more places. The Missions tab draws it in its `Interact` column at the Interact geometry (`RecordingsTableUI.DrawReFlyColumnCell`: `Fly` / `Stash` + `Seal` as two 48 px halves) on vessel and interval rows, in both UI complexity modes. The Timeline draws `Fly` + `Seal` on `UnfinishedFlightSeparation` rows, resolving the slot through the same `RecordingsTableUI.ResolveUnfinishedFlightRewindRoute`, and its `Re-Fly` view lists only rows whose `Fly` or `Seal` is actionable.

6.25.2 Seal confirmation dialog

`PopupDialog.SpawnPopupDialog` with a `MultiOptionDialog`. Title: `Confirm: Seal Unfinished Flight`. Dialog id and input-lock id: `ParsekUFSealDialog` (`UnfinishedFlightSealHandler.DialogName` / `DialogLockId`). Body, as emitted by `UnfinishedFlightSealHandler` (`UnfinishedFlightSealHandler.cs:160`):

```
Seal "<vessel-name>" (<terminal-state> at UT <ut>)?

This cannot be undone. After sealing, this entry is permanently merged
to the timeline in its current state.

If you might want to re-fly this later, click Cancel.
```

Buttons: `Seal Permanently` (destructive style, fires the seal handler), `Cancel`.

6.26 Unfinished Flights group tooltip refresh (v0.9.1)

The group draws in the Recordings tab under the name `STASH` (`UnfinishedFlightsGroup.GroupName`), and its header tooltip (`UnfinishedFlightsGroup.Tooltip`) reads:

> Crashed or stranded flights to re-fly from their separation. Only flights that earned nothing after it are listed. Fly takes control at the separation; Seal closes the slot for good.

The middle sentence is section 2.12's (Rule 1): it tells the player why a crashed flight that earned something is missing. The whole text stays inside the single-line Recordings help-strip budget (`TooltipEchoBudgetTests.UnfinishedFlightsGroupTooltip_FitsTheRecordingsStrip`), which is why the older wording ("Vessels and kerbals you may want to re-fly - crashed, abandoned in orbit, stranded on a surface") was shortened rather than extended; an EVA kerbal is never offered anyway (section 1.5).

---

7. Edge Cases

Each entry: scenario / expected behavior / shipped status. Status codes: **Shipped (test)** = dedicated unit / integration test; **Shipped (integration)** = covered by multi-test wiring or in-game test; **Deferred (v1 limitation)** = explicitly not in v1, documented; **N/A** = design evolved out. Test names name `*.cs` test classes in `Source/Parsek.Tests/`.

7.1 Single-controllable split

No RP. Debris path unchanged. **Shipped (test)**: `MultiControllableClassifierTests`.

7.2 Three or more controllable children at one split

One RP with N ChildSlots + N entries in each PID map. Player iterates through the slot rewind buttons. **Shipped (test)**: `RewindPointRoundTripTests` + `RewindPointAuthorTests`.

7.3 Scene-transition during deferred save

§6.6 step 1 of the deferred coroutine re-checks the scene guard; if we left FLIGHT, `RollbackBegin` removes the RP and clears the BranchPoint back-reference. Log `[RewindSave] Warn: Aborted mid-coroutine: scene=<loaded>`. **Shipped (test)**: `RewindPointAuthorTests`.

7.4 Partial PidSlotMap failure at split

One slot's lookup fails at split time; RP still created with partial maps. The affected slot is marked `Disabled = true` with reason `no-live-vessel`; other slots usable. Log `[Rewind] Warn: Slot <i> disabled: no live vessel`. **Shipped (test)**: `RewindPointAuthorTests`.

7.5 Invoking RP while a re-fly is active

UI + CanInvoke gate rejects with `"Another re-fly session is already active"`. No nested sessions in v1. **Shipped (test)**: `RewindTests` (precondition matrix).

7.6 Re-fly target with BG-recorded descendants

SessionSuppressedSubtree includes descendants (forward-only closure). Merge step 2 adds supersede relations for each descendant. **Shipped (test)**: `SessionSuppressionWiringTests` + `SupersedeCommitTests`.

7.7 Cross-tree dock during re-fly

Station `S` from a different tree's Immutable recording is real at its chain tip. The re-fly can physically dock. Dock event is added to the re-fly's recording. No cross-tree merge; S's tree is not modified. **Acceptable v1 limitation**: no dedicated in-game test for cross-tree dock; covered by `InvokeRPStripAndActivateTest` behavior when a docked station is present.

7.8 Non-Parsek stock vessel interaction

Standard — the stripper leaves unrelated vessels alone (§6.9 step 4). **Shipped (test)**: `PostLoadStripperTests`.

7.9 Visual divergence for cross-recording stock vessel

Ghost-B recorded a dock with S at UT=X; the live re-fly also docks with S at UT=X−100 and moves S. Ghost-B's playback tries to dock with S-where-ghost-B-remembers-it. Accepted v1 visual limitation.

7.10 EVA duplicate on load

Strip detects a kerbal present in EVA form AND in a source vessel's crew. Canonical live location is the selected child's crew manifest; dedup via KSP crew removal. **Shipped (integration)**: KSP's own reload logic handles the dedup; the stripper's ghost-ProtoVessel guard + left-alone path cover the remaining cases.

7.11 Nested session-provisional cleanup on parent discard

Nested RP's `CreatingSessionId` matches the discarded session; load-sweep spare-set logic removes it. **Shipped (test)**: `LoadTimeSweepTests`.

7.12 F5 mid-re-fly + quit + load

Marker validates; all session-tagged recordings AND RPs spared; re-fly resumes. Atomic §6.10 write means no save can capture an intermediate state. **Shipped (integration)**: in-game `F5MidReFlyResumeTest`.

7.13 Contract supersede retires old-branch contract rows

Unreachable under section 2.12 (designed 2026-10-19): a `ContractComplete` is a sealing row, so a flight that completed a contract after its separation is not offered for a Re-Fly. The text below is the shipped behaviour, kept for the patcher contract it documents (the same patch path still runs for a tree discard). BG-crash completed contract X. Player re-flies, merges. The old `ContractComplete` action is tombstoned, disappears from ELS, and the post-tombstone patch removes unsupported active stock contract state plus terminal stock state only for the explicit tombstoned Parsek contract id. Terminal preservation is outcome-aware, so a surviving retry failure/cancel for the same contract id does not preserve the old completed stock row. A retry completion can then be credited from the surviving branch subject to normal once-ever and deadline rules. **Shipped (integration)**: `TombstoneEligibilityTests` and `PatchContractsPreservationTests`. The in-game `ContractTombstonesAcrossSupersedeTest` no longer drives this path (it skipped on every host because the state is unreachable); since 2026-10-24 it asserts section 2.12 in a live session: no contract row in the merge's retire set, and no stock Active contract accepted after the session clock.

7.14 Contract failed by BG-crash; re-fly succeeds

Unreachable under section 2.12: a `ContractFail` is a sealing row (during a session stock already holds the contract as failed, so the re-fly could never fail it again and retiring the old failure would erase its penalty for good). A booster whose crash failed a contract is not re-flyable. Shipped behaviour, kept for the patcher contract: BG-crash failed contract X. The old `ContractFail` action is tombstoned with the old subtree. The contract patcher removes unsupported terminal stock state only for that explicit tombstoned Parsek contract id when the surviving ledger no longer carries a compatible fail/deadline-expired outcome, so a retry can supply the canonical result without erasing unrelated stock finished history.

7.15 Milestone earned by superseded recording

Unreachable under section 2.12: a `MilestoneAchievement` is a sealing row. It had to be: during a session stock already counts the milestone as achieved (the invoke walk replays the whole ledger), so the re-fly cannot earn it again, and the merge used to retire the original, losing the reward (the LAB-10 log shows `set achieved 'Kerbin/Science'` followed by `cleared achieved`). Shipped behaviour: the old milestone action is tombstoned with the old subtree. The milestone/repeatable-record patch path rebuilds authored progress from the surviving ledger state.

7.16 Kerbal death + recovery

Still the case Re-Fly exists for (section 2.12 keeps deaths in the loss class). `KerbalAssignment`-to-Dead + bundled `ReputationPenalty` tombstoned at merge; the death penalty goes through the loss swap (section 2.12 Rule 2), so a re-fly that loses more crew than the original keeps the original's reputation penalty instead of a larger one. `CrewReservationManager.RecomputeAfterTombstones` re-derives; kerbal returns to active. **Shipped (integration)**: `CrewReservationRecomputeTests` + in-game `KerbalRecoveryOnSupersedeTest`.

7.17 Re-fly crashes; player merges

Re-fly commits as `CommittedProvisional` with `TerminalKind = Crashed` per `TerminalKindClassifier`. Supersede relation appends (`A → A'`). A' satisfies `IsUnfinishedFlight` only if A' filed no sealing row (section 2.12: a crash after a science transmission, a milestone or a contract result seals). Player CAN invoke RP again to try once more. If A' lost more reputation than A (more crew killed), the loss swap keeps A's reputation penalty (section 2.12 Rule 2). **Shipped (integration)**: in-game `MergeCrashedReFlyCreatesCPSupersedeTest`.

7.18 Re-fly reaches stable outcome; player discards at dialog

Provisional discarded; marker cleared; original BG-crash remains Unfinished; RP durable. **Shipped (integration)**: the discard path drops the provisional via `RecordingStore.RemoveCommittedInternal` + `ActiveReFlySessionMarker = null`.

7.19 Simultaneous multi-controllable splits

Two independent RPs; two quicksave writes serial through KSP. **Shipped (integration)**: `RewindPointAuthorTests` (deferred-one-frame ensures serialization through KSP's save pipeline).

7.20 F9 during re-fly

Standard quickload + auto-resume. Marker validates against the loaded state; if consistent, the session continues. **Shipped (integration)**.

7.21 Warp during re-fly

Standard Parsek warp. Not a new code path.

7.22 Rewind click during scene transition

CanInvoke returns `"Scene transition in progress - please wait"`. **Shipped (test)**: `RewindTests`.

7.23 Two children crash at same location

Independent rewinds via independent RPs.

7.24 Re-fly vessel with no engines

Allowed. The rewind only restores a quicksave state.

7.25 Drag Unfinished Flight into manual group

`GroupPickerUI.CanAddToUserGroup` rejects with `[UnfinishedFlights]` Verbose log + ScreenMessages toast. **Shipped (test)**: `UnfinishedFlightsDragRejectTests`.

7.26 Mod-modified joint-break / destruction

Classifier is `ModuleCommand`-based; mods that add alternative controller-like parts may not be seen. **Deferred (v1 limitation)** outside stock parts.

7.27 High physics warp at split

Forced to 0 during save (`RewindPointAuthor.ExecuteDeferredBody` step 2). **Shipped (integration)**: in-game `WarpZeroedDuringSaveTest`.

7.28 Disk usage diagnostics

Settings → Diagnostics shows total RP disk usage with a 10-second cache. **Shipped (test)**: `DiskUsageDiagnosticsTests`.

7.29 Mod part missing on re-fly load

Deep-parse precondition (`RewindInvoker.CanInvoke` step 6) walks the quicksave's PART nodes through `PartLoader.getPartInfoByName`. Any missing part marks the RP `Corrupted` and disables the Rewind button. **Shipped (test)**: `RewindTests` precondition matrix.

7.30 Cannot hide Unfinished Flights group

`GroupHierarchyStore.CanHide` denies. UI hide checkbox is not drawn for the virtual group. **Shipped (test)**: `UnfinishedFlightsGroupCannotHideTests`.

7.31 Non-crashed BG sibling — focus vs non-focus (v0.9.1 contract)

Terminal kind classifies as `Landed` / `Orbiting` / `SubOrbital` / `InFlight` via `TerminalKindClassifier`. v0.9.1's broadened predicate splits this case in two:

- **Focus-continuation slot** (`slot.SlotIndex == RP.FocusSlotIndex`) reaching `Landed` / `Splashed` / `Orbiting` / `SubOrbital`: `IsUnfinishedFlight` returns **false**. The recording does NOT enter the Unfinished Flights group and NO Re-Fly button is drawn — this slot is the player's continued mission, and broadening to it would pollute the list with every routine upper-stage-to-orbit. The recording plays back from its committed trajectory and the end-of-recording spawn puts the real vessel where it actually ended up. Manual Stash (§6.25.1) is the explicit escape hatch when the player decides post-hoc that they want such a slot re-flyable.
- **Non-focus slot** reaching `Orbiting` / `SubOrbital`: `IsUnfinishedFlight` returns **true** under the v0.9.1 stable-leaf contract (the 4-probe deploy case in §4.4, the upper-stage-left-coasting case in §4.5). Site A promotes the recording to `CommittedProvisional` and the row appears in Unfinished Flights with `Fly` + `Seal` affordances. `Landed` / `Splashed` / `Recovered` / `Docked` / `Boarded` non-focus terminals still return false (stable conclusion, no re-fly opportunity by default); manual Stash is available for `Landed` / `Splashed`.
- **EVA kerbal** (`EvaCrewName != null`): never qualifies, whatever its terminal (`evaNotSeparation`, §1.5 / §4.6).
- **Legacy / no-focus-signal RPs** (`FocusSlotIndex == -1`): conservatively short-circuit Orbiting / SubOrbital to false (the focus signal is missing), preserving v0.9.0 Crashed-only behavior on pre-feature saves.

Corollary: if a sibling that **should** have been destroyed shows up with a non-crashed terminal (e.g. a booster left behind on reentry but misclassified `Orbiting`), the bug is in the terminal-state classifier, not the UF predicate. The scene-exit finalizer (`IncompleteBallisticSceneExitFinalizer` / `BallisticExtrapolator`) is the usual suspect; its `[Extrapolator]` log tag traces the decisions. A `Start: body=… alt=-<large>` preceded by `PatchedConicSnapshot: solver unavailable` (NullSolver) means KSP has already invalidated the vessel, and the extrapolator now short-circuits to `TerminalState.Destroyed` via `ExtrapolationFailureReason.SubSurfaceStart` rather than silently horizon-capping to Orbiting. Fixing the classifier upstream restores the Crashed Unfinished Flights entry automatically — do not work around it by further broadening the UI predicate.

7.32 Classifier drift across Parsek versions

Old RPs retained; no reclassify. `[Rewind] Verbose` notes the mismatch.

7.33 Rename / hide on Unfinished Flight row

Rename persists through the generic `Recording.VesselName` path. Hide warns via `ScreenMessages` + refuses. **Shipped (test)**: `RenameOnUnfinishedFlightTests`.

7.34 Corrupted quicksave on invocation

RP marked Corrupted via `HandleQuicksaveMissing` at CanInvoke time (file missing) or via `PartLoaderPrecondition.MarkCorrupted` (parse failure / missing parts). Rewind button disabled with the precondition reason. **Shipped (test)**: `RewindTests`.

7.35 Auto-save concurrent with RP save

One-frame defer + root-save-then-move + KSP's serial save pipeline. Auto-save is blocked during the synchronous `GamePersistence.SaveGame` inside the deferred body. **Shipped (integration)**.

7.36 Rewind while unrelated new mission in-flight

CanInvoke gate rejects if a session is already active; if the player is flying a new unrelated mission outside any session, they must first Revert-to-Launch / return-to-Space-Center that mission. **Shipped (integration)**: `RewindTests`.

7.37 Pre-existing vessel reservation conflict during re-fly

`CrewReservationManager.IsLiveReFlyCrew` carve-out exempts the live re-fly crew from reservation lock for the session duration. **Shipped (test)**: `SessionSuppressionWiringTests` + in-game `KerbalDualResidenceCarveOutTest`.

7.38 Strip encounters ghost ProtoVessel

`PostLoadStripper.Strip` step 1 ghost guard via `GhostMapPresence.IsGhostMapVessel`. **Shipped (test)**: `PostLoadStripperTests`.

7.39 Strip encounters unrelated committed real vessel

Step 4 left-alone path. Log `[Rewind] Verbose: Strip leaveAlone: unrelated v=<pid>`. **Shipped (test)**: `PostLoadStripperTests`.

7.40 SessionSuppressedSubtree with mixed-parent descendant

Closure halts at the mixed-parent node. Descendant stays in ERS. **Shipped (test)**: `EffectiveStateTests` (mixed-parent halt case).

7.41 Null-scoped action in supersede scope

Supersede tombstones only attach to actions whose `RecordingId` is in the supersede subtree. Null-scoped actions (deadline `ContractFail`, stand-in generation, etc.) are not subtree-owned and stay in ELS. They are not in any retire set, so they never seal a slot either (section 2.12). **Shipped (test)**: `TombstoneEligibilityTests`.

7.42 KSC action during active re-fly

Player doesn't typically leave flight during re-fly. If a KSC action fires (mod or edge case), it's added to the ledger with null `RecordingId`; not in supersede scope; stays ELS. It also survives a Discard, which removes only rows tagged to the attempt (section 2.12 Rule 3). **Shipped (integration)**.

7.43 Chain extends through multiple merged crashes

Each `CommittedProvisional Crashed` merge extends the supersede chain. `EffectiveRecordingId` walks forward through all relations. Unfinished Flights moves along with the chain's tip. **Shipped (test)**: `ChildSlotEffectiveRecordingIdTests` exercises the forward walk through 0/1/2/3-length chains; the chain-extension flow over multiple merges is covered by the in-game `MergeCrashedReFlyCreatesCPSupersedeTest` matrix.

7.44 Vessel-destruction rep penalty

Original BG-crash had a vessel-destruction rep penalty (not kerbal-death; just "vessel blew up"). Broad reviewed-career tombstones now retire recording-owned `ReputationPenalty` actions in the superseded subtree, including vessel-destruction penalties. Under section 2.12 a vessel-destruction penalty (source `Other`) is a loss row: it does not seal, and it goes through the loss swap. Null-scoped reputation penalties are still not subtree-owned and stay in ELS. **Shipped (test)**: `TombstoneEligibilityTests`.

7.45 Merge interruption recovery

Journaled commit + finisher triggered by journal presence on load, regardless of marker state. Five distinct crash windows covered. **Shipped (test)**: `MergeCrashRecoveryMatrixTests` + `MergeJournalOrchestratorTests` + in-game `MergeInterruptionRecoveryTest` / `JournalFinisherMarkerPresentVariantTest`.

7.46 EffectiveRecordingId walk direction + orphan handling

Walks forward via `RecordingSupersedes`. Cycle detection via visited set (`EffectiveState.cs:88`). One-sided orphan relation (new endpoint missing) is treated as a chain terminator at prior node and logs Warn. Fully orphaned rows are removed by load-time sweep before they can warn forever. **Shipped (test)**: `EffectiveStateTests` + `ChildSlotEffectiveRecordingIdTests`.

7.47 Orphan supersede relation on load

Step 4 of load sweep logs Warn and retains one-sided orphans; fully orphaned rows (`oldResolved=false && newResolved=false`) are removed because neither endpoint can affect effective state. Walk treats retained one-sided orphans as terminators. **Shipped (test)**: `LoadTimeSweepTests`.

7.48 Immutable-merged re-fly — player wants to retry anyway

They cannot. `Immutable` seals the slot. The same holds for a slot sealed by section 2.12's value gate (the original flight or the re-fly earned something): the earning may already have paid for something later, and re-flying it away is the paradox the gate exists to prevent. To retry, the player must Full-Revert, which clears the rewind point + supersede / tombstone state for the tree (the committed recordings stay in the timeline). **Shipped (integration)**: documented behavior + `TreeDiscardPurgeTests` for the full-revert path.

7.49 4-probe deploy from a Mun mothership (the canonical v0.9.1 stable-leaf case)

Mothership is focus → Immutable. 4 probes are non-focus, terminal Orbiting → 4 CP slots → 4 UF rows. RP stays alive while any probe slot is unsealed. **Shipped (in-game test)**: end-to-end coverage in `RuntimeTests` (Mun-mothership-deploy → fly probe #2 → land → merge → slot closure + supersede chain extension).

7.50 Auto-parachute booster, focus on upper stage

Booster non-focus, terminal Landed → not UF (Landed always returns false from `TerminalOutcomeQualifies`) → Immutable. Upper stage is focus, terminal Orbiting → not UF (focus exclusion) → Immutable. RP reaps cleanly. **Critical regression guard test** (largest-impact failure mode of an over-broad predicate).

7.51 Inverted: focus on booster, upper stage left orbiting

Booster is focus, terminal Landed → not UF → Immutable. Upper stage non-focus, terminal Orbiting → UF → CP. RP stays alive while upper-stage slot is unsealed.

7.52 Stranded EVA kerbal (alive)

EVA kerbal terminal Landed (or Splashed for water) on a body, `EvaCrewName` non-null. Not UF (`evaNotSeparation`, 2026-09-27 ruling, §1.5): no EVA Rewind Point is authored, and a legacy one loads best effort without surfacing (section 7.68). The player uses F9 or Discard. **Shipped (test)**: `UnfinishedFlightsMembershipTests.EvaKerbalUnderRp_NeverMember`.

7.53 EVA kerbal reboards

Board BP fires. Not UF: an EVA kerbal never qualifies (`evaNotSeparation`, section 1.5), so the old leaf-gate reasoning (the Board BP is downstream of the matching RP's BranchPoint) is no longer the operative reject. The VESSEL he re-boarded is another matter: when it is the separated vessel of a staging / undock slot, that slot walks through the EVA and the re-board to the vessel's real tip (section 1.6). A Board into a different vessel blocks the slot (`evaCrewJoinedForeignVessel`). **Shipped (test)**: `ReFlyThroughEvaTests`.

7.54 Dead EVA kerbal

Terminal `Destroyed`. Not UF: the EVA gate runs before the Crashed branch (`evaNotSeparation`, section 1.5). A kerbal death on EVA is recovered with F9 or Discard, not a Re-Fly of the kerbal - unless he went EVA from a separated vessel whose slot is open: re-flying that vessel from its separation rewrites the EVA and tombstones the death (section 1.6), and only then does the Kerbals window's Lost hover offer the Re-Fly way back. **Shipped (test)**: `TreeCommitTests.CommitTree_EvaKerbalUnderLegacyEvaRewindPoint_NeverPromoted`.

7.55 Breakup-survivor active parent, terminal Crashed

Survivor V's chain TIP terminal Destroyed. Crashed branch returns true regardless of focus. UF. v0.9.0 behavior preserved exactly.

7.56 Breakup-survivor active parent, terminal Landed

Survivor V's chain TIP terminal Landed. Landed always returns false from `TerminalOutcomeQualifies`. Not UF. RP reaps when other slots close. v0.9.0 behavior.

7.57 Breakup-survivor active parent, terminal Orbiting, focus slot

Survivor V is FocusSlot, terminal Orbiting → not UF (focus exclusion). Player loses access to re-fly the breakup moment via UF by default unless they use Stash (§6.25.1) while the backing RP still exists. The origin-only fallback from §4.8 does not bypass focus exclusion; it only restores the missing RP-slot link when the slot would otherwise qualify.

7.58 Cross-tree dock during stable-leaf re-fly

Re-flown probe docks with another tree's station. Dock BP fires; probe-re-fly's chain TIP gets `ChildBranchPointId = dockBp.Id`. Per-RP leaf gate fails. Site B-1 sees the downstream structural/world interaction → `TerminalOutcomeQualifies` returns false → Immutable, auto-sealed. AppendRelations closure walk (rooted at SupersedeTargetId) is tree-scoped and halts at the mixed-parent BP; station's tree unaffected.

7.59 Re-fly a stashed probe, end in Mun orbit

Re-fly merge under the v0.9.1 auto-seal revision (§4.9): Site B-1 calls the classifier with `focusSlotOverride: slotIdx`. The chain tip is Orbiting and the slot the player chose to fly equals the override, so the classifier returns `classifierReason=stableTerminalFocusSlot`. `ShouldKeepReFlySlotOpenAfterMerge` exits at `!classifierQualifies` with `closeReason=ClassifierClosed/stableTerminalFocusSlot`; the provisional tip commits `MergeState=Immutable` (the slot is sealed). Supersede still appends `{priorTip -> provisional}` so the timeline points at the new recording. The row drops from UF. The player intentionally engaged this slot to fly it; the merge concludes the engagement. If the player wants the stable-orbit retry to remain open, they don't merge — they Discard or Retry the Re-Fly. The variant where the re-fly also earned something (credited science, funds, reputation, a milestone, a contract outcome, a recovery) follows the same Immutable + auto-seal path through the `recordingAction:*` reason (the section 2.12 value gate fires before the focus override branch; it reads the attempt's own retire set, so the replaced flight's rows no longer decide it). KSC-scene player decisions (`ScienceSpending`, `ContractAccept`, `KerbalHire`, `FacilityUpgrade`, ...) carry no flight-recording tag in practice, so they are in no retire set; only loss rows (a kerbal death, its reputation penalty, a destroyed facility) stay non-sealing.

7.60 Re-fly an auto-included stable-leaf probe, end Landed

The chain tip is Landed (a stable conclusion) and the slot the player chose to fly equals `focusSlotOverride`. Under the v0.9.1 focus-override contract, `IsReFlyOverrideStableTerminal` accepts `{Orbiting, SubOrbital, Landed, Splashed}`, so Site B-1's classifier returns `classifierReason=stableTerminalFocusSlot`; the provisional tip commits `MergeState=Immutable` (the slot is sealed). The row drops from UF and the rewind point reaps once all sibling slots are closed. Re-fly merges from non-Re-Fly call sites (`focusSlotOverride=null`) still fall through to the existing `terminal != Orbiting && terminal != SubOrbital` branch and return `stableTerminal` without auto-seal — used by `EffectiveState.IsUnfinishedFlightCandidate`, the natural scene-exit promotion path, and reap-eligibility checks. Retry-blocking `ScienceEarning` recording-linked actions still fire ahead of the override and report `recordingAction:*` + auto-seal. Manual-Stash variant: a stashed slot Re-Fly to Landed routes through the same `stableTerminalFocusSlot` close reason because the override precedes the stashed-keep-open branch by design.

7.61 Re-fly a stashed stranded EVA kerbal (retired 2026-09-27)

No longer reachable: an EVA kerbal is never stashable or re-flyable (§1.5). A Re-Fly session a player started on a legacy EVA Rewind Point before upgrading still concludes: the classifier returns `evaNotSeparation` and Site B-1 closes the slot.

7.62 Player Seals a UF row

The slot's effective tip flips to `Immutable`; row drops from group; reaper runs (RP deleted if all sibling slots are also closed, i.e. every tip is `Immutable`). Recording trajectory unchanged; ghost playback unchanged on subsequent rewinds. **Shipped (test)**: Seal handler + reaper integration tests.

7.63 Player Seals a Crashed row (not a stable leaf)

Same Seal handler. The Crashed terminal originally qualified the row; Sealing closes it as "I accept the crash as canonical." Provides v0.9.0 users with a cleanup affordance they didn't have before.

7.64 Sealed row was never Immutable

A CP slot whose recording is in chain extension (e.g. probeReFly1 with subsequent CP attempts). Player Seals: the slot's effective tip flips `CommittedProvisional -> Immutable`, so the reaper sees it as closed. RP reaps when the other slots also close.

7.65 Sealed slot whose effective recording is NotCommitted

Should not occur in normal operation (NotCommitted recordings are not in ERS, so the row never appears as UF, so the player can't Seal it via the UI). If a load-time race leaves a slot's effective tip at NotCommitted, the reaper's rule (a NotCommitted tip blocks reap, §6.23) prevents the RP from being deleted. **Defense-in-depth test.**

7.66 Pre-feature save load — Orbiting non-focus sibling from before upgrade

Legacy RP loads with `FocusSlotIndex == -1`. `noFocusSignal` short-circuit returns false for Orbiting/SubOrbital. The legacy Immutable Orbiting sibling stays Immutable; row does not appear in UF. **Forward-only migration for vessels.** See §9.4.

7.67 Pre-feature save load — Crashed sibling from before upgrade

Legacy RP loads with `FocusSlotIndex == -1`. Crashed branch returns true regardless of focus. Row appears in UF as it did in v0.9.0. **Regression guard.**

7.68 Save load with a legacy EVA Rewind Point

No longer handled (1.0 save-format ruling, 2026-11-04): the load-time EVA RP reap is deleted. Such an RP loads best effort and never surfaces in UF (`evaNotSeparation`) and is subject only to the ordinary reaper.

7.69 New post-feature RP, no slot was focused at split time

RP captures with explicit `FocusSlotIndex = -1` (player was focused on an unrelated vessel outside the split). Same `noFocusSignal` behavior as legacy RPs: Orbiting/SubOrbital suppressed; Crashed qualifies. Manual Stash allows the player to add Orbiting siblings from these rare RPs while the backing RP still exists.

7.70 BG-only multi-controllable split with all controllable Orbiting siblings

RP captures with `FocusSlotIndex = -1` (no focus involved). All sibling slots stay Immutable post-commit. No UF rows. RP reaps. Same outcome as legacy save case.

7.71 In-place re-fly merge ending Orbiting (Site B-2 behavior)

Player drives a re-fly merge through `MergeDialog.TryCommitReFlySupersede` (in-place path). Under the v0.9.1 revision, Site B-1 sees Orbiting on the chain tip with `focusSlotOverride=slotIdx` matching the merge-time slot → `stableTerminalFocusSlot` → `Immutable` (slot sealed). Site B-2 does not override; it lets the reaper remove the RP when all sibling slots are closed. Retry-blocking recording-linked actions still fire ahead of the focus override branch and auto-seal via `recordingAction:*` (unchanged).

7.71b In-place re-fly merge after a session-authored decouple

Player Re-Flies a stashed slot whose chain tip is stable and stages debris during the session. Under the v0.9.1 focus-override contract, the classifier reaches `stableTerminalFocusSlot` before the stashed keep-open branch, so the chosen slot closes as `Immutable` (slot sealed). The structural-mutation helper still detects the new `Breakup` BP authored at `UT > rp.UT` (and not in `marker.PreSessionBranchPointIds`) as a defensive backstop for safe-stable retry paths that do not close through the override. Pure-ablation part loss without spawning a sibling vessel does NOT trip this gate (no `BranchPoint` is authored). Destroyed in-place merges use the same gate: pure terminal failure, plus loss rows only (kerbal deaths, their reputation penalties, facility destruction), stays `CommittedProvisional` ; any sealing row in the retire set closes the slot (section 2.12 Rule 1).

7.72 Hybrid star+linear supersede graph

Save loaded with legacy star portion `{probeOrig -> probeReFly1, probeOrig -> probeReFly2}` from a pre-§6.22 chain extension. Player invokes a new re-fly into the same slot. New invocation computes `priorTip = slot.EffectiveRecordingId(supersedes)` — the walker picks the oldest member of the star (probeReFly1) and walks linearly from there. New relation appended: `{probeReFly1 -> probeReFly3}` . Resulting graph is hybrid: star portion preserved, linear portion grows from probeReFly1. `TryResolveRewindPointForRecording(probeReFly3, ...)` returns the slot via the forward walk from slot.OriginChildRecordingId → probeReFly1 → probeReFly3. **Tolerated, no migration sweep needed.**

7.73 Crash-quit during a stable-leaf re-fly

Marker validates against on-disk session-provisional + RP. Session resumes. **No new state vs v0.9.0.** `MarkerValidator.Validate` extension weakly validates `SupersedeTargetId` ; failure clears the field and AppendRelations falls back to `OriginChildRecordingId` .

7.74 Player reverts a stable-leaf re-fly mid-flight

`RevertInterceptor` 3-option dialog appears. Discard Re-fly preserves the stashed row in UF (origin RP promoted, slot still CP). **Same as v0.9.0** (the merge-close gate in §6.21 doesn't affect Discard Re-fly because the Discard path doesn't reach the merge classifier).

7.75 Tree discard during chain-extension state

`TreeDiscardPurge.PurgeTree` removes every RP, supersede relation, and tombstone scoped to the discarded tree. The `slot.Stashed` flag on the tree's RP slots is removed alongside the RPs (open/closed lives in the recordings' `MergeState`, which the tree-discard purge also clears). `marker.SupersedeTargetId` clears alongside the marker. **No new code paths needed in TreeDiscardPurge** — the discard already removes the parent objects.

7.76 Recording with no terminal state

`TerminalStateValue` is null (finalization didn't run cleanly). `TerminalOutcomeQualifies` returns false → not UF. Logged as `reason=noTerminal`. The upstream finalization-reliability contract in `parsek-recording-finalization-design.md` is responsible for ensuring this is rare.

7.77 Booster crashed after the upper stage earned everything (section 2.12, the purpose case)

The upper stage reaches orbit and earns the orbit milestone and a contract reward; the booster, background-recorded, crashes with no crew. The booster's retire set is empty (a crash with no crew files no row, or only a vessel-loss reputation penalty, a loss row), so it stays in Unfinished Flights. The player re-flies it to a landing and merges: the vessel-loss penalty, if any, is retired, the slot seals (`stableTerminalFocusSlot`), and recovering the landed booster later adds its value as a new earning. Every balance after the merge is at least what it was before. The upper stage's rows are never in the booster's retire set.

7.78 Crashed booster that earned science before it crashed (section 2.12 Rule 1)

The booster transmitted a crew report on the way down, then crashed. Its retire set holds a `ScienceEarning` (a sealing row), so it is not offered (`recordingAction:ScienceEarning:<id>`, the reason the shipped gate already gave). The same now holds for a milestone, a contract result, lab science, a recovery or any funds or reputation earning filed on the booster after the split.

7.79 Earn, spend at the Space Center, then try to re-fly (the bank-then-retire exploit)

A flight earns a contract reward on a background booster, the booster crashes, the player commits and spends the money on a facility upgrade, then comes back to re-fly the booster. Under the shipped scope the merge retired the reward and left the upgrade unpaid (funds below zero). Under section 2.12 the reward is a sealing row, so the booster was never offered. **Red-first test**: the facility upgrade case on main must show the negative balance before the fix and a sealed slot after it.

7.80 Re-fly kills more crew than the original (section 2.12 Rule 2)

The original crash killed one kerbal; the re-fly crashes too and kills two. Variant A (retire the original's penalty, count the re-fly's two) would lower reputation, so the merge takes variant B: the original's death penalty stays, the re-fly's two penalties are tombstoned with the re-fly as the retiring recording. The roster follows the re-fly (both kerbals dead, the original victim among them). The slot stays open (a crash that filed only losses).

7.81 Re-fly fails a contract the committed timeline still holds active (section 2.12 Rule 5)

Contract X was accepted before the rewind point and is still active in the committed timeline (a later committed flight completes it, or nothing resolves it yet). During the session stock holds X active, the re-fly's vessel is destroyed and stock fails X. The failure's penalty lands earlier than anything in `L`, and a later committed completion of X would pay nothing, so funds or reputation drop at some UT. The dominance check refuses the merge when the merge dialog opens (`[LedgerSwap] Info: Merge dominance check ... result=refuse pool=Funds ...`), before the tree commits; the dialog explains it and offers Discard, which removes the failure row (section 2.12 Rule 3). A contract the committed timeline already completed or failed before the session's UT is terminal in stock during the session and cannot be failed again.

7.82 Player tries to switch away during a re-fly (section 2.12 Rule 3)

`]`, map Switch-To, Tracking Station Fly, a KSC marker Fly or an alarm's Jump button to any vessel other than the re-flown vessel and its own crew on EVA is refused, with a `[ReFlySession] Info: Switch refused` line. Going EVA from the re-flown vessel, boarding it again, and switching between the vessel and that kerbal stay allowed. A probe the re-fly itself decoupled is refused too; it keeps its session rewind point and can be re-flown after the merge.

7.83 Rewind point brings back a vessel the committed future uses (section 2.12 Rule 3)

Vessel V was live at the rewind point (UT 1000), a committed recording flies it from UT 1500 and recovers it later for 50k. The strip hands V to the claimed-vessel hold (section 6.9 step 5): a hold ghost in its rewind-save shape until UT 1500, then the recording's ghost until the recovery. Its recovery rows stay, and the player cannot dock with, board or recover V during the session. Shipped instead: Step 3b tombstoned V's recovery rows at invoke, so a facility upgrade that recovery paid for went unpaid, and the tombstones stayed after a Discard. **Red-first test**: the Step 3b example (20k start, +50k recovery, -65k upgrade, re-fly of another split) reads -45k after a Discard on main.

7.84 Deadline contracts during a session (section 2.12 Rule 3)

A contract with a deadline after the session's UT, and after every committed row, stays active in stock during the session. One whose deadline a committed row walks past expired in the committed timeline: the session walk fails it at the deadline with its penalty, as every other walk of that ledger does. Shipped instead: the invoke walk passes its `double.MaxValue` horizon as the deadline clock, so every deadline contract reads as expired, a synthetic failure is charged in the session's pools, and the contract leaves stock's active list until the merge walk restores it. **Red-first test**: a unit cell on `ContractsModule.PrePass` with a horizon past the deadline and a clock before it.

7.85 Existing save with an open slot that holds value (section 2.12 Migration)

A save made before the change has a crashed booster in Unfinished Flights whose flight earned a milestone. On load the sweep concludes its tip `Immutable` (`[LoadSweep] Value-sealed slots ...`), the row leaves Unfinished Flights, and the reaper can delete its rewind point once every slot is closed. A deficit an earlier merge already wrote stays; nothing is repaired.

---

8. What Doesn't Change

- **Flight-recorder principle 10** (additive-only recording tree). Enforced rigorously: supersede is a relation, not a field; tombstones are append-only; no field on a committed Recording is ever written post-commit; orphan supersede relations are never deleted outside tree-discard (and the resumed-session discard prune of rows naming the deleted attempt, section 6.14, and the quickload that hands a resumed flight's rows back to its save, section 6.16).
- **Single-controllable split path.** Unchanged.
- **Merge dialog UI shell.** Unchanged; the feature adds a supersede-advisory line when an active session is present, and (section 2.12 Rule 5) the reason in the body text when the dominance check refuses the merge.
- **Ghost playback engine.** Reads ERS via `SessionSuppressionState` ; `SessionSuppressedSubtree` is upstream of the engine, not inside it. The v0.9.1 closure-helper split (§6.22) preserves the wrapper contract — runtime ghost suppression continues calling `ComputeSessionSuppressedSubtree(marker)` and gets the same return value as before.
- **Active-vessel ghost suppression.** Natural ERS filter (`NotCommitted` recordings are excluded; the live provisional is `NotCommitted`).
- **Ledger recalculation engine.** Reads ELS via `EffectiveState.ComputeELS` ; no engine change.
- **Rewind-to-launch (the existing T0 quicksave-reload feature).** Same reload path, with Re-Fly-aware edges added since v0.9: a plain Rewind-to-Launch clears any loaded Re-Fly marker before replay resumes, keeps every supersede row, because a rewind moves the clock and unseals or undoes nothing (owner ruling 2026-10-24: a sealed slot stays sealed, an open slot stays open, a merged re-fly stays the tip). A dangling row (its fork missing because the fork's recording failed to load, or still uncommitted) stays too and keeps hiding the flight it replaced, exactly as the load-time sweep keeps a one-sided orphan (section 2.2): the ledger still holds the re-fly's rows and the replaced flight's tombstones (owner ruling O3), so bringing the replaced flight back would show a timeline the career does not match. The rewind writes no rewind retirement; one `Rewind keeps supersede rows:` line counts the rows it kept by fork state. As of v0.10.5 it carries rewind points, supersede rows, rewind retirements, kerbal-death tombstones and the merge journal across the reload from memory rather than from the possibly stale `persistent.sfs` ; a rewind point left in the player's future stays in Unfinished Flights with Fly disabled until the clock reaches its split; and a Rewind-to-Launch taken while a Re-Fly is still open (its vessel destroyed, focus moved on) ends that Re-Fly exactly like leaving it unmerged, so the slot stays re-flyable.
- **Loop / overlap / chain.** Read ERS through the same filter; no per-feature code change.
- **Recording sidecar format.** No changes.
- **Reservation manager internals.** Re-derivation from ERS is existing; the carve-out in `IsLiveReFlyCrew` is a single-method filter.
- **Career state (KSP-sticky subsystems).** Supersede does not rely on stock systems re-emitting events. It retires reviewed recording-scoped ledger rows, then runs a recalc/patch pass over the surviving ELS to reconcile `ContractSystem` , progress tracking, facilities, strategies, tech, science, funds, reputation, and safe roster cleanup.

- **Vessel-destruction reputation.** Recording-scoped reputation penalties in the superseded subtree are tombstone-eligible after review, while null-scoped/system rows remain pass-through.
- **Tombstone eligibility list.** Unchanged by section 2.12. What changed is which slots can be re-flown: only those whose eligible rows are all loss rows.
- **Science rewards.** Recording-scoped `ScienceEarning` rows in the superseded subtree are tombstone-eligible (and, under section 2.12, sealing, so a slot holding one is never superseded); the committed-science cache rebuilds from the full surviving ledger so removed subjects clear without losing recovery metadata for surviving future subjects.
- **v0.9.0 Crashed UF behavior (preserved by v0.9.1 extension).** Every recording that was UF under v0.9.0 is still UF under v0.9.1 (preserved by §7.55 + §7.67). Site A's broader predicate subsumes the v0.9.0 Crashed-only check; the Crashed branch returns true regardless of focus / kerbal status / `FocusSlotIndex` short-circuit.
- **Reaper "every slot closed → reap" rule.** The close definition reads the slot's effective tip `MergeState`: closed iff `Immutable`; both `CommittedProvisional` and `NotCommitted` tips block reap.
- **`marker.OriginChildRecordingId` contract.** Still the slot's immutable origin, even after the v0.9.1 invocation linearization. Existing consumers (`RevertInterceptor.FindSlotForMarker`, in-place continuation, ghost suppression) continue to read it unchanged.
- **CommitTombstones tombstone scope.** Still uses the closure from AppendRelations; the closure now correctly walks the prior-tip's descendants on chain extension (more accurate, not different in shape).
- **Discard Re-fly + Retry from RP semantics.** Both run before the merge classifier; the v0.9.1 Site B-1 / Site B-2 changes don't affect them. Section 2.12 Rule 3 changes one thing in both: the attempt's ledger rows are removed (`Ledger.RemoveActionsForRecordings`) instead of untagged; Retry removes them in `MergeDialog.RepairSessionBeforeRetry`, before the re-invoke captures the reconciliation bundle, and the load-time sweep removes a zombie's rows with it (`LoadTimeSweep.LedgerRowsRemovedByLastRun`; the in-session branch recalculates again when it removed any).
- **TreeDiscardPurge.** New v0.9.1 fields are removed alongside their parent objects; no purge-side changes.
- **ERS / ELS routing rules.** No new raw-read sites; the v0.9.1 classifier reads through `EffectiveState.ComputeERS()` like every other consumer.
- **EVA kerbal-death retry behavior.** Retired 2026-09-27 (§1.5): EVA crash / stranded rows no longer qualify. Tombstone eligibility itself is unchanged: kerbal-death rows in a superseded vessel subtree are still retired alongside the other reviewed recording-scoped career consequences.
- **Legacy saves.** Pre-1.0 saves load best effort (section 9): a ledger row with no `actionId` keeps the fresh id its constructor assigns, and an absent `mergeState` reads as `Immutable`; new ConfigNode sections (`REWIND_POINTS`, `RECORDING_SUPERSEDES`, `LEDGER_TOMBSTONES`, `REFLY_SESSION_MARKER`, `MERGE_JOURNAL`) are optional and absent on pre-v0.9.0 saves. v0.9.1 fields (`ChildSlot.Sealed` / `Stashed`, `RewindPoint.FocusSlotIndex`, `ReFlySessionMarker.SupersedeTargetId` / `PreSessionBranchPointIds`) are absent on pre-v0.9.1 saves; defaults match v0.9.0 behaviour.
- **Standalone ghost playback mod (Gloops).** The feature touches only Parsek-internal code; nothing the engine depends on moves.
- **Existing chain semantics outside a re-fly session.** BranchPoints, chain tips, ghost extension, trajectory walkback, etc. are read-only consumers of the new ERS view; the session-suppressed subtree only takes effect while a marker is live.

---

9. Backward Compatibility

From the 1.0 save format (owner ruling 2026-11-04) the rewind, supersede, tombstone, marker and journal nodes are versioned by the save-wide `parsekSaveFormat` stamp: a shape change bumps it and ships an upgrade step, and saves from before 1.0 load best effort with no repair pass (policy `docs/dev/design-save-format-policy.md`). The subsections below record the pre-1.0 releases' behavior.

9.1 Pre-v0.9.0 saves on v0.9.0+

- Load cleanly. All five new ConfigNode sections are absent → treated as empty lists / null singletons (`REWIND_POINTS`, `RECORDING_SUPERSEDES`, `LEDGER_TOMBSTONES` → `List<T>()`; `REFLY_SESSION_MARKER`, `MERGE_JOURNAL` → `null`).
- `Recording.MergeState` is read from its `mergeState` key; an absent key reads as `Immutable` (any recording reachable from a committed tree was already sealed). The pre-v0.9.0 binary `committed` flag is no longer read.
- `Recording.SupersedeTargetId` — never set on pre-v0.9.0 saves. Non-empty values on `Immutable` / `CommittedProvisional` recordings (should only happen on a corrupt or hand-edited save) log Warn and are treated as cleared (`LoadTimeSweep.ClearStraySupersedeTargets`, `LoadTimeSweep.cs:326`).
- `BranchPoint.RewindPointId` — absent on pre-v0.9.0 saves. No `IsUnfinishedFlight` predicate matches on pre-feature BG-crashed siblings because the predicate requires a non-null parent `RewindPointId`. The player's existing crashed siblings are NOT retroactively made Unfinished Flights.
- Pre-v0.9.0 recordings have no cargo/crew manifest changes; Rewind to Separation does not read or write those fields.

9.2 Post-v0.9.0 saves on v0.8.x

Not supported. A post-v0.9.0 save carries `REWIND_POINTS` / `RECORDING_SUPERSEDES` / `LEDGER_TOMBSTONES` / `REFLY_SESSION_MARKER` / `MERGE_JOURNAL` nodes that pre-v0.9.0 Parsek does not know. Depending on the KSP version's ConfigNode parser tolerance, the save may:

- Load cleanly but silently drop the new sections on next save (losing supersede relations → superseded recordings reappear in ERS).
- Fail to load with a KSP-side parse error.

No downgrade path is provided.

9.3 Feature removal

All new entities are ignorable as orphans. Removing the feature from a save would leave `REWIND_POINTS` etc. unused; the shipped code does not offer a "strip rewind state" toggle.

9.4 Pre-v0.9.1 saves on v0.9.1+

- Load cleanly. The four new ConfigNode keys (`sealed`, `stashed`, `focusSlotIndex`, `supersedeTargetId`) are absent on disk. Default values match v0.9.0 behavior:
  - no slots are pre-sealed; existing CommittedProvisional-tip slots stay open.
  - `slot.Stashed = false` → no slots are pre-Stashed; default predicate continues to govern membership.
  - `RewindPoint.FocusSlotIndex = -1` → the `noFocusSignal` short-circuit suppresses Orbiting/SubOrbital qualification on every legacy RP. **Forward-only migration for vessels.**
  - `marker.SupersedeTargetId = null` → AppendRelations coalesces with `?? OriginChildRecordingId` fallback. **No change in supersede behavior for legacy markers** (they continue to produce star-shaped graphs from the prior re-fly invocation).
  - `marker.PreSessionBranchPointIds = null` → structural-mutation gate (§4.9) skips conservatively on those markers, preserving pre-fix keep-open behavior on merge for in-flight sessions at upgrade time.

9.5 Stranded EVA kerbals: retroactive carve-out (retired 2026-09-27)

v0.9.1 surfaced stranded EVA kerbals from legacy saves retroactively. The 2026-09-27 ruling (§1.5) removed EVA from Re-Fly entirely: no EVA kerbal surfaces.

9.6 v0.9.1 CHANGELOG split

The v0.9.1 migration + behavior-change story splits into three notes that must be presented separately to the player:

> **Vessels: forward-only.** Past missions where you deployed probes or stages and left them parked are not retroactively converted to Unfinished Flights. The feature only surfaces splits made after the upgrade. (Why: legacy data has no focus signal, and we'd rather under-include than flood your list with every routine upper stage from your career.)

> **Stranded EVA kerbals.** (Superseded 2026-09-27.) The v0.9.1 note announced stranded EVA kerbals in Unfinished Flights. EVA kerbals no longer appear there and cannot be re-flown; use F9 or Discard.

> **Re-Flying a stashed entry to a stable conclusion seals the slot.** When you Fly a UF entry and the merge commits the slot's chain tip at a stable terminal (`Orbiting`, `SubOrbital`, `Landed`, `Splashed`, plus the existing `Recovered` / `Docked` / `Boarded` set), Parsek seals that retry slot — the recording is `Immutable`, the row drops from Unfinished Flights, and the rewind point's quicksave is reaped on the next sweep. The same applies if the Re-Fly authors any structural shape change during the session (decouple, stage, undock, joint break, kerbal EVA — anything that creates a new sibling vessel). The intent: when you actively engage a slot to fly it yourself, the merge concludes that engagement. Destroyed vessels still stay retryable for another attempt unless they also credited science via Crew Report / EVA Report / Surface Sample / Transmit / Recover during the recording. Automatic milestones / contract complete-fail / funds-rep rewards, KSC-scene player decisions you happened to make while the recording was live (tech unlock, contract accept/cancel, hire, facility upgrade/repair, strategy toggle, vessel build cost), and tombstoneable kerbal death stay retryable. Routine non-structural part actions like deploying solar panels, ladders, gear, cargo bays, or braking a rover do NOT count as structural mutation. If you want to keep a stable Re-Fly retryable, don't merge — Discard the Re-Fly and revisit the slot later.

9.7 Post-v0.9.1 saves on v0.9.0

Not supported. The four new ConfigNode keys (`sealed`, `stashed`, `focusSlotIndex`, `supersedeTargetId`, plus `preSessionBranchPointIdsPresent` / `preSessionBranchPointId`) would be silently dropped on next save. The player would lose Sealed / Stashed state and the FocusSlotIndex; UF predicate would degrade to the v0.9.0 Crashed-only behavior on the now-modified save. **No downgrade path provided.**

9.8 Hybrid supersede graphs

Per §6.22 migration concern + §7.72: legacy star-shaped supersede portions in pre-v0.9.1 chains stay as-is. New post-feature chains extend linearly from whichever leaf the walker picks in the star portion. No migration sweep.

9.9 Saves with open slots that hold value (section 2.12, designed 2026-10-19)

Forward-only, like section 2.10. A slot that is open in an existing save but whose retire set holds a sealing row is concluded on the first load (`LoadTimeSweep`, `[LoadSweep] Value-sealed slots ...`), so it leaves Unfinished Flights and its rewind point becomes reap-eligible once every slot of it is closed. Tombstones earlier merges wrote, and any deficit they left, stay as they are: repairing them would rewrite past events again. A `MERGE_JOURNAL` written before the change has no `lossSwapVariant` and resumes as variant A, which is what the old code would have done.

---

10. Known Limitations / Future Work

10.1 ERS-exempt consumers (index-keyed state)

The ERS-audit allowlist (`scripts/ers-els-audit-allowlist.txt`) now carries many more files than the original thirteen; the map/TS render rewrite and ledger-trace subsystems added exemptions, each with an inline `[ERS-exempt]` comment and a `TODO(phase 6+)` marker. The twelve enumerated below (`ChainSegmentManager.cs` was the thirteenth until its removal on 2026-09-27) are the index-keyed subset holding raw state keyed by positions in `RecordingStore.CommittedRecordings`:

- `Source/Parsek/GhostMapPresence.cs`
- `Source/Parsek/WatchModeController.cs`
- `Source/Parsek/ParsekFlight.cs`
- `Source/Parsek/ParsekPlaybackPolicy.cs`
- `Source/Parsek/ParsekTrackingStation.cs`
- `Source/Parsek/Diagnostics/DiagnosticsComputation.cs`
- `Source/Parsek/FlightRecorder.cs`
- `Source/Parsek/ParsekKSC.cs`
- `Source/Parsek/ParsekUI.cs`
- `Source/Parsek/UI/RecordingsTableUI.cs`
- `Source/Parsek/UI/GroupPickerUI.cs`
- `Source/Parsek/UI/SettingsWindowUI.cs`

The follow-up work is a recording-id-keyed refactor so the remaining grep-audit exemptions can be lifted. Each exemption has a call-site comment explaining why the raw index read is currently required. The refactor is structurally large (touches every subsystem that keys on committed-recordings index) and deferred beyond v0.9.

10.2 Background-split RP capture relies on cached PID payload

For BG-recorded splits that originate outside the focus vessel, the RP author consumes a cached PID payload produced by `BackgroundRecorder`. A joint-break during high warp on a truly unloaded vessel may not emit the event, in which case no RP is captured and the BG-crashed sibling cannot be rewound. Covered by `BackgroundSplitRpTests` for the supported paths; edge cases with on-rails breakups remain deferred work.

10.3 Broad tombstone runtime smoke coverage

The broad reviewed-career tombstone scope has shipped for contracts, milestones, facilities, strategies, science, funds/reputation, and kerbal consequences. Runtime smoke coverage inside KSP for the stock systems xUnit can only exercise through wrappers is now partly shipped: `ContractTombstonesAcrossSupersedeTest` (Mission Control contract state in a live session),

`KerbalRecoveryOnSupersedeTest` (Astronaut Complex roster cleanup), and the non-circular `LedgerGroundTruthHarness` (recalc-vs-on-disk-save diff). Remaining follow-up narrows to R&D tech availability and KSC facility levels.

10.4 Cross-tree supersedes

`EffectiveState.EffectiveRecordingId` has an inline TODO (`TODO(phase >=3)` in `EffectiveState.cs`) to halt the walk at cross-tree boundaries once cross-tree supersedes become producible. v1 does not produce them; the invariant is enforced by the merge logic (supersede relations are scoped to the session's tree).

10.5 End-to-end automated in-game RunInvoke test

Stubbed. Scene reload is not drivable under xUnit, and the KSP in-game test runner does not yet support the full Restore → Strip → Activate → AtomicMarkerWrite pipeline as a single test. The in-game `InvokeRPStripAndActivateTest` covers the strip + activate portion; the pre-load copy-to-root / scene reload is exercised only via manual playtest.

10.6 Auto-purge policies for reap-eligible RPs

There are none in v1. The disk-usage diagnostic surfaces the current total. (The player could once clear them with Wipe All Recordings; that button was removed 2026-09-26, since recordings are never player-deletable.) A scheduled purge for old reap-eligible RPs (TTL-based) is future work.

10.7 Recovery-snapshot feature

The pre-impl doc's `SplitTimeSnapshot` concept (`_split.craft` sidecar, redundant to the quicksave) was dropped before v1. A concrete recovery mechanism for corrupt quicksaves is deferred; v1's Corrupted flag keeps the RP visible for diagnostic purposes only.

10.8 Player-selectable merge mode for crashes

v1 auto-picks `CommittedProvisional` for crash outcomes per `TerminalKindClassifier`. v2 could offer "commit as final (no further rewind)" for players who want to seal the slot and accept the crash.

10.9 SpawnGhost_PrimesFreshGhostToCurrentPlaybackUT

A pre-existing Unity-gated test skip unrelated to Rewind to Separation, but exercised heavily by the feature's in-game tests. Carried forward as an open follow-up from earlier phases.

10.10 v0.9.1 invocation linearization (shipped)

The §6.22 closure-helper split + marker-write change shipped as a v0.9 prerequisite PR before the v0.9.1 stable-leaf extension. Its requirements:

- Adds `marker.SupersedeTargetId` field with weak `MarkerValidator.Validate` extension.
- Modifies `RewindInvoker.AtomicMarkerWrite` to compute `priorTip` and stamp both marker fields + the provisional (guarded for in-place).
- Splits `EffectiveState.ComputeSessionSuppressedSubtree(marker)` into the public wrapper + new `ComputeSubtreeClosureInternal(marker, rootOverride)`.
- Modifies `SupersedeCommit.AppendRelations` to call the internal helper with `marker.SupersedeTargetId ?? marker.OriginChildRecordingId`.

Listed in the limitations section as historical context; the prerequisite is not a future-work item.

10.11 Auto-purge of long-lived sealed RPs (deferred)

No TTL-based reaper extension for concluded RPs. Disk usage is the player's responsibility via the Seal button, surfaced through the Settings → Diagnostics disk-usage line and its crashed-open / stable-open / concluded RP breakdown (§6.18). A later release could add a TTL-based reaper extension or a "Wipe All Sealed RPs" button, but the v0.9.1 design does not invent destructive cleanup policy.

10.12 SealedBy / SealedReason persisted metadata (deferred schema follow-up)

There is no per-slot seal metadata to persist: open/closed is the recording's `MergeState`. The auto-seal reason from the merge gate is logged at merge time (`mergeState=<old>-><new>` / `reason=<closeReason>`) but not written to the recording. Persisted `SealedBy` (`player` vs `mergeAutoSeal`) and `SealedReason` metadata remains a deferred follow-up (now keyed on the recording / supersede relation rather than a slot bit); the original todo entry is marked obsolete in `docs/dev/todo-and-known-bugs.md`.

10.13 Vessels with a committed future stayed live after a plain rewind (fixed)

Section 2.12 makes a Re-Fly invoke hold back every vessel a committed recording uses after the rewind UT (section 6.9 step 5), through the claimed-vessel hold that already serves chain claims. Before the fix, a plain Rewind-to-Launch left such a vessel live when its committed future was a root-lineage or switch-continuation flight with no claiming branch point, so it could be bumped, docked with or destroyed before the flight its recording replays. A physical paradox, not a ledger one in itself; it was filed as todo `REWIND-LEAVES-VESSELS-WITH-A-COMMITTED-FUTURE-LIVE`, with the Re-Fly rule as the fix direction.

Built (2026-10-22). The plain rewind takes the Re-Fly rule, in every scene. The rewind lands in the rewind save's own world (it writes the pre-processed rewind game to persistent.sfs, which the Space Center start reloads, with the live Parsek node kept), and its OnLoad holds back every vessel of that world an active ghost chain claims or a committed recording of its launch flies, recovers, docks with or boards after the rewind UT (`RewindVesselHold`, `ParsekScenario.ApplyRewindVesselHolds`, the same `ReFlyInvokeHold` evidence and positive launch identity), counting only a use the rewind will replay (a historical recording would never bring the vessel back, so the vessel stays live). Once the pools are patched the rewind saves persistent.sfs again, so a crash never cold-loads the rewind world against the latest pool reference. The held vessel leaves the world before any scene loads it (crew Available), a flying recording's end spawn brings it back, and a recovery keeps it gone. The holds are saved with the rewind save's copy of each vessel, so every flight load draws its hold ghost until the first claim; the Space Center and the Tracking Station show nothing for it until the committed flight's own ghost appears (owner ruling 2026-10-22). **Discard Re-fly (built 2026-10-24).** A Discard Re-fly also lands in an earlier world, the rewind point's, which brings back every vessel live at the split: the split's own vessels and every vessel the Re-Fly start held. Its OnLoad (the in-session branch, after the spawn-state restore and before the ledger recalculation, `ParsekScenario.ApplyDiscardReFlyVesselHolds`) runs the same hold pass at the loaded save's UT (never the Planetarium clock, which still reads the pre-load time inside a Space Center OnLoad; an unusable clock skips the hold with a Warn), then reconciles the spawn state and frees the held vessels' crew as the plain rewind does. The game rule is the same: after a discard, a vessel the committed future flies, docks with, boards or recovers after the split is not a live vessel until that flight brings it back. One crash window is accepted: persistent.sfs still holds the vessels until the next save (a scene change, a quicksave), because a save from inside OnLoad is forbidden.

A vessel from another mission is held, never removed (built 2026-10-24 and 2026-10-25). The hold covers a vessel the committed future reaches from outside its own mission too: a vessel a later mission's kerbal moves cargo into or out of, boards through a hatch or takes a seat on, and a vessel one mission spawned that a later mission adopted that way. Such a vessel existed before the later mission, so a Rewind-to-Launch keeps it in the rewind save and holds it with a hold ghost until the later flight reaches it, instead of stripping it as a vessel the replay re-produces (todos REWIND-HOLD-MISSES-CROSS-TREE-ADOPTED-VESSEL and FOREIGN-BOARD-TARGET-STRIPPED-AT-REWIND; lanes EVA-13, EVA-14, SEAT-3). A vessel the replay genuinely produces again is still stripped.

---

11. Risks

The shipped feature carries a handful of known risks. Each is mitigated in code or playtest practice; this section is the inventory for code reviewers and reviewers of follow-up changes.

- **Fewer re-flyable flights (section 2.12).** The value gate removes every crashed or stranded flight that earned anything after its separation from Unfinished Flights, including ones the player could re-fly today. That is the accepted cost of never taking credit back. The motivating case (a background booster that crashed and earned nothing) is unaffected; the regression lane for it must stay green. A player who wants a value-holding flight back can only Full-Revert the tree, as for any sealed slot (section 7.48).

- **Loss list drift.** The loss class is a closed allowlist; a new loss-shaped row type that is not added to it seals slots it should not. That fails safe (fewer re-flies, never a negative balance). The reverse mistake, a gain added to the loss list, is the dangerous one: the dominance check (section 2.12 Rule 5) is the backstop and must stay wired before `Begin`.

- **Disk usage growth.** Stable-leaf RPs persist while they are offered in Unfinished Flights (Site A keeps them open). Under the v0.9.1 auto-seal revision (§4.9), a *successful* Re-Fly merge now seals the slot and the RP becomes reap-eligible — so the disk-usage growth from this feature is bounded by un-Re-Flown rows and crash/death retries, not by repeated re-flies of the same slot. Crash/death retry RPs still persist until the player succeeds, hits a safety gate, or explicitly Seals them. Mitigation: the

Settings → Diagnostics RP disk-usage line includes live crashed-open, stable-open, and concluded RP buckets next to the byte/file total (§6.18).

- **Over-inclusion of the stable-leaf predicate.** The simple terminal-state classifier surfaces some cases where the player meaningfully flew or didn't intend to leave a vessel and it ended Orbiting/SubOrbital (Mun-transfer-and-park, briefly-nudged-probe, deep-space probe carried by transfer stage and undocked there). The Seal button is the design's answer. **Do not re-introduce voluntary-action heuristics** — that path was explicitly rejected in the R1-R3 research-note exploration.
- **Predicate-vs-merge policy confusion.** Site A's UF-display predicate and Site B's post-merge policy share the classifier reason but compare against different focus signals. Site A uses the static `rp.FocusSlotIndex`, so non-focus stable leaves stay open. Site B-1 passes `focusSlotOverride: slotIdx` so the merge-time slot becomes the de-facto focus and a successful Re-Fly closes it. Tests must pin both the Site A classifier reason (`stableLeafUnconcluded` for non-Re-Fly callers) and the Site B merge result (`stableTerminalFocusSlot` + `Immutable` + auto-seal) so this contract does not regress into accidental closure or paradox-prone re-fly exposure.
- **Irreversible RP reap on stable Re-Fly.** Site B-1 auto-seal makes the slot reap-eligible immediately at merge, so the post-merge sweep deletes the rewind-point quicksave file. A successful Re-Fly to stable orbit, or one that authored a structural BP, cannot be re-flown again afterward — the on-disk file is gone. This is intended behaviour ("player is done with that line of flight") but it removes the synchronization pass that an earlier draft of the design's keep-open contract was meant to enable. Players who want that retry path must Discard / Retry the Re-Fly instead of merging.
- **Structural-mutation false-positive surface.** Detection scans tree branch points with `UT > rp.UT` and excludes `marker.PreSessionBranchPointIds` (the snapshot taken at marker creation). The session-local baseline closes the obvious false-positive vector — `SpliceMissingCommittedRecordingsIntoLoadedTree` re-grafting pre-Re-Fly post-RP BPs back into the loaded tree. The remaining surface is BPs authored after marker creation but before the player actually engaged the slot (e.g. an RP captured during a F5 quicksave that snapshotted a tree mid-stage). The marker is created synchronously with the RP load, so this is a narrow window; tests must pin the cutoff and baseline together to prevent regression.
- **Reversal of v0.9.0 §7.31 stance.** v0.9.0 said "stable-end splits explicitly not in scope." v0.9.1 says some non-focus stable-end splits ARE in scope (Orbiting/SubOrbital non-focus; EVA-stranded was in scope until the 2026-09-27 ruling, §1.5). Focus-continuation stable terminals continue to NOT get a row, preserving the "your mission's upper stage didn't suddenly become unfinished" intuition. The CHANGELOG must be precise about what changed and what didn't (see §9.6).
- **Focus-slot breakup-survivor with stable-orbit terminal still isn't auto-added to UF** (§7.57). Origin-only RP-slot resolution fixes tree-branching parent rows that are otherwise eligible, including crashed parents and non-focus Orbiting/SubOrbital parents. It deliberately does not bypass focus exclusion: a focus-slot survivor that reaches stable orbit is considered the player's continued mission unless they explicitly use Stash while the backing RP still exists.
- **Site B-2 in-place duplicate-row invariant.** The in-place path has no separate provisional, so the recording remains its own effective slot target. Site B-1 owns the close/open decision under the v0.9.1 revision; Site B-2 lets the reaper clean up after stable Re-Fly outcomes (now `Immutable` + auto-sealed) and preserves CP only for terminal-failure outcomes that still allow retry.
- **Legacy hybrid supersede graphs.** §7.72 — tolerated by the design but the hybrid-graph regression test must run green for confidence. If the walker behavior on hybrids differs from expectations, a migration sweep may need to be added back.
- **Optimizer chain length (RESOLVED).** Earlier drafts flagged eccentric-orbit BG-recorded vessels with periapsis-grazing atmosphere as a possible source of unbounded chains. Resolved by the `docs/dev/research/optimizer-meaningful-split-rule.md` investigation: `BackgroundOnRailsState` omits `currentTrackSection` / `trackSections` / `environmentHysteresis` entirely (`BackgroundRecorder.cs:157`), and `OnBackgroundPhysicsFrame` early-returns on `bgVessel.packed`. On-rails BG vessels can't generate optimizer-splittable Atmospheric↔ExoBallistic toggles. Guarded by `EccentricOrbitOptimizerInvariantTests`. No action needed for this feature.

---

12. Diagnostic Logging

Every decision point in §6 emits a log line. The tag catalog below enumerates what appears in `KSP.log` for a Rewind-to-Separation session. **All tags are verified against the shipped source** in the files named in §1.1. The pre-impl spec's candidate tags `Reap` , `Strip` , and `Tombstone` did NOT ship as distinct tag names — their log lines fold into `[Rewind]` and `[LedgerSwap]` respectively. `RewindSave` , `ERS` , and `ELS` DID ship as distinct tags and are cataloged below.

12.1 `[Rewind]`

Broad tag for RP lifecycle and strip outcomes. Used by `RewindPointAuthor`, `RewindInvoker`, `PostLoadStripper`, `RewindPointReaper`, `TreeDiscardPurge`, `LoadTimeSweep`, `RewindPointDiskUsage`.

- RP creation: `Info: RewindPoint begin: rp=<id> bp=<bpId> slots=<N> controllablePids=<M> ut=<x>`.
- Slot capture summary: `Info: RewindPoint slot capture: rp=<id> populated=<p>/<total> disabled=<d>`.
- Partial capture warning: `Warn: Slot <i> disabled: no live vessel for rec=<rid> (rp=<id>)`.
- All-slot failure: `Warn: All slots disabled; RP unusable (rp=<id>). Keeping for diagnostic visibility.`
- Rollback: `Info: RewindPoint rolled back: rp=<id> removals=<n>`.
- Strip: `Info: Strip stripped=[...] selected=<idx> ghostsGuarded=<N> leftAlone=<M> fallbackMatches=<f>`.
- Strip fallback match: `Warn: Fallback match via root-part v=<pid> rootPart=<rootPid> slotIdx=<idx>`.
- Strip leave-alone: `Verbose: Strip leaveAlone: unrelated v=<pid> name=<n>`.
- Strip ghost guard: `Verbose: Strip guard: ghost-ProtoVessel v=<pid> name=<n>`.
- (Section 2.12, designed 2026-10-19) Strip of a vessel the committed future uses: `Info: Strip heldForCommittedFuture v=<pid> rec=<id> firstUT=<ut>`. Replaces the Step 3b retirement's lines.
- Reaper: `Info: ReapOrphanedRPs: reaped=<R> remaining=<rem> fileDeleteOk=<n> fileDeleteFail=<n> bpBackrefCleared=<n>`. `Verbose: ReapOrphanedRPs: reaped=0 remaining=<n>` when nothing to do.
- Tree discard: `Info: TreeDiscardPurge: tree=<id> removedRps=<r> removedSupersedes=<s> removedTombstones=<t>`.
- Load summary: `Info: [LoadSweep] Marker valid=<b>; spare=<n> discarded=<r> ...` (see `[LoadSweep]`).
- Disk usage Warn (directory walk failure): `Warn: RewindPointDiskUsage directory walk failed: <reason>`.

12.2 `[RewindSave]`

Narrower tag for the RP save pipeline. Used by `RewindPointAuthor`.

- Scene abort: `Warn: Aborted: scene=<loaded>`.
- Deferred-coroutine abort: `Warn: Aborted mid-coroutine: scene=<loaded>`.
- Warp drop: `Info: Warp dropped from <prior> to 0 for rp=<id>`.
- Warp restore: `Info: Warp restored to <rate> for rp=<id>` / `Warn: Warp restore to <rate> failed for rp=<id>: <reason>`.
- Save success: `Info: Wrote rp=<id> path=<relPath> bytes=<N> ms=<t>`.
- Save failure: `Error: Failed rp=<id> reason=<…>`.

12.3 `[RewindUI]`

UI-layer decisions. Used by `RewindInvoker.ShowDialog` and `ReFlyRevertDialog`.

- Invoke button clicked: `Info: Invoked rec=<rid> rp=<rpId> slot=<idx> listIndex=<i>`.
- Dialog cancelled: `Info: Cancelled rp=<rpId> slot=<idx> listIndex=<i>`.
- ReFlyRevertDialog input lock: `Verbose: ReFlyRevertDialog input lock set (<lockId>)` / `... cleared`.
- Null callbacks on dialog buttons: `Warn: ReFlyRevertDialog: <Button> button had null callback sess=<sid>`.
- Callback throws: `Error: ReFlyRevertDialog <Button> callback threw: <Type>: <message>`.

12.4 `[ReFlySession]`

Session lifecycle. Used by `RewindInvoker`, `SessionSuppressionState`, `MergeJournalOrchestrator`, `LoadTimeSweep`, `RevertInterceptor`, `ReFlyRevertDialog`, `TreeDiscardPurge`.

- Session start: `Info: Started sess=<sid> rp=<id> slot=<idx> provisional=<rid> origin=<oid> tree=<tid>`.
- Session-suppressed subtree Start/End: `Verbose: SuppressedSubtree entered sess=<sid> ids=[...]` / `... exited`.
- Retry: `Info: Retry sess=<old> → <new>` (issued by `RevertInterceptor.RetryHandler`).
- End (merged): `Info: End reason=merged sess=<sid> provisional=<rid>`.
- End (full revert): `Info: End reason=fullRevert sess=<sid>`.
- Revert dialog shown: `Info: Revert dialog shown sess=<sid>`.
- Revert dialog cancelled: `Info: Revert dialog cancelled sess=<sid>`.
- Marker valid on load: `Info: Marker valid sess=<sid> tree=<tid> active=<rid> origin=<oid> rp=<rpid>`.
- Marker invalid on load: `Warn: Marker invalid field=<f>; clearing` (v0.9.1: includes `field=SupersedeTargetId` for the new weakly-validated field).

- Nested cleanup: `Info: Nested session-prov cleanup sess=<sid> recordings=<n> rps=<m>`.
- (Section 2.12) Switch refused: `Info: Switch refused sess=<sid> path=<vesselCycle|mapSwitchTo|tsFly|kscMarkerFly> target=<pid> reason=<notSessionVessel|targetUnknown|sessionVesselUnresolved>` (`ReFlySwitchGate`; a vessel the session split off reads `notSessionVessel`, and `sessionVesselUnresolved` means the session tree or its provisional could not be found, which refuses fail-closed). An allowed switch logs a Verbose `Switch allowed` line.

12.5 `[Supersede]`

Supersede relation writes + ERS-effective observations. Used by `SupersedeCommit`, `EffectiveState`, `LoadTimeSweep`, `TreeDiscardPurge`.

- Relation append: `Info: rel=<relId> old=<oid> new=<nid>`.
- Subtree count summary: `Info: Added <N> supersede relations for subtree rooted at <originId> (subtreeCount=<sc> skippedExisting=<se>)`.
- Flip MergeState: `Info: provisional=<rid> mergeState=<state> terminalKind=<kind> priorTarget=<oid>`.
- Advisory: `Info: Supersede tombstone effects: tombstoned <N> recording-scoped career actions; contracts/milestones/facilities/strategies/science-spending/science/funds/reputation/kerbals from the old subtree are removed from ELS.`
- Cycle detection: `Warn: EffectiveRecordingId: cycle detected in supersede chain starting from <id> ...`.
- Orphan relation on load: `Warn: Orphan supersede relation=<relId> oldResolved=<b> newResolved=<b>`.
- Fully orphaned relation on load: `Info: Fully orphaned relation=<relId> old=<oid> new=<nid>; removing`.
- Tree-discard purge count: `Info: PurgeTree: removed <N> supersedes with endpoint in tree=<id>`.
- (v0.9.1) Site B-1 verdict: `Info: provisional=<rid> mergeState=<state> qualifies=<b> slot=<slotIdx> rp=<rpId> focusSlot=<rp.FocusSlotIndex> focusSlotOverride=<slotIdx> classifierReason=<reason> autoSeal=<b> autoSealReason=<reason>` — the merge-time classifier verdict including separate `focusSlot` and `focusSlotOverride` fields so the source of the focus decision is greppable.
- (v0.9.1) Auto-seal observation: `Info: Auto-sealed re-fly slot=<slotIdx> rec=<rid> rp=<rpId> terminal=<state> reason=<reason>` (where `reason` is one of `stableTerminalFocusSlot`, `recordingAction:ScienceEarning:<actionId>`, `structuralMutation:<bpType>`, `downstreamBp`, `IsHardSafetyTerminal`).
- (v0.9.1) Site B-1 slot lookup failure: `Error: Site B-1 slot lookup failed for provisional=<rid> rpId=<marker.RewindPointId> originChildRec=<marker.OriginChildRecordingId> supersedeTargetId=<marker.SupersedeTargetId>` — fires when `TryResolveRewindPointForRecording` fails after `AppendRelations`; indicates a §6.22 prerequisite regression or `AppendRelations` bug.

12.6 `[LedgerSwap]`

Tombstone writes + reservation-recompute diagnostics. Used by `SupersedeCommit`, `LoadTimeSweep`, `TreeDiscardPurge`.

- Tombstone batch: `Info: Tombstoned <N> actions (KerbalDeath=<d> repBundled=<r>); <M> actions matched subtree but type-ineligible (breakdown: contract=<c> milestone=<m> facility=<f> strategy=<s> tech=<t> science=<sc> funds=<fn> rep-unbundled=<ru>)`.
- Skip existing: `Verbose: skip existing tombstone action=<aid>`.
- Reservation recompute: `Info: CrewReservation re-derived; kerbals returned active: [names]`.
- Orphan tombstone on load: `Warn: Orphan tombstone=<tid> actionId=<aid>`.
- (Section 2.12, designed 2026-10-19) Dominance check before `Begin`: `Info: Merge dominance check sess=<sid> result=<pass|refuse> pool=<Funds|Science|Reputation|-> ut=<u> before=<b> after=<a>`; on a world-object refusal `world=<kerbal|facility>:<id>` replaces the pool fields; on a retire set holding a sealing row `reason=recordingAction:<Type>:<ActionId>`.
- (Section 2.12) Loss swap: `Info: Loss swap sess=<sid> variant=<A|B> repLossOld=<x> repLossNew=<y>`. Both lines are written at dialog open and again at the Merge click, followed by `context=<dialog-open|merge-click> attemptRows=<n> retireSet=<n> closure=<n>` (and `spend=<Type>:<ActionId>` on an affordability refusal).
- (Section 2.12) Variant B at the Tombstone step: `Info: Loss swap variant B applied sess=<sid> keptReplacedRepLoss=<n> tombstonedAttemptRepLoss=<n> retiring=<rid>`.

12.7 `[MergeJournal]`

Staged-commit phase transitions + finisher outcomes. Used by `MergeJournalOrchestrator`, `TreeDiscardPurge`.

- Phase start (Begin): `Info: sess=<sid> phase=Begin lossSwapVariant=<A|B>`.

- Phase advance: `Verbose: sess=<sid> phase=<phase>`.
- Fault injection (tests only): `Warn: Fault injected at phase=<Phase>`.
- Finisher: `Info: Finisher sess=<sid> markerWas=<present|absent> phase=<phase>`.
- Unknown phase: `Warn: Finisher: unknown phase=<p> sess=<sid> - driving forward from TreeMerge (every step is idempotent)`.
- Completion outcome: `Info: Completed from phase=<p> sess=<sid> stepsDriven=<n>`.
- Final cleared: `Info: Completed sess=<sid>`. `Verbose: sess=<sid> cleared`.
- TagRpsForReap: `Info: TagRpsForReap: promoted <n> session-provisional RP(s) for sess=<sid>`.

12.8 `[LoadSweep]`

Single summary line per load after `LoadTimeSweep.Run` completes.

- `Info: [LoadSweep] Marker valid=<b>; spare=<n> discarded=<r> orphanSupersedes=<os> orphanTombstones=<ot> strayFields=<s> discardedRps=<rps>`.
- Pre-summary Verbose: `Verbose: LoadTimeSweep.Run: no ParsekScenario instance — skipping` when no scenario live.
- (Section 2.12, designed 2026-10-19) Migration: `Info: [LoadSweep] Value-sealed slots (section 2.12): found=<n> sealed=<m>` with one Verbose line per slot naming the sealing row (`recordingAction:<Type>:<ActionId>`).

12.9 `[UnfinishedFlights]`

Classifier + UI group decisions. Used by `EffectiveState`, `UnfinishedFlightClassifier` (v0.9.1), `UnfinishedFlightsGroup`, `UnfinishedFlightSealHandler` (v0.9.1).

Predicate verdicts (Verbose; rate-limited per `(rec, reason)` pair):

```
IsUnfinishedFlight=false rec=<rid> reason=mergeState:<state>
IsUnfinishedFlight=false rec=<rid> reason=notControllable headIsDebris=true
IsUnfinishedFlight=false rec=<rid> reason=noParentBp
IsUnfinishedFlight=false rec=<rid> reason=noScenario
IsUnfinishedFlight=false rec=<rid> reason=noMatchingRP bp=<bpId> rpCount=<n>
IsUnfinishedFlight=false rec=<rid> reason=slotSealed slot=<idx>
IsUnfinishedFlight=false rec=<rid> reason=downstreamBp
    chainTipChildBp=<bpId> matchedRpBp=<bpId>
IsUnfinishedFlight=false rec=<rid> reason=stableTerminalFocusSlot
    slot=<idx> focusSlot=<idx> terminal=<state>
IsUnfinishedFlight=false rec=<rid> reason=stableTerminal terminal=<state>
IsUnfinishedFlight=false rec=<rid> reason=noTerminal
IsUnfinishedFlight=false rec=<rid> reason=noFocusSignalOrbiting
    terminal=<state>
IsUnfinishedFlight=true  rec=<rid> reason=crashed terminal=Destroyed
IsUnfinishedFlight=true  rec=<rid> reason=stableLeafUnconcluded
    slot=<idx> terminal=<state>
IsUnfinishedFlight=false rec=<rid> reason=evaNotSeparation terminal=<state> crew=<name>
```

Site A promotion (Info, one-shot per promotion, v0.9.1):

```
[UnfinishedFlights] Info: CommitTree promoted rec=<rid>
    slot=<slotIdx> rp=<rpId>
    reason=<crashed|stableLeafUnconcluded>
```

Seal handler (Info, v0.9.1):

```
[UnfinishedFlights] Info: Seal cancelled rec=<rid>
[UnfinishedFlights] Info: Sealed slot=<slotIdx> rec=<rid>
    bp=<bpId> rp=<rpId> terminal=<state>
    reaperImpact=<willReap|stillBlocked>
[UnfinishedFlights] Error: Seal could not resolve slot for rec=<rid>
```

UI drag-into rejection: Verbose log + `ScreenMessages` toast (`Unfinished Flights is a system group and cannot receive dropped recordings`).

12.10 `[CrewReservations]`

Tree-discard reservation recomputation. Used by `TreeDiscardPurge`.

- Re-derive after tombstone set shrunk: `Info: Reservation re-derived after tree discard tree=<id>`.
- No-op: `Verbose: PurgeTree: no reservations affected tree=<id>`.

12.11 `[RevertInterceptor]`

Harmony prefix decisions. Used by `RevertInterceptor.Prefix`.

- Pass-through (no session): `Verbose: Prefix: no active re-fly session — allowing stock RevertToLaunch`.
- Block: `Info: Prefix: blocking stock RevertToLaunch sess=<sid> — showing re-fly dialog`.
- Error paths: `Error: ReflyRevertInterceptor internal failure: <reason>`.

12.12 `[ReconciliationBundle]`

Invocation Restore / Capture observations. Used by `ReconciliationBundle`.

- Capture: `Info: Captured bundle: recordings=<n> trees=<m> ledgerActions=<l> supersedes=<s> tombstones=<t> rps=<rps> marker=<b> journal=<b>`.
- Restore: `Info: Restored bundle: <same fields>`.

12.13 `[ERS]` / `[ELS]`

Cache rebuilds. Used by `EffectiveState`.

- ERS rebuild: `Verbose: Rebuilt: <n> entries from <raw> committed (skippedNotCommitted=<k> skippedSuperseded=<s> skippedSuppressed=<u> marker=<sessionId|none>)`.
- ELS rebuild: `Verbose: Rebuilt: <n> entries from <raw> (skippedTombstoned=<k>)`.

12.14 `[Recording]`

Stray-field cleanup on load. Used by `LoadTimeSweep`.

- `Warn: Stray SupersedeTargetId on committed rec=<id>; treating as cleared`.

12.15 `[Boundary]`

Multi-controllable classifier decisions. Used by `SegmentBoundaryLogic`.

- Classification outcome: `Info: ClassifyJointBreakResult: <outcome> originalPid=<pid> postBreakCount=<n>`.

---

13. Testing

13.1 Unit tests

The Rewind-scoped test classes in `Source/Parsek.Tests/`:

- `ActionIdMigrationTests` — `ActionId` assignment and round trip.
- `AtomicMarkerWriteTests` — `RewindInvoker.AtomicMarkerWrite` including the `Phase1And2_NoOnSaveBetween` atomicity guard.
- `BackgroundSplitRpTests` — BG-split RP capture with cached PID payload.
- `BranchPointRewindPointIdRoundTripTests` — BranchPoint.RewindPointId serialization.
- `ChildSlotEffectiveRecordingIdTests` — forward walk with cycle guard + orphan terminator.
- `CopyRewindSaveToRootTests` — quicksave copy to save-root for `GamePersistence.LoadGame`.
- `CrewReservationRecomputeTests` — dead kerbal returns active after tombstone; no-op-safe paths.
- `DiskUsageDiagnosticsTests` — directory walk, missing-dir zero fallback, 10s cache TTL.
- `EffectiveStateTests` — ERS/ELS computation, mixed-parent halt, IsVisible, IsUnfinishedFlight, EffectiveRecordingId.
- `GrepAuditTests` — CI gate: verifies the `scripts/grep-audit-ers-els.ps1` allowlist matches the current source.
- `LedgerTombstoneRoundTripTests` — tombstone serialization.

- `ActionIdMigrationTests` — ActionId assignment and round trip.
- `LoadTimeSweepTests` — marker validation, spare/discard sets, orphan logs, stray field, nested cleanup, summary.
- `MergeCrashRecoveryMatrixTests` — per-window state-invariant assertions for the 5-point crash matrix.
- `MergeJournalOrchestratorTests` — happy path + rollback + completion + idempotence + null-guard.
- `MergeJournalRoundTripTests` — MergeJournal ConfigNode round-trip.
- `MultiControllableClassifierTests` — `IsMultiControllableSplit` gate coverage.
- `PostLoadStripperTests` — ghost guard, primary match, fallback match, unrelated left alone, multi-slot conflict.
- `ReFlyRevertDialogTests` — Retry / Discard Re-fly / Cancel handlers + null callback paths; Discard Re-fly covers session-artifact clear, origin-RP promotion, journal-gate + defensive refusal, scene dispatch to `SPACECENTER` (Launch) or `EDITOR` with `EditorDriver.StartupBehaviour=START_CLEAN` + facility (Prelaunch), and survival of the origin RP across `LoadTimeSweep`.
- `ReFlySessionMarkerRoundTripTests` — marker ConfigNode round-trip.
- `ReconciliationBundleTests` — capture + restore over every field.
- `RecordingSupersedeRelationRoundTripTests` — relation ConfigNode round-trip.
- `RenameOnUnfinishedFlightTests` — rename persistence + hide-row advisory.
- `RewindCleanupClearTests` / `RewindContextTests` / `RewindLoggingTests` / `RewindTests` / `RewindTimelineTests` / `RewindTreeLookupTests` / `RewindUtCutoffTests` — overall invocation flow, log capture, tree lookup, UT-boundary behaviors.
- `RewindPointAuthorTests` — scene guard, warp drop, PID map capture, partial failure.
- `RewindPointReaperTests` — eligibility matrix, supersede walk, orphan slot, file-delete failure, idempotence.
- `RewindPointRoundTripTests` — RP ConfigNode round-trip including both PID maps.
- `RewindPrelaunchStripTests` — PRELAUNCH guard interactions.
- `SessionSuppressionWiringTests` — predicate + transition logging + chain-walker skip + map-presence gate + crew carve-out.
- `SpawnStateReconciliationTests` — post-strip state reconciliation.
- `SupersedeCommitTests` / `SupersedeCommitTombstoneTests` — commit + tombstone matrix + subtree + mixed-parent halt + idempotence.
- `TombstoneEligibilityTests` — full type matrix + 1s pairing window.
- `TreeDiscardPurgeTests` — RPs + supersedes + tombstones + reservations + marker/journal + sibling-tree isolation + state-version bumps.
- `UnfinishedFlightsDragRejectTests` / `UnfinishedFlightsGroupCannotHideTests` / `UnfinishedFlightsMembershipTests` — UI group invariants.
- `EarningsReconciliationTests` — earnings-ledger interactions with tombstoned rep bundles.

The v0.9.1 stable-leaf extension adds:

- `UnfinishedFlightClassifierTests` — predicate gate matrix: `IsDebris`, per-state `TerminalOutcomeQualifies` (8 `TerminalState` values × 2 `isFocus` × 2 `noFocusSignal` × 2 EVA-vs-vessel; an EVA kerbal is `evaNotSeparation` in every cell), null-terminal, per-RP leaf gate, slot-closed gate. **Fails if:** any predicate branch verdict regresses.
- `RewindPointRoundTripTests` (ChildSlot section) — `ChildSlot.Stashed` + `StashedRealTime` round-trip through ConfigNode save/load. (The former `ChildSlotSealedRoundTripTests` was deleted with the `Sealed` / `SealedRealTime` fields; open/closed now lives in the recording's `MergeState`, covered by `CollapseSealMergeStateRegressionTests`.)
- `RewindPointRoundTripTests` (legacy-defaults cell) — there is no separate `ChildSlotStashedRoundTripTests`; the legacy half of the same class covers it. `RewindPoint_LegacyMissingStableLeafFields_LoadsDefaults` hand-builds a `POINT` node whose `CHILD_SLOT` carries no `stashed` / `stashedRealTime` keys and asserts the slot loads `Stashed = false`, `StashedRealTime = null`.
- `RewindPointRoundTripTests` — same class again, no dedicated `RewindPointFocusSlotIndexRoundTripTests`: the fully-populated cell saves `FocusSlotIndex = 1` and reads it back, and the hand-built legacy `POINT` node (no `focusSlotIndex` key) loads `-1`.
- `ReFlySessionMarkerRoundTripTests` — the `SupersedeTargetId` claim lives here, not in a `...SupersedeTargetIdRoundTripTests`: the field round-trips on both the all-fields marker and the mandatory-fields-only marker, and a hand-built legacy `REFLY_SESSION_MARKER` node without the key loads null (alongside `SelectedRootPartPersistentId = 0`).
- `ReFlySessionMarkerRoundTripTests` — likewise for `PreSessionBranchPointIds`: a populated list round-trips through repeated values, an EMPTY list survives as empty (the presence sentinel that separates "baseline known, tree had no BPs" from "legacy marker"), and a legacy node without the key loads `null` so the structural-mutation gate conservatively skips.
- `UnfinishedFlightClassifierTests` / `TreeCommitTests` / `UnfinishedFlightsMembershipTests` / `CollapseSealMergeStateRegressionTests` (the only caller of the `RecordingStore.ApplyRewindProvisionalMergeStatesForTesting` seam) — Site A coverage: stable-leaf non-focus controllable → CP; stable-leaf focus controllable → Immutable; stable-leaf debris → Immutable; crashed-leaf focus → CP (active-parent crash regression guard); crashed-leaf non-focus → CP. **Fails if:** the broader predicate misclassifies any case.

- `SupersedeCommitTests` — Site B-1 coverage, under the general class name rather than a `...MergeClassifierTests`: every cell below drives the real `SupersedeCommit.CommitSupersede` and asserts the `[Supersede]` `mergeState=` / `classifierReason=` / `autoSeal=` triple. Re-fly ending Orbiting or Landed on the chosen slot with no sealing row → `stableTerminalFocusSlot` + `Immutable`, whether the slot is the RP's static focus, a non-focus slot promoted by the override, or an RP captured with `FocusSlotIndex = -1`; Splashed → `Immutable` through the plain terminal path; `ScienceEarning` → `Immutable` (slot sealed) through `recordingAction:ScienceEarning:*`, including the origin-only marker-target and prior-supersede-lineage resolutions; Recovered/Docked/Boarded (a theory over all three) → `stableTerminal` + sealed, and a downstream structural interaction → `downstreamBp` + sealed; Destroyed with no ledger rows → CP and not sealed, and the same with non-structural part events or a loss row (a tombstone-eligible kerbal death) stays open, while Destroyed with any sealing row seals (section 2.12 Rule 1: credited `ScienceEarning`, a `ScienceSpending` row, a milestone row, every non-loss type over `TryFindRetryBlockingWorldAction_SealsOnEverySealingRowOfTheRetireSet`). **Not asserted at this site:** the SubOrbital-stays-open branch — the "still in flight, do not seal" verdict is asserted one level down against the shared classifier in `UnfinishedFlightClassifierTests` (both the override and static-focus paths return `stableLeafUnconcluded`), and `SupersedeCommitTests` only reaches SubOrbital through the slot-lookup-failure fallback cell. **Fails if:** chosen stable outcomes (Orbiting/Landed/Splashed) stay open, outcomes holding a sealing row remain re-flyable, or terminal-failure outcomes stop being retryable.
- `SupersedeCommitTests` (in-place cells) + `Bug585InPlaceContinuationRestoreTests` — **the Site B-2 matrix as written is NOT covered.** No in-place merge-classification suite exists under any name. What is in-place-specific: `InPlaceContinuationSlotLookupFailure_UsesFallbackWithoutError` (an in-place marker whose slot lookup misses degrades to the v0.9 terminal-kind classifier at Verbose, never the Site B-1 `Error`), the three `AppendRelations` self-skip cells that stop an in-place commit writing an `old == new` row or superseding one chain member with another, and `Bug585InPlaceContinuationRestoreTests` for the restore-side target swap and fork reconciliation. The seal verdicts the bullet describes — Orbiting seals, stashed-Orbiting/Landed seal (override-precedes-stashed), SubOrbital stays open, `recordingAction:*` closes, Destroyed stays retryable — are exercised only through the FORK path in `SupersedeCommitTests` and the shared classifier in `UnfinishedFlightClassifierTests`. Since both paths reach the same `ClassifyMergeStateOrThrow`, the verdicts are behaviour-covered, but no test drives them with an in-place marker.
- `SupersedeCommitTests` (`HasReFlySessionStructuralMutation_*` cells) — the gate's own coverage, not a `...StructuralMutationGateTests`: UT cutoff from `rp.UT` with the `marker.InvokedUT` fallback when the RP is missing from the scenario, and a pre-rewind breakup that stays undetected; the `PreSessionBranchPointIds` baseline filter (a pre-existing post-RP BP excluded, a new one still detected alongside it, a legacy marker without the baseline skipping the gate); lineage-set scope (same-chain tail detected, unrelated same-tree sibling and a different tree not detected, a standalone null-chain provisional detected by id alone); and the `BranchPointType` matrix (non-structural types not detected).
- `UnfinishedFlightSealHandlerTests` / `CollapseSealMergeStateRegressionTests` — Seal flips the slot's effective tip `MergeState` `CommittedProvisional -> Immutable`; `SupersedeStateVersion` bumps; idempotent on an already-`Immutable` tip; the first-commit guard (committed-tree membership OR supersede-fork identity) never re-opens a sealed tip on a later re-commit. `RewindPointReaper.IsReapEligible` matrix: every slot's tip `Immutable` → reap; any tip `CommittedProvisional` (open) → no-reap; any tip `NotCommitted` (recorder running) → no-reap. `reaperImpact=stillBlocked` vs `willReap` log values.
- `RewindPointAuthorTests` — the focus-slot cells live in the author's general class, not a `...FocusSlotIndexTests`: `RewindPointAuthor.Begin` sets `FocusSlotIndex` to the slot whose recording resolves to the active vessel pid (and logs `focusSlot=`), sets `-1` when the active vessel matches no slot, and `ResolveFocusSlotIndex` returns the FIRST slot with a warning when two slots share the active pid.
- Split across three real classes; no `InvocationLinearizationTests` exists. `AtomicMarkerWriteTests` — the marker write and the recording it creates are stamped together on both branches: the in-place branch forks a fresh provisional and stamps `SupersedeTargetId` on marker and fork, the pid-mismatch branch stamps the prior tip on marker and placeholder, and `Phase1And2_NoOnSaveBetween` / `Phase1And2_SameSyncBlock_NoInterleaving` pin that no `OnSave` can observe a half-written pair. `SessionSuppressedSubtreeTests` — the wrapper contract: `ComputeSessionSuppressedSubtree(null)` and a null-origin marker both return empty, the wrapper hands out a defensive copy (poisoning the first result does not leak into the second), and `InternalClosure_OriginRootMatchesPublicWrapper` is the closure-equivalence refactor guard. `SupersedeCommitTests` — linear chain extension: `AppendRelations_ChainExtension_RootsAtSupersedeTargetPriorTip` roots the new row at the prior tip rather than the origin (so a 3-link chain resolves the origin forward to the newest attempt), and `ChainExtendsThroughCrashedReFly` guards the commit-state / ERS-visibility half — the crashed provisional stays visible and the origin drops out.
- `SupersedeCommitTests.HybridStarAndLinearGraph_ResolvesDominantTipAndAllSlotTrails` — the §7.72 hybrid topology test, a single cell rather than a `HybridSupersedeGraphTests` class: a legacy star (`probeOrig` superseded by both `probeReFly1` and `probeReFly2`) plus a linear extension onto `probeReFly1`; `EffectiveRecordingId` resolves the origin to the chain tip, and `TryResolveRewindPointForRecording` finds the same RP and slot from both the new tip and the orphaned star branch.
- `UnfinishedFlightStashHandlerTests` / `PendingStashLifecycleTests` / `RecordingsTableUIStashRewindTests` — Stash resolver matrix: spawnable stable terminals (`Landed`, `Splashed`, `Orbiting`, `SubOrbital`) accepted; `Recovered` / `Docked` / `Boarded` / Destroyed rejected; `ScienceEarning`-bearing recordings rejected; already-reaped RPs rejected (no resurrection).

Section 2.12 (designed 2026-10-19, to be written with the implementation; red-first on main where marked):

- **Red-first:** a facility upgrade paid by the replaced flight's contract reward reads a negative balance after the merge (section 7.79); a tech node paid by the replaced flight's lab science is never unlocked; a first-time milestone of the replaced flight is lost; a contract failure of the replaced flight is erased; the Step 3b retirement of a later-recovered vessel survives a Discard and leaves a later upgrade unpaid (section 7.83); a deadline contract reads as expired during a session (section 7.84).
- **Value gate:** the loss / sealing classification pinned per `GameActionType` and per `ReputationPenaltySource`, with a reflection cell that fails when an enum member is added without a class; every classifier arm, the Stash resolver, `CanInvoke` and Site B-1 refuse on a sealing row with `recordingAction:<Type>:<ActionId>`; a slot holding only loss rows stays open; pre-rewind rows on a spanning origin do not seal.
- **Loss swap:** variant A when the re-fly loses no more reputation than the original at every UT, variant B otherwise; variant B's tombstones carry the re-fly as `RetiringRecordingId` and are removed by the attempt-row prune; the journal round-trips `lossSwapVariant` and a crash-recovered Tombstone step writes the same set.
- **Dominance check:** runs before `CommitPendingTree` from the pure row-build step and refuses with no state touched (section 7.81); the build step's rows equal the rows `FileRecordingCommitRows` then files (amounts and UTs; tags may move with the optimizer split); a re-fly that kills a kerbal a later committed recording flies is refused only when stock respawn would not return him in time; a property cell walks generated ledgers and asserts that no admitted merge lowers any pool at any UT or makes an affordable spend unaffordable.
- **Session isolation:** each switch entry point refuses a target outside the allowed set and allows the re-flown vessel and its EVA crew; awards are tagged to the causing vessel's recording; every discard path and Retry leave `Ledger.Actions` equal to the pre-invoke list apart from untagged session rows, the discard of an attempt resumed after its merge included (the science-node example from the design review is a red-first cell: the quickload undoes the node bought after the save, the discard removes the attempt's science, and science never goes below zero); the strip holds back every vessel a committed recording uses after the rewind UT.
- **Migration:** the load sweep concludes an open value-holding slot and logs it; a slot holding only losses is untouched.
- **Harness lane:** the booster case (upper stage to orbit, booster crashes in the background, re-fly it to a landing, merge) stays green and every pool after the merge is at least its value before; a second lane where the replaced flight's contract reward paid for a later facility upgrade proves the slot is never offered.

13.2 In-game tests (`Source/Parsek/InGameTests/RuntimeTests.cs`, Ctrl+Shift+T)

- `CaptureRPOnStaging` — asserts both PID maps populated, quicksave exists in `Parsek/RewindPoints/`.
- `SavePathRootThenMove` — no stray file left in save root.
- `WarpZeroedDuringSave` — warp drops to zero during save.
- `InvokeRPStripAndActivate` — strip counts + active-vessel PID unchanged.
- `MergeLandedReFlyCreatesImmutableSupersedeTest` — relation in list, provisional Immutable, RP reaps.
- `MergeCrashedReFlyCreatesCPSupersedeTest` — provisional `CommittedProvisional`, is Unfinished Flight, RP does not reap; re-invoke path exercises chain extension on next merge.
- `ContractTombstonesAcrossSupersedeTest` - in a live Re-Fly session, the merge's retire set holds no contract row (section 2.12, Rule 1) and no contract accepted after the session clock is Active in stock (Rule 3).
- `GhostSuppressionDuringReFlyTest` — no ghost rendering for supersede-target subtree.
- `KerbalRecoveryOnSupersedeTest` — kerbal returns active; reservation re-derived.
- `KerbalDualResidenceCarveOutTest` — live re-fly crew exempt from reservation lock.
- `UnfinishedFlightsRenderingAndNoHideTest` — UI group renders correctly and cannot be hidden.
- `F5MidReFlyResumeTest` — atomic phase 1+2 means marker validates; spare set survives.
- `MergeInterruptionRecoveryTest` — inject exception mid-merge; verify finisher behavior.
- `JournalFinisherMarkerPresentVariantTest` — marker-present variant of the finisher.
- `TreeDiscardRemovesSupersedesAndTombstonesTest` — tree-discard purge covers all scoped collections.
- `RPReapOnLastSlotImmutableTest` — RP reaps when the last slot becomes Immutable.
- `PartPersistentIdStabilityTest` — critical precondition probe: `Part.persistentId` is stable across the save/load cycle (required for `RootPartPidMap` fallback to work).

The v0.9.1 stable-leaf extension was specified as the eight in-game tests below. **NONE of them was built under those names** (grep-verified against `Source/` on 2026-08-05: zero hits for all eight). What was actually built is recorded against each, so this list reads as a record rather than as a plan. Three were built as harness lanes instead of in-game tests, two were already covered under other names, and three are NOT built and carry a named blocker.

The harness lanes referenced below live in `harness/scenarios/`; their armed save-structure gates follow the S4.1/CL-3 three-run discipline (a report-only reading run, then arming, then a negative control that is watched to fail and reverted).

- `MunMothershipFourProbeDeployTest` — **BUILT AS `GS-2-orbital-probe-deploy`** (harness lane, `[expectations.rewind]` ARMED: `rewindPoints = { min = 1 }`; reading run `2026-08-05_0853`, armed `_0856`, negative control `_0858`). The canonical S1 invariants are all carried: a real staging split authors a RewindPoint with a resolved `FocusSlotIndex`, the non-focus Orbiting child promotes to `CommittedProvisional` and appears in Unfinished Flights, the focus slot stays `Immutable`, and the RP survives. **Deviation, recorded honestly:** N = 1 deployed child rather than 4. The invariants are identical modulo slot count — nothing in `ApplyRewindProvisionalMergeStates` or `IsReapEligible` is cardinality-sensitive — but the 4-probe *fan-out* itself is not exercised. Per-slot Fly / Seal semantics are covered separately by the in-game `Rewind` category (below), not by this lane.
- `AutoParachuteBoosterFocusUpperStageTest` ("Critical regression guard") — **BUILT AS `GS-1-auto-chute-booster`** (harness lane, `[expectations.rewind]` ARMED: `rewindPoints = { max = 0 }`; reading run `2026-08-05_0824`, armed `_0832`, negative control `_0836`). A routine two-stage flight: the booster is non-focus and Landed, the focus upper stage is Landed, both resolve `Immutable`, and **the RP REAPS** — asserted as `rewindPoints = { max = 0 }` in the produced save plus the reaper's own log tokens. **Deviation, and it is a physics constraint rather than a shortcut:** the named test's "upper stage focus + **Orbiting**" is UNREACHABLE in stock. A chuting booster left unloaded below ~25 km is deleted by stock KSP, so a focus stage that actually reaches orbit cannot coexist with a booster that lands — that shape is S2 (crashed), and `GS-1` flight 2 measured exactly it when the two halves failed to separate and collided. The Orbiting-focus-gate direction is carried by `GS-2`'s focus-slot measurement instead. **Between `GS-1` and `GS-2` both branches of `RewindPointReaper.IsReapEligible` are now armed** — every slot tip `Immutable` (reap) and at least one tip `CommittedProvisional` (no reap).
- `InvertedFocusBoosterUpperStageOrbitingTest` — **BUILT AS `GS-2-orbital-probe-deploy`**, which is exactly this shape: focus stays on the crewed upper stage, the probe-cored half is released Orbiting and non-focus, it IS in Unfinished Flights, and the RP stays alive (the armed `rewindPoints = { min = 1 }` floor).
- `StrandedEvaKerbalRescueTest` — **RETIRED 2026-09-27** (§1.5: no EVA Re-Fly exists to test). Former note, kept for history: **NOT BUILT.** Blocker: the harness cannot address a specific RewindPoint slot from a mission, because the live RewindPoint id and its slot count are not in the `RecordingState` seam payload. The additive fix is already proposed and scoped in `docs/dev/todo-and-known-bugs.md` ("Rewind-in-flight", proposed fix (b): extend `TestCommandRecordingState.BuildPayload` — additive, pure, xUnit-coverable, read-only in an AnyScene verb). Automatable once that lands.
- `BreakupSurvivorMatrixTest` — **NOT BUILT AS A FLIGHT**, deliberately: a deterministic mid-air structural breakup is flake-bait, and `GS-1` flight 2 is the evidence (its unplanned breakup was a collision that no spec could have scheduled). The classifier matrix it describes is UNIT-covered — crashed → `CommittedProvisional`, Landed → `Immutable`, Orbiting-on-the-focus-slot → `Immutable` — across `UnfinishedFlightClassifierTests` (including the `reason=crashed` assertion), `TreeCommitTests`, `UnfinishedFlightsMembershipTests` and `CollapseSealMergeStateRegressionTests` (the only caller of the `ApplyRewindProvisionalMergeStatesForTesting` seam). **Still uncovered:** the per-RP leaf gate for a survivor that keeps the SAME recording id through the breakup. No test asserts that case.
- `CrossTreeDockDuringStableLeafReFlyTest` — **NOT BUILT.** Blocker: an RP quicksave's slot vessels are `CreateCopy()` clones of the host save's donor craft with only identity fields re-stamped (`StampVesselIdentity` touches no coordinate), so a re-fly always begins at the donor's position — it cannot reach a foreign orbiting station. This is the same limitation `CL-3-refly-crew-tombstone` records as "remedy (b) not taken". Named follow-up; needs a fixture whose RP quicksave carries a genuinely orbital slot vessel.
- `ReflyStableTerminalAutoSealTest` — **ALREADY COVERED UNDER OTHER NAMES.** The in-game cells are `MergeNonFocusReFlyToOrbitImmutableTest`, `MergeReFlyToSubOrbitalKeepsSlotOpenTest`, `MergeAndSealReFlyClosesSlotTest` and `MergeReFlyStructuralMutationAutoSealsTest`, all in the `Rewind` category and all driven live by `R7a-rewind-session-absent` / `R7c-rewind-spacecenter`. The manual-Stash variant is unit-covered by `UnfinishedFlightStashHandlerTests`, `PendingStashLifecycleTests` and `RecordingsTableUIStashRewindTests`; the supersede side by `SupersedeCommitTests` / `SupersedeCommitTombstoneTests`.
- `SealAndNoUnsealTest` — **ALREADY COVERED UNDER OTHER NAMES:** `UnfinishedFlightSealHandlerTests` and `CollapseSealMergeStateRegressionTests` (the unit half — a sealed `Immutable` tip is never re-demoted by promotion), plus the seal-dialog and stable-leaf group-seal cells in the `Rewind` category driven by `R7c-rewind-spacecenter`.

Question raised by `GS-3-switch-nudge-deployed`, RESOLVED AS AN IMPLEMENTATION CHANGE (2026-08-05). A no-input stock switch onto a deployed stable leaf caused the classifier to evaluate the SWITCH SEGMENT instead of the slot's origin recording — the segment's `VesselSwitchContinuation` branch point matches no RewindPoint, so it was rejected `reason=noMatchingRP` while the origin recording (the one the RP slot points at) was never evaluated at all. No promotion followed, both slots resolved `Immutable`, and the RP reaped: the leaf was dropped from Unfinished Flights after a single no-input click. Measured as a controlled A/B — `GS-3` run `2026-08-05_0903` against `GS-2` run `2026-08-05_0853`, one fixture, one mission, one changed step, `rewindPoints` 0 vs 1 in the produced saves. This contradicted S17 of `docs/dev/research/extending-rewind-to-stable-leaves.md` §8 ("probe still terminal Orbiting after the nudge. UF ... v1 over-includes nudged probes; player handles via Seal"), where over-inclusion is the deliberate bias and the player is meant to close the row themselves.

S17 stands and the code was fixed, not the intent. Reading the produced save closed the last gap in the forensics: the origin recording carries **no `terminalState` key at all** — a BG-member switch consumption closed it without classifying a terminal, so it failed `IsUnfinishedFlightCandidateShape` before `TryQualify` ever ran, which is why no `[UnfinishedFlights]` line named it. The fix is two-part: (1) the BG-member switch-consumption route stamps a classified terminal on the recording it closes, reusing the same

finalization-cache-then-live-classify pair every other background close uses; (2) both UF gates resolve the terminal through `VesselSwitchContinuation` branch points via `EffectiveState.ResolveTerminalRecordingAcrossSwitchContinuations` — a switch bp is an observation boundary on the SAME vessel, not a consumption, while every other branch-point type still stops the walk so the `downstreamBp` reject keeps its original meaning. Tracked in `docs/dev/todo-and-known-bugs.md` under `GS3-NUDGE-DROPS-UNFINISHED-FLIGHT`; the harness spec re-flew green post-fix (run `2026-08-05_1132`), its window inverted to `rewindPoints = {min 1}`, and it is now ARMED as the fix's regression guard (armed confirm `_1142`, negative control `_1147`).

13.3 Grep-audit CI gate

- `scripts/grep-audit-ers-els.ps1` scans every source file outside the allowlist for raw `RecordingStore.CommittedRecordings` or `Ledger.Actions` reads.
- `scripts/ers-els-audit-allowlist.txt` enumerates the 35 permitted exemptions with rationale comments. Each exemption has an inline `[ERS-exempt]` comment at the call site.
- `GrepAuditTests.GrepAudit_AllRawAccessIsAllowlisted` fails the build on any un-allowlisted raw read.

13.4 Crash-recovery reviewer sign-off

The pre-impl spec's v0.5 sign-off condition was "crash recovery matrix covers every window." `MergeCrashRecoveryMatrixTests` asserts per-window state invariants for the crash-point cases; `MergeJournalOrchestratorTests` exercises the rollback and forward-completion paths via `FaultInjectionPoint` + `DurableSaveForTesting` seams. Together they cover every crash window referenced in §6.12's failure-recovery matrix — the single `Begin`-rollback window plus the post-Begin forward-completion windows (`Split` / `Supersede` / `Tombstone` / `Finalize` / `Durable1Done` / `RpReap` / `MarkerCleared` / `Durable2Done` / `Complete`).

13.5 v0.9.1 migration tests

- Legacy save with already-reaped split RPs has empty UF group for those splits (no retroactive surfacing). **Fails if:** the upgrade tries to resurrect deleted RPs.
- Legacy save with live RPs whose Landed siblings are Immutable: row count unchanged from v0.9.0. **Fails if:** Landed terminals start qualifying.
- Legacy save with live RPs whose Orbiting non-focus siblings are Immutable: `ApplyRewindProvisionalMergeStates` does NOT re-promote. **Fails if:** the `noFocusSignal` short-circuit doesn't fire on legacy RPs.
- Legacy save with a live EVA RP whose kerbal Landed: no row surfaces on either side (the load-time EVA reap is deleted, section 7.68). **Fails if:** an EVA kerbal or the vessel side of an EVA RP surfaces in UF.
- Post-upgrade fresh deploy: spawn a multi-controllable split, leave probes orbiting, commit. Verify the new RP has `FocusSlotIndex >= 0`, the probes promote to CP, and the row appears in UF. **Fails if:** the motivating case stops working for new RPs.

13.6 Log-assertion tests (v0.9.1)

- `[UnfinishedFlights] Verbose` reasons for each predicate gate appear with the right structured values. **Fails if:** the diagnostic catalog drifts from the predicate.
- `[UnfinishedFlights] Info: Sealed ... reaperImpact=...` log appears with the correct impact computation.
- `[Supersede] Error: Site B-1 slot lookup failed ...` log appears in the (intentionally-induced) test scenario where the §6.22 prerequisite is broken.
- `[Supersede] Info: provisional=... focusSlot=N focusSlotOverride=M` includes both fields so the source of the focus decision is greppable.
- Shared-classifier diagnostic test: assert that Site B logs the same classifier reason Site A would have produced on the same `(Recording, ChildSlot, RewindPoint)` triple, while separately asserting Site B's stricter merge result. **Fails if:** the diagnostic predicate drifts or the merge-close policy regresses.

Appendix A — Gameplay Scenarios

A.1 Booster recovery (the motivating case)

1. Player launches an AB stack: upper stage B (crewed capsule) on top of booster A (probe core + parachute, intended for recovery).
2. Staging: the decoupler fires. A and B split. Both controllable → RP-1 is captured (one-frame-deferred quicksave under `Parsek/RewindPoints/`). PidSlotMap records B and A by Vessel.persistentId; RootPartPidMap records their root-part PIDs.

3. Player flies B to orbit. A is BG-recorded; the player gets no chance to deploy the chute in time and A crashes into the ocean.
4. Player returns to Space Center. Merge dialog → Merge to Timeline. Tree commits: ABC Immutable, B Immutable (Orbited), A CommittedProvisional (Crashed, terminal kind classifies as Crashed). RP-1 survives the scene load as a normal staging RP (`SessionProvisional=true`, `CreatingSessionId=null`) and is promoted to persistent when the tree commits. Recordings tab shows the Unfinished Flights virtual group (`STASH`) containing A with `Fly` + `Seal` buttons.
5. Player opens the Recordings tab, finds A under Unfinished Flights, clicks **Fly**. The `Confirm: Re-Fly` dialog names the split UT and the supersede/recalculation advisory. Player clicks **Fly**.
6. Scene reloads into FLIGHT at the staging moment. A is the active vessel (live physics); B has been stripped and now plays back as a ghost from its committed recording. ABC plays back as a ghost up to the split moment, then terminates. Player's career state matches what it was immediately before Rewind (B's orbit entry + all milestones / contracts are intact).
7. Player deploys A's parachute and lands it safely on the launch pad. Recorder finalizes A' with TerminalKind=Landed.
8. Merge dialog → Merge to Timeline. `MergeJournalOrchestrator.RunMerge` drives through the eleven phases. A → hidden (superseded); A' → Immutable + effective slot-1 representative. This example writes no tombstones because A has no reviewed recording-scoped career action rows to retire. RP-1 reap-eligible (both slots Immutable) → quicksave deleted, scenario entry removed, BranchPoint back-reference cleared.
9. Unfinished Flights is now empty. The timeline shows B's orbit entry AND A's safe landing (the sealed slot points at `A'` via the supersede relation); both play back on any subsequent rewind-to-launch.

A.2 EVA re-fly (retired 2026-09-27)

This example walked a kerbal who slipped off a ladder being re-flown from an EVA Rewind Point. Under the 2026-09-27 ruling (§1.5) an EVA authors no Rewind Point, so the example no longer exists: the player recovers an EVA mistake with F9 (quickload) or by discarding the flight before it is merged.

A.3 Docking merge + re-fly

1. Player launches a tanker to rendezvous with a station. Lander undocks from the station; both controllable → RP captured.
2. Player flies the tanker. Lander is BG-recorded. Due to an autopilot bug the lander de-orbits and crashes.
3. Merge. Lander CommittedProvisional Crashed → Unfinished Flight. Tanker Immutable.
4. Player rewinds. Lander is live; tanker plays back as a ghost; station's own tree is unrelated and stays physical (or ghost, depending on its own chain state).
5. Player manually flies the lander back up and docks with the station. Recording terminates at dock; `TerminalKind = Docked` → maps to Landed kind → Immutable merge.
6. Supersede `{Lander → Lander'}`. Station's tree unchanged. RP reaps if the tanker slot is also Immutable.

A.4 Revert-during-re-fly (3-option dialog)

1. Player is in the middle of an A.1-style booster re-fly. Rocket barely controllable after re-entry.
2. Player hits Esc → Revert to Launch. `RevertInterceptor.Prefix` sees the active marker → spawns the 3-option dialog.
3. **Retry from Rewind Point**: dialog dismisses. `RetryHandler` clears the marker, generates a fresh session id, re-invokes `RewindInvoker.StartInvoke(rp, slot)`. Scene reloads back to split moment with a new provisional A''. The old A' is now a zombie for the next load-time sweep (or for the current-session load cycle's sweep if another save happens first).
4. **Discard Re-fly**: dialog dismisses. `DiscardReFlyHandler` removes the provisional A' from `CommittedRecordings`, promotes the origin RP to persistent (`SessionProvisional=false`, `CreatingSessionId=null`) so `LoadTimeSweep` does not reap it, clears the marker + journal, bumps the supersede state version, then stages the origin RP's quicksave through the save-root copy path and calls `GamePersistence.LoadGame` + `HighLogic.LoadScene(GameScenes.SPACECENTER)`. The player lands in the Space Center at the RP's UT. The Unfinished Flights entry is still present (the origin crash recording remains visible as an `Immutable` legacy crash or `CommittedProvisional` current crash, and its RP was promoted, so `IsUnfinishedFlight` continues to resolve true). Career state is unchanged. The tree's other committed supersede relations, tombstones, and Rewind Points are preserved — other re-fly attempts already merged into the tree are unaffected.
5. **Continue Flying**: dialog dismisses; nothing happens. Player keeps trying.

The same 3-option dialog appears when the player picks Revert-to-VAB (or Revert-to-SPH for a spaceplane). Retry's behaviour is identical (RP quicksave is FLIGHT-scoped — Retry lands back at the split moment in FLIGHT, not in the editor). Discard Re-fly in that case pre-sets `EditorDriver.StartupBehaviour = EditorDriver.StartupBehaviours.START_CLEAN` and `EditorDriver.editorFacility = facility` before `HighLogic.LoadScene(GameScenes.EDITOR)` so the scene swap lands in the VAB or SPH as originally clicked, with a fresh editor against the just-loaded career.

A.5 Crash-quit-resume mid-re-fly

1. Player is 30 seconds into a re-fly. Home power fails. KSP dies without saving.
2. Player reopens KSP, loads the save. `ParsekScenario.OnLoad` loads the scenario including the `REFLY_SESSION_MARKER` that was on disk from the last F5 quicksave (if one was taken during the session) or from the last auto-save.
3. `MergeJournalOrchestrator.RunFinisher` runs first; no journal present (the player wasn't in the middle of merging), returns false.
4. `RewindInvoker.ConsumePostLoad` runs; no `RewindInvokeContext.Pending`, returns.
5. `LoadTimeSweep.Run` runs. `MarkerValidator.Validate` checks the six durable fields against the current scenario state. If the provisional recording from the session is still in `CommittedRecordings` with `MergeState == NotCommitted` AND the origin recording + RP + tree all resolve AND `InvokedUT` is in the past → marker valid. Spare set = {provisional, RP}. No discard.
6. `RewindPointReaper.ReapOrphanedRPs` runs. The RP is in the spare set and session-provisional; nothing to reap.
7. Player picks up where they left off. The live scene may need them to recover physics state (KSP's own auto-save quirks), but the Parsek-side re-fly session is intact.

Alternative: if the scenario state drifted (e.g. the provisional was corrupted on disk), `MarkerValidator` fails on the first mismatching field, emits `[ReFlySession] Warn: Marker invalid field=<f>; clearing`, clears the marker, and the nested-cleanup pass discards the provisional + session-provisional RPs. The player reverts to the pre-invocation state, with Unfinished Flights still showing the original retired sibling (because its CommittedProvisional stays). They can click Rewind again.

End of Rewind to Separation design doc.